

**POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY**



ASSIGNMENT 06 REPORT

**INFORMATION SYSTEM ANALYSIS
AND DESIGN**

Multimodal Search System for E-Commerce

Requirements, UML Architecture, Python Implementation
and Experimental Evaluation

Instructor: **Tran Dinh Que**

Student name: **Vu Dinh Hieu**

Student ID: **B23DCCE036**

Class: **E23TTNT02**

October 2026

Abstract

This report connects requirements analysis, three-layer architecture, native UML modeling and an executable search prototype. Text queries, simulated voice transcripts and image files are converted into a common query contract, processed by candidate retrieval and filtering, and ordered by a separate ranking service. The implementation uses a validated catalogue of 12 products, an 88-dimensional handcrafted image descriptor, cosine similarity and normalized score fusion. Twelve primary queries pass their declared top-1 relevance criterion; two extension cases and five robustness cases pass, while two challenge cases expose unsupported synonym and negation behavior. The report explains the complete Python implementation, records the actual demonstration outputs, preserves the approved UML and screenshots, and includes every project Python source file in full. The results demonstrate architectural consistency on a small synthetic dataset, rather than production readiness or general visual understanding.

The report follows the assignment's eleven-section structure. Complete implementation listings, dataset fixtures and evidence inventories are provided in the appendices.

Contents

1	Introduction	5
2	Problem Description	5
2.1	Customer problem and system boundary	5
2.2	Input modalities and outputs	5
2.3	Interpreting the assignment consistently	6
3	Requirements Analysis	6
3.1	Functional requirements	6
3.2	Non-functional requirements	7
3.3	Data and validation requirements	8
3.4	Requirement traceability	8
4	Use Case Model	8
4.1	Native model and relationship choices	8
4.2	UC-01: Search Product	10
4.3	UC-02: Search by Keyword	10
4.4	UC-03: Search by Voice	10
4.5	UC-04: Search by Image	10
4.6	UC-05: Search Order	10
4.7	UC-06: View Product	10
4.8	UC-07: View Order	11
5	Three-Layer Architecture	11
5.1	Layer responsibilities	11
5.2	Physical mapping and dependency injection	13
5.3	Why retrieval and ranking are separate	13
5.4	Storage and replacement boundaries	13
6	UML Sequence Diagram	14
6.1	Voice-search collaboration	14
6.2	Message-to-code explanation	16
7	Python Implementation	16
7.1	Repository structure and execution environment	16
7.2	Composition root and command dispatch	17
7.3	Presentation adapters and result rendering	17
7.4	Product and order repositories	17
7.5	Query normalization and common representation	17
7.6	Simulated speech service	19
7.7	Image encoding and index integrity	19
7.8	Candidate retrieval and hard filtering	19
7.9	Ranking and response contract	20
7.10	Dataset preparation and reproducible index construction	20
7.11	Running and reproducing the program	20
8	Multimodal Search Method	21
8.1	Lexical representation and text score	21
8.2	Handcrafted image representation	21
8.3	Cosine similarity and index scoring	21
8.4	Candidate union, filtering and fusion	21
8.5	Worked examples and complexity	22
9	Experimental Results	22
9.1	Evaluation design and ground truth	22
9.2	Aggregate results	22
9.3	Complete query-level evaluation	23
9.4	Demonstration outputs and score visibility	24
9.5	Tests, coverage and verification scope	27

9.6	Latency measurement	27
9.7	Observed incorrect results	27
10	Discussion	28
10.1	What the prototype demonstrates	28
10.2	Validity limits of the experiments	28
10.3	Simulation and omitted production features	28
10.4	Design choices and trade-offs	28
10.5	Future improvements in a controlled order	29
11	Conclusion	29
A	Assignment Coverage and Deliverables	31
A.1	Submission inventory and reproducibility	32
B	Complete Python Source Code	33
B.1	Entry point and presentation	33
B.2	Application and intelligence	37
B.3	Data access and vector index	43
B.4	Dataset, evaluation, report and delivery utilities	46
B.5	Desktop evidence and Visual Paradigm launcher	72
B.6	Complete automated test suite	75
C	Dataset and Approved Image Catalogue	83
D	Complete Data Fixtures and Configuration	87
D.1	data/products.json	87
D.2	data/orders.json	89
D.3	data/dataset_metadata.json	89
D.4	evaluation/queries.json	90
D.5	requirements.txt	90
D.6	requirements-dev.txt	90
D.7	requirements-automation.txt	91
D.8	pyproject.toml	91

List of Figures

1	Approved native Use Case Diagram exported from Visual Paradigm. The default VP colors and orthogonal connectors are preserved.	9
2	Actual Visual Paradigm screenshot of the editable use-case model. The export in Figure 1 provides the larger reading view.	9
3	Approved component/package architecture: three native packages, sixteen components and fifteen dependencies. Arrows run from a client to the component it uses.	12
4	Actual VP screenshot of the three-layer architecture. Native package ownership and dependencies remain editable in the delivered project.	12
5	Approved voice-search sequence, with seven participants and fourteen native messages. The note identifies simulated STT and the common downstream pipeline.	14
6	Actual Visual Paradigm screenshot of the saved voice sequence. Existing lifelines, message order, activations and self-call geometry are preserved.	15
7	Actual Python result-viewer screenshot for text input “black shoes”. It shows tokens, category filtering and scores read from a fresh CLI JSON response.	25
8	Actual Python viewer for the voice transcript “find black running shoes”. The displayed simulation label is part of the evidence.	25
9	Actual Python viewer for the black-shoe query image, using an 88D pixel descriptor and cosine similarity.	26
10	Actual Python viewer for text–image fusion, with separate text, image and final scores. Existing screenshot pixels are preserved.	26
11	Original synthetic catalogue illustrations, products 1–6. Images are preserved from the approved dataset.	84
12	Original synthetic catalogue illustrations, products 7–12. Images are preserved from the approved dataset.	85
13	All three readable transformed image-query fixtures. Brightness and resolution differ from their source illustration; these are derived synthetic samples, not independent photographs. .	86

1 Introduction

E-commerce customers describe the same shopping intent through different interfaces. A customer may type “black running shoes”, provide a spoken request or supply an image of a desired product. These inputs have different formats, but the application should offer a consistent retrieval and result-presentation process. Treating each input mode as a disconnected program would duplicate filtering, ranking and error handling, making later changes harder to maintain.

Assignment 06 asks for a small prototype that demonstrates the relationship between system design and executable code [1]. Its central principle is

$$\text{Different inputs} \rightarrow \text{Common query} \rightarrow \text{Retrieval} \rightarrow \text{Ranking} \rightarrow \text{Results.} \quad (1)$$

This project follows that principle through a Python dictionary contract and injected services. It implements keyword search, voice search using supplied transcripts, pixel-based image similarity, a text–image fusion extension, price/category filters, product details and customer-scoped order lookup.

The contributions are concrete: twelve validated product records and their illustrations; three editable UML diagrams created in Visual Paradigm; a Presentation–Application–Data implementation; observable score components and deterministic ranking; repeatable demonstrations; and an evaluation that retains incorrect results. The full source appendix makes the relationship between these deliverables inspectable.

This report uses the assignment’s recommended structure in the prescribed order. The native UML exports and application screenshots are preserved as evidence of the implemented system. Appendix listings and data tables allow the design claims to be checked against the complete executable source.

2 Problem Description

2.1 Customer problem and system boundary

The primary actor is a **Customer**. The customer wants to discover suitable products, inspect their details and locate an existing order. The system receives an input through the presentation layer, constructs a query, retrieves eligible candidates and returns a ranked list containing product metadata and score explanations. Product details and order lookup are distinct read-only interactions.

The prototype has no checkout, payment, account registration, inventory updates or external deployment. Product prices are educational sample values in USD. Brand names are sample metadata and do not imply affiliation. Order data contain fictional customer identifiers; there are no real customer records. The CLI customer identifier is a simulated context, not an authenticated identity.

2.2 Input modalities and outputs

Mode	Actual input	Processing	Output
Text	Keyword or limited English natural-language string	Normalize, tokenize, extract filters, retrieve and rank	Ranked products and text/final scores
Voice	Already transcribed text string	Simulated SpeechService, then the same textual pipeline	Voice provenance and explicit simulation label
Image	Readable image file path	Encode pixels, compare to the vector index, filter and rank	Image/final scores and encoder metadata
Multimodal	Text and one image	Common query, candidate union, hard filtering and score fusion	Text/image/final score components

Mode	Actual input	Processing	Output
Order	Exact order ID and customer context	Scoped lookup through OrderService	Order status, date and total, or not-found
Product	Positive product ID	Application-mediated metadata lookup	Product details or not-found

The main search response has three keys: `query`, `processing` and `results`. Processing metadata expose tokens, filters, candidate counts, weights, encoder identity and `top-k`. A valid no-match query returns an empty list; it is not classified as an execution error.

2.3 Interpreting the assignment consistently

The document cover identifies Assignment 06, although several examples retain “Assignment 05” and an older directory name. This implementation consistently uses Assignment 06. The earlier demonstration section mentions at least two modes, whereas Task 6 explicitly requires text, voice and image; all three are demonstrated. Order search is named in the required use-case model and presented as an implementation extension, so a minimal exact-ID lookup is included. Multimodal fusion is an extension and also appears in the rubric; it is implemented.

Artificial vectors are allowed by the assignment. This prototype instead computes deterministic features from image pixels, while clearly identifying them as handcrafted descriptors. The brief’s example “where is my latest order?” illustrates broader system intent; natural-language latest-order reasoning is not implemented. OrderItem management is not implemented because the selected order extension only requires lookup/status display.

3 Requirements Analysis

3.1 Functional requirements

The requirements below are implementation commitments derived from Tasks 1–6 and the extensions. A requirement’s evidence is the observable behavior and its implementation, rather than an assumed rubric score.

ID	Requirement	Acceptance criterion
FR-01	Search products by text	Nonempty keywords are normalized; relevant candidates receive lexical scores; unknown words may produce an empty list.
FR-02	Search by simulated voice	The transcript passes through SpeechService, retains voice provenance and uses the same retrieval/ranking behavior as equivalent text.
FR-03	Search by image	Image bytes are decoded and encoded; cosine similarity against matching product vectors determines visual scores.
FR-04	Use a common query representation	Every product-search modality provides the same schema for filters, <code>top-k</code> , weights and provenance.
FR-05	Rank and display results	Sort final score descending, then product ID ascending; apply <code>top-k</code> only after ranking and show score components.

ID	Requirement	Acceptance criterion
FR-06	View product details	An existing ID returns name, category, color, price, stock, description and image path; absent IDs return not-found.
FR-07	Search/view an order	OrderService requires customer context; a wrong owner or unknown ID returns no order.
FR-08	Provide a valid catalogue	At least ten unique products have valid fields and real readable image files; index IDs, dimension and fingerprint match the catalogue.
FR-09	Handle invalid input and no-match	Reject empty text, invalid parameters, corrupt/missing images and invalid vectors clearly; valid empty results remain successful execution.
FR-10	Fuse text and image	Use both modalities in one query, preserve candidate union and publish normalized component/final scores.
FR-11	Support price/category filters	“Under/below” are exclusive and “at most” inclusive; conflicting explicit/inferred categories are rejected.
FR-12	Provide reproducible evidence	Preserve CLI commands, demo outputs, screenshots, frozen relevance judgments, result rows and derived metrics.

FR-01–06, FR-08 and FR-09 refine the baseline. FR-07 and FR-10–11 cover implemented extensions. FR-12 supports demonstration and evaluation. Twelve functional requirements exceed the assignment’s minimum of five.

3.2 Non-functional requirements

ID	Quality objective	Evidence and limitation
NFR-01	Maintainability	Presentation has no direct Data imports; application logic and repositories are injected. Architecture tests check import boundaries.
NFR-02	Reproducibility	Deterministic fixture generation, explicit encoder/version, stable tie-break and frozen ground truth. SHA-256 identifies inputs.
NFR-03	Robustness	Boundary tests cover NaN/infinity, malformed schemas, stale index, image failures, zero vectors and invalid configuration.
NFR-04	Usability	Help, readable output, pure JSON option, explicit errors and honest simulation labels are available.
NFR-05	Performance	A proposed local target is under one second per query after loading this small dataset; recorded times are much lower, but do not measure concurrent or production load.
NFR-06	Portability	Paths are resolved with pathlib and the CLI works from a different current directory. The runtime requires Python and pinned packages, not VP.
NFR-07	Data isolation	Application lookup enforces the supplied customer context. No authenticated user identity or production authorization is claimed.
NFR-08	Offline operation	Runtime search requires no network, API key, microphone or remote model. VP and XeLaTeX are separate modeling/report tools.

These objectives deliberately distinguish demonstrated qualities from future production requirements. The one-second target is a project choice, not a hard constraint stated by the assignment.

3.3 Data and validation requirements

Products require positive unique integer IDs, nonempty textual metadata, a supported category, finite non-negative price, nonnegative integer stock and an image inside the project root. Boolean values do not count as numeric IDs, price or stock. ProductRepository returns deep copies so callers cannot mutate stored records accidentally.

QueryService accepts top- k from 1 to 100 and textual input from 1 to 2000 characters. Filter values and weights must be finite. A query image vector must contain exactly 88 finite numbers. ImageService rejects images larger than twenty million pixels. The index checks encoder metadata, product-ID coverage, vector shape and a fingerprint of catalogue content plus image bytes. Changing catalogue data or image content requires rebuilding the index.

3.4 Requirement traceability

FR-01/02/04/11 map to QueryService and SearchUI; FR-03 maps to ImageService and VectorIndex; FR-05 maps to RankingService and SearchResultView; FR-06 maps to SearchService.view_product; FR-07 maps to OrderService and OrderRepository; FR-08/09 map to repositories and boundary validation; FR-10 maps to multimodal query construction, retrieval and ranking; FR-12 maps to demonstration and evaluation utilities. Detailed task-to-evidence mapping appears in Appendix A and docs/traceability.md.

4 Use Case Model

4.1 Native model and relationship choices

The approved native diagram contains Customer and all seven named use cases. The system boundary is “Multimodal E-Commerce Search System”. Customer associates with Search Product, Search Order, View Product and View Order. Search by Keyword, Search by Voice and Search by Image specialize Search Product; the hollow generalization triangle points toward that general use case.

This specialization represents alternative input choices. An include relationship that forces all three modalities on every search would be incorrect. Viewing a detail is a separate customer goal; it is not automatically required whenever a search executes. Fusion is an extension of Search Product and does not replace any required use case.

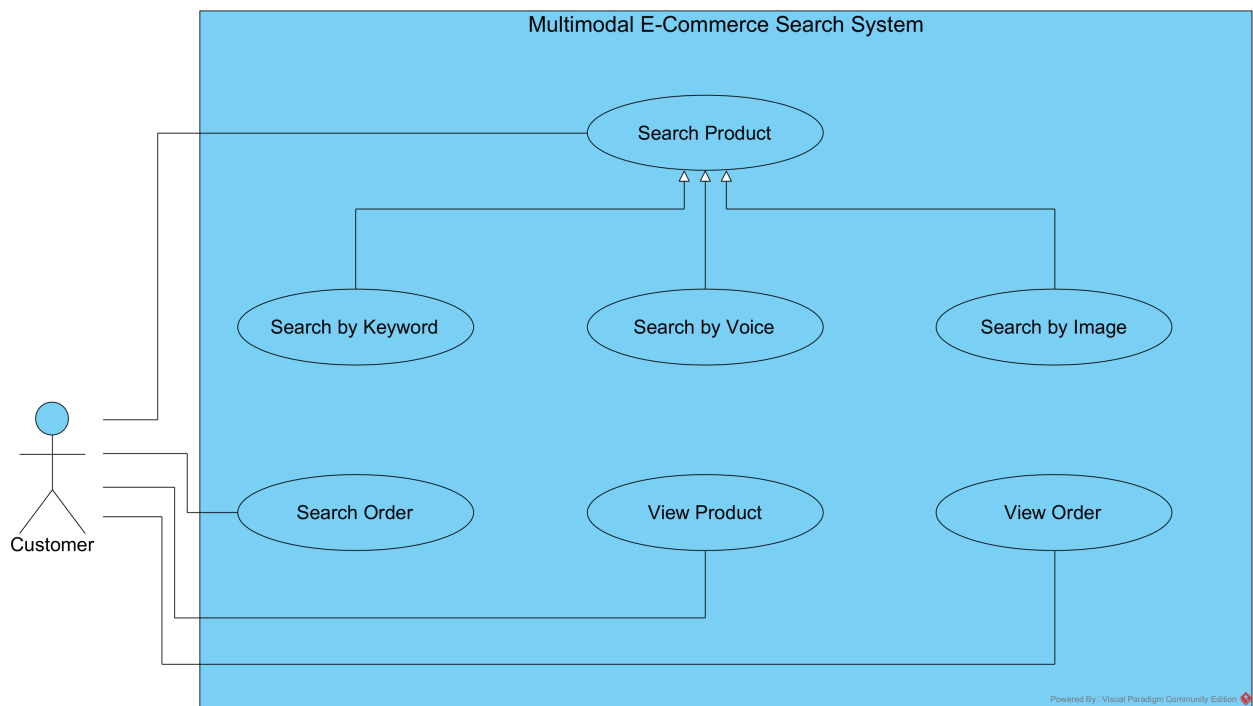


Figure 1: Approved native Use Case Diagram exported from Visual Paradigm. The default VP colors and orthogonal connectors are preserved.

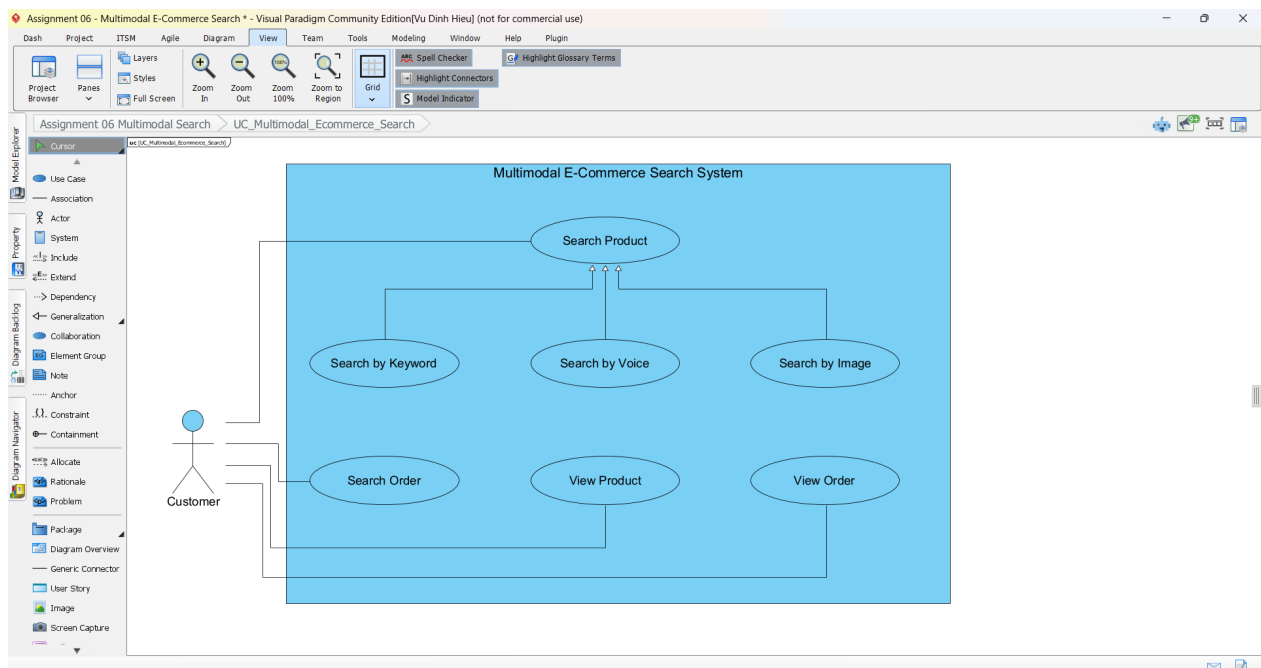


Figure 2: Actual Visual Paradigm screenshot of the editable use-case model. The export in Figure 1 provides the larger reading view.

4.2 UC-01: Search Product

Goal and preconditions. Customer discovers matching products. Valid repositories and index are loaded and at least one supported modality is supplied. **Main flow.** SearchUI receives input, the application constructs a common query, SearchService retrieves and filters candidates, RankingService orders them, and the result view displays metadata and scores. **Alternatives.** Invalid input is rejected; a valid no-match query returns an empty list; a stale index produces a rebuild message. **Postcondition.** A result envelope is returned without modifying catalogue or index records.

4.3 UC-02: Search by Keyword

Trigger. `--mode text --query "black shoes"`. **Main flow.** Normalize case/whitespace, extract supported price filters, tokenize with stopwords and aliases, infer category, retrieve products with at least one matching term, apply hard filters and rank. **Alternatives.** Unknown vocabulary returns empty results; filter-only queries examine the whole catalogue; conflicting categories or invalid top- k are rejected. **Postcondition.** Text and final scores, matched terms and filter decisions are observable.

4.4 UC-03: Search by Voice

Precondition. Input is already a transcript, not a microphone recording. **Main flow.** SearchUI calls SpeechService.transcribe, constructs a voice query and invokes the same search/ranking pipeline. **Alternatives.** Empty/non-string/overlong transcripts fail validation; valid transcripts without a lexical match return empty results. **Postcondition.** Query type remains voice, and processing marks simulated STT. Equivalent transcript/text configurations yield equivalent product rankings.

4.5 UC-04: Search by Image

Precondition. The supplied file is readable and compatible encoder/index metadata are available. **Main flow.** ImageService reads pixels and encodes them; QueryService validates the common image query; VectorIndex calculates cosine scores; SearchService associates scores with metadata; RankingService orders and truncates. **Alternatives.** Missing/corrupt/oversized images and invalid vectors are rejected; zero-norm vectors receive zero similarities. **Postcondition.** Image and final scores plus encoder identity are returned. There is no semantic-confidence threshold.

4.6 UC-05: Search Order

Trigger. Exact order ID with supplied customer context, for example O001/C001. **Main flow.** SearchUI calls OrderService, which validates identifiers and requests scoped repository lookup. **Alternatives.** Unknown orders and orders belonging to another context both return no order. **Postcondition.** A matching order or a clear not-found response is returned. This path does not use product ranking.

4.7 UC-06: View Product

Trigger. A positive product ID, supplied by CLI or selected from search results. **Main flow.** SearchUI calls the Application service; ProductRepository retrieves a copy; ResultView shows details. **Alternatives.** Invalid identifiers are errors and absent valid identifiers are not-found. **Postcondition.** Metadata are exposed without direct presentation-layer database access or mutation.

4.8 UC-07: View Order

Precondition. Order ID and customer context are supplied. **Main flow.** The scoped lookup path returns status, date and total for display. Search Order and View Order share the same application operation in this minimal CLI. **Alternatives.** Invalid context fails validation; wrong-owner and absent records produce the same not-found response. **Postcondition.** An order in the supplied context is visible, without modifying it. Supplying another customer ID is still possible; authenticated identity remains future work.

5 Three-Layer Architecture

5.1 Layer responsibilities

The design separates interaction, application behavior and persisted information:

$$\text{Presentation} \rightarrow \text{Application / Intelligence} \rightarrow \text{Data}. \quad (2)$$

The Presentation layer contains SearchUI, logical VoiceInput and ImageUpload adapters, and SearchResultView. It collects arguments and formats results; it does not load JSON, compute cosine similarity or rank candidates. VoiceInput and ImageUpload are implemented as SearchUI methods rather than additional standalone Python classes. These are logical UML responsibilities, not claims that every component has its own file.

The Application layer contains QueryService, SpeechService, ImageService, SearchService, RankingService and the OrderService extension. It validates requests, converts modalities, performs retrieval/filtering, ranks and scopes order lookup. The Data layer contains ProductRepository, OrderRepository and VectorIndex, backed by product/order JSON, the image folder and a persisted vector file.

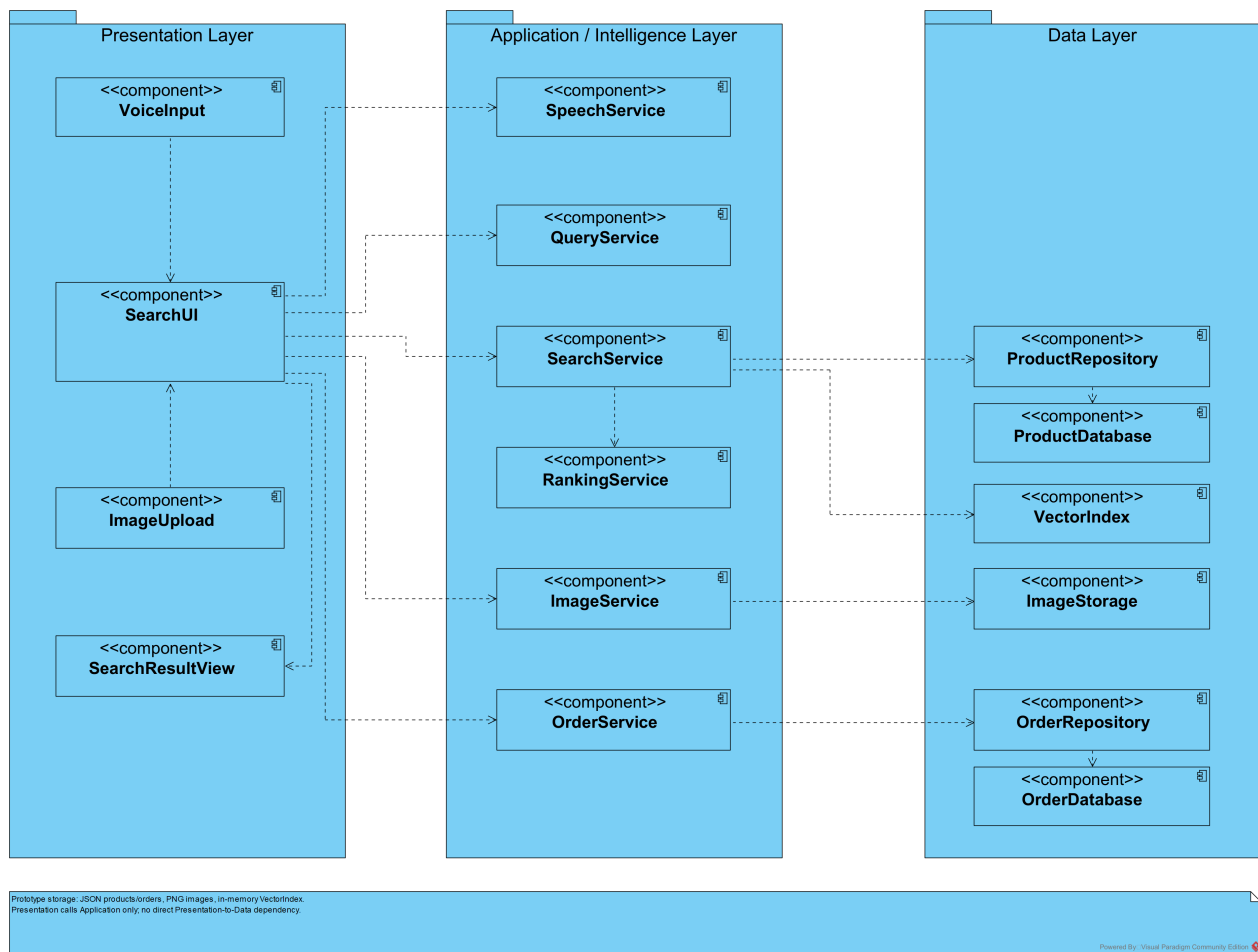


Figure 3: Approved component/package architecture: three native packages, sixteen components and fifteen dependencies. Arrows run from a client to the component it uses.

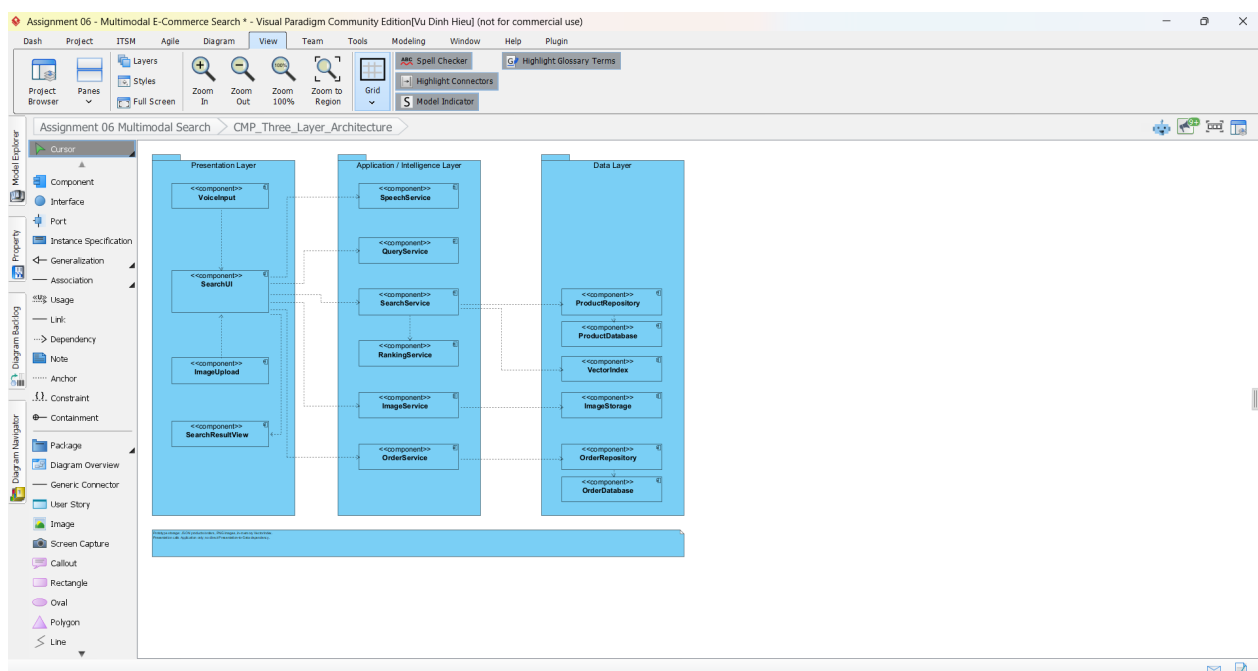


Figure 4: Actual VP screenshot of the three-layer architecture. Native package ownership and dependencies remain editable in the delivered project.

5.2 Physical mapping and dependency injection

Logical component	Physical implementation	Responsibility
SearchUI / input adapters	presentation/search_ui.py	Route text, transcript, image, fusion, order and detail requests through injected services.
SearchResultView	presentation/result_view.py	Readable CLI output without repository access or score recalculation.
QueryService	application/query_service.py	Common query, tokens, filters, weights and bounds.
SpeechService	application/speech_service.py	Validate and return the supplied transcript.
ImageService	application/image_service.py	Read image files and calculate the 88D descriptor.
SearchService	application/search_service.py	Candidate generation, filtering, response metadata and product details.
RankingService	application/ranking_service.py	Final scores, stable ordering and top- k .
OrderService	application/order_service.py	Require customer context and perform exact order lookup.
ProductRepository	data/product_repository.py	Validated product JSON and copy-safe access.
OrderRepository	data/order_repository.py	Validated order JSON and scoped lookup.
VectorIndex	data/vector_index.py	Index validation, fingerprint and unsorted cosine scores.
Product / Order storage	data/products.json, data/orders.json	Local JSON storage, not relational databases.
ImageStorage	data/images/, data/queries/	Original illustrations and reproducible image inputs.

main.py is the composition root. It creates repositories and index, injects them into application services, and injects those services into SearchUI. Importing all layers at this wiring point is intentional. Presentation modules themselves do not import Data, and Data modules do not import upper layers. This rule is checked by the architecture test.

5.3 Why retrieval and ranking are separate

Retrieval answers which records are candidates and gathers component scores. Ranking answers their final order and the selected top- k . SearchService._retrieve returns unsorted candidates and pre-filter statistics; its public retrieve_candidates wrapper exposes the list. SearchService.search uses the private helper to retain statistics before delegating to RankingService.rank. The separation prevents early truncation from discarding a product that becomes strong under a combined score.

5.4 Storage and replacement boundaries

ProductDatabase and OrderDatabase in UML denote storage responsibilities implemented with JSON. VectorIndex is an in-memory exact scan loaded from a JSON vector file; it is not a vector database or approximate-nearest-neighbor engine. ImageStorage is the local filesystem. SpeechService and ImageService provide re-

placement points for real ASR or learned encoders, but replacing a model also requires rebuilding its index, maintaining contracts and reevaluating relevance.

6 UML Sequence Diagram

6.1 Voice-search collaboration

The approved voice sequence contains the seven required participants: Customer, SearchUI, SpeechService, QueryService, SearchService, ProductRepository and RankingService. Calls and dashed returns communicate the order of collaboration. Activation bars indicate active execution, and self-calls identify internal retrieval/filtering work.

The conceptual brief sends audio to speech recognition. The actual prototype sends transcriptInput because simulated recognition is explicitly permitted. This distinction is recorded in the diagram note and user-facing output. No audio recording or recognition accuracy is claimed.

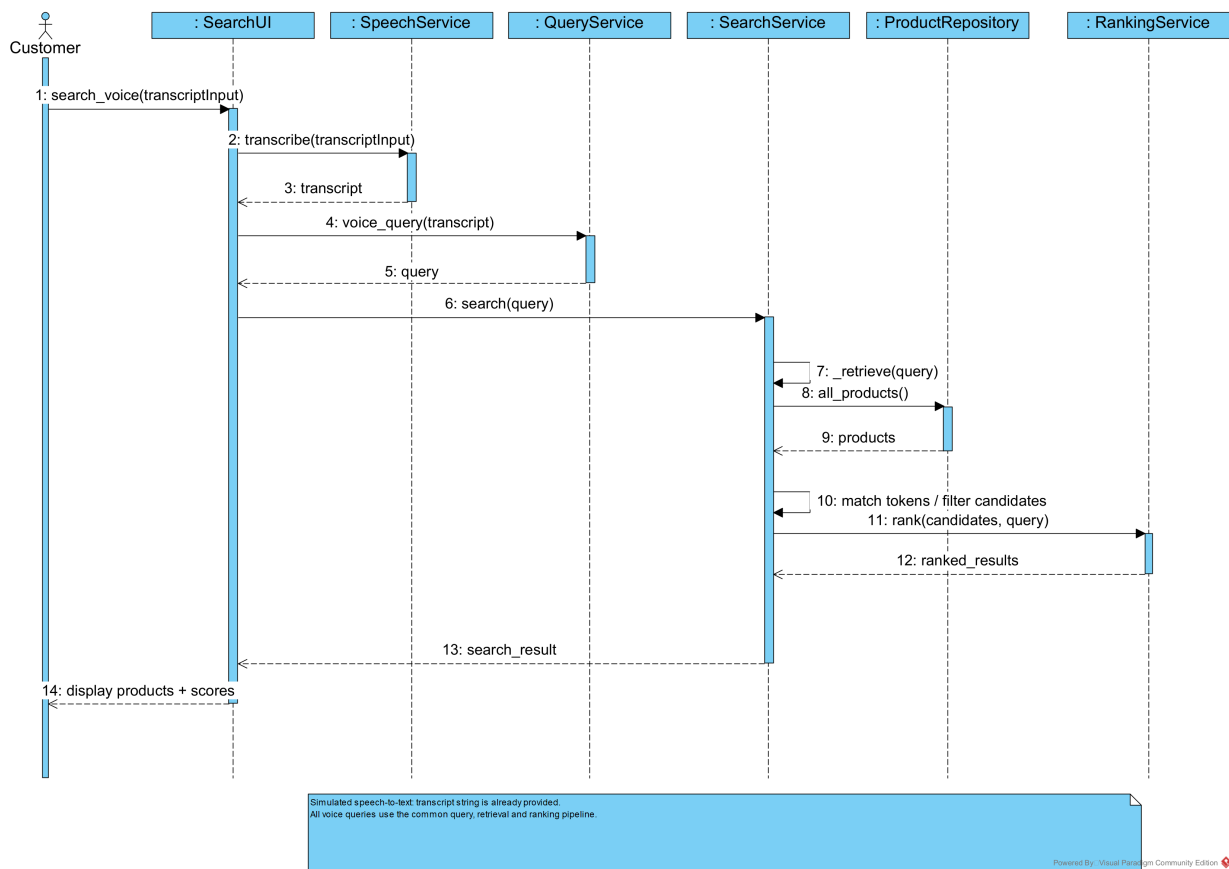


Figure 5: Approved voice-search sequence, with seven participants and fourteen native messages. The note identifies simulated STT and the common downstream pipeline.

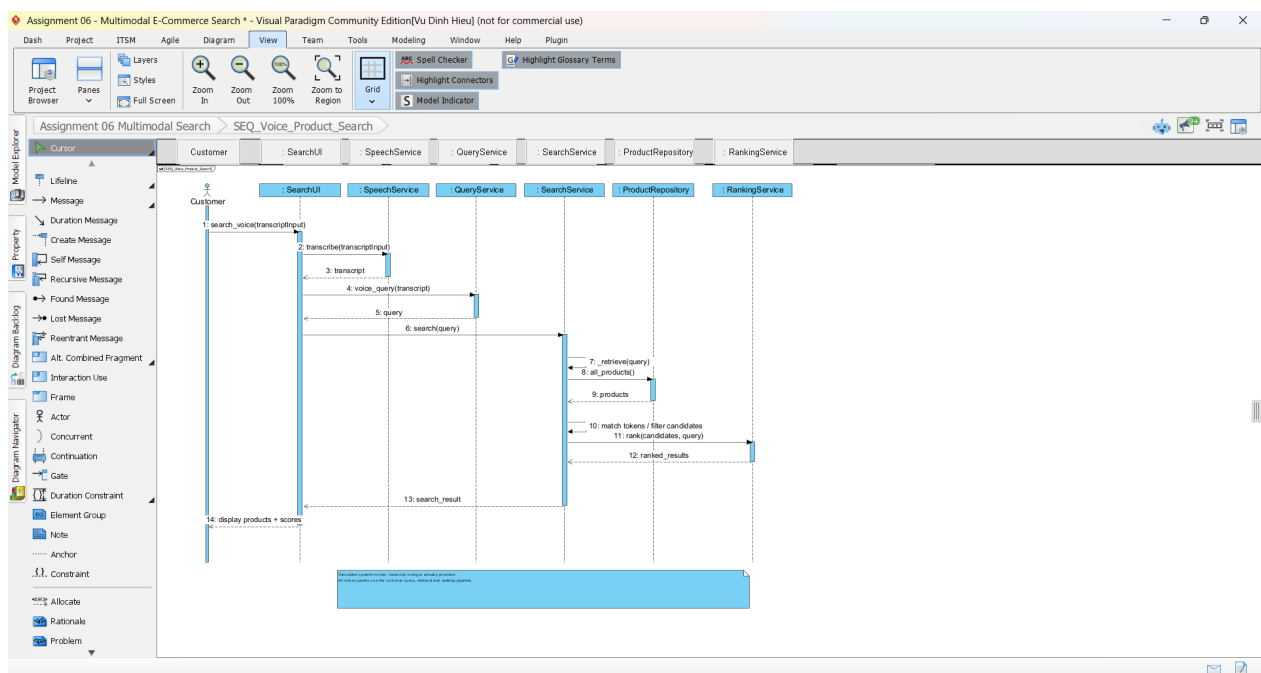


Figure 6: Actual Visual Paradigm screenshot of the saved voice sequence. Existing lifelines, message order, activations and self-call geometry are preserved.

6.2 Message-to-code explanation

Step	Message / return	Meaning in the implementation
1	Customer to SearchUI: search_voice	CLI supplies an already transcribed request.
2	SearchUI to SpeechService: transcribe	Validate transcript type, emptiness and length.
3	SpeechService returns transcript	Return stripped text; no external recognizer runs.
4	SearchUI to QueryService: voice_query	Construct validated common query with voice provenance.
5	QueryService returns query	Return tokens, filters, top- k , weights and type.
6	SearchUI to SearchService: search	Orchestrate retrieval and ranking.
7	SearchService self-call: _retrieve	Prepare tokens, similarity map and candidate statistics.
8	SearchService to ProductRepository: all_products	Read validated product copies.
9	ProductRepository returns products	Supply catalogue records without exposing mutable originals.
10	SearchService self-call: match/filter	Calculate text matches and apply category/price constraints.
11	SearchService to RankingService: rank	Calculate final scores, sort with stable tie-break and select top- k .
12	RankingService returns ranked results	Supply product records with final scores and rank.
13	SearchService returns search result	Return the query/processing/results envelope.
14	SearchUI displays products and scores	CLI ResultView formats returned values; rendering does not rerank.

For an empty or invalid transcript, execution stops at validation and the CLI reports a clear error. A valid transcript with no lexical match reaches ranking with an empty list and returns normal empty results. No additional UML was created to represent these alternatives; their behavior is documented and tested.

7 Python Implementation

7.1 Repository structure and execution environment

Runtime verification uses Python 3.12.8 on Windows 11. The runtime requirements pin NumPy 2.5.3 and Pillow 12.3.0. Test/report dependencies are separate from runtime packages; desktop automation dependencies are only needed for optional evidence/model tooling. XeLaTeX is an external report-building tool and is not required to run search.

Listing 1: Project structure and responsibility boundaries

```

1 assignment6/
2   main.py                # CLI and composition root
3   presentation/          # input routing and result rendering
4   application/           # queries, modalities, retrieval, ranking, orders
5   data/                  # repositories, vector index and fixtures
6     products.json orders.json embeddings.json
7     images/ queries/
8   tests/                 # acceptance and boundary regression tests
9   evaluation/queries.json # fixed relevance and robustness judgments
10  scripts/               # preparation, evaluation, reporting and delivery
11  models/                # approved native VP project
12  docs/report.tex         # complete English report source

```

13	artifacts/	# outputs, screenshots, metrics and report
----	------------	--

Appendix B contains every Python file in these source directories, including initialization files, utilities, desktop launchers and tests. Listings are generated from exact file snapshots, not rewritten examples or selected fragments.

7.2 Composition root and command dispatch

`build_services` in `main.py` creates `ProductRepository`, a matching `VectorIndex`, `QueryService`, `SpeechService`, `ImageService`, `SearchService` with `RankingService`, and `OrderService` with `OrderRepository`. It returns `SearchUI` with these dependencies injected. Services are built once per CLI invocation, before query execution.

`Argparse` selects text, voice, image, multimodal, order or product mode. No mode, or `--demo`, runs the complete five-interaction demonstration. Relative image paths are resolved against the project root, so they do not depend on the caller's current directory. `--json` emits only a JSON response to stdout; `--output` writes UTF-8 JSON. `ValueError` and `OSError` produce a readable error on stderr and exit code 2. Valid no-match execution returns code 0.

7.3 Presentation adapters and result rendering

`SearchUI.search_text` creates a text query and delegates to search. `SearchUI.search_voice` first invokes the speech adapter and preserves the original transcript as provenance. Image and fusion methods call `ImageService.encode` before query construction. Order and product details route through application services. None of these methods loads storage directly.

`SearchResultView` formats an envelope into input, normalized tokens, filters, candidate counts, weights and ranked records. Product rows include ID, name, price, stock and component/final scores. A simulation line is added for voice. Detail and order responses are rendered as JSON. The separate screenshot helper uses a Python Tkinter viewer populated from a fresh CLI JSON result; it does not invent scores or act as a web interface.

7.4 Product and order repositories

`ProductRepository` loads the JSON catalogue and validates every required field. It verifies unique IDs, categories, finite numeric prices, integer stock and image containment. `all_products` returns deep copies and `get_by_id` selects a copied record. This prevents search or rendering from adding score/rank fields to the stored product itself.

`OrderRepository` validates list shape, required identifiers/date/status strings, distinct order IDs and finite non-negative totals. Its optional customer parameter is a low-level lookup capability; the public application path always requires a validated context. `OrderService` accepts IDs matching `O\d{3}` and `C\d{3}` and passes both to the repository. A response explicitly describes authentication as a simulated customer context. It is incorrect to equate this context filter with authenticated authorization.

7.5 Query normalization and common representation

Text is lowercased and trimmed. Price phrases are extracted before tokenization: “under” and “below” give exclusive upper limits, and “at most” gives an inclusive upper limit. Multiple limits of the same kind keep the minimum; if both kinds exist, both are enforced. A separate `--max-price` argument is exclusive.

Tokenization uses lowercase alphanumeric word boundaries, removes a small explicit stopword set and preserves unique tokens. Supported aliases include shoe to shoes, bags to bag, backpacks to backpack, and shirts

to shirt. Category inference detects shoes, bag/backpack, or shirt/jacket/clothing terms; category becomes a hard constraint. An explicit category conflicting with that inferred category is rejected. Unknown synonyms and negation are not interpreted.

Field	Role	Validation / behavior
type / raw_input	Modality and original input	Text, voice, image or multimodal; provenance remains visible.
query / tokens	Normalized lexical representation	Textual modes require 1–2000 characters; unique searchable terms or a supported filter.
embedding	Visual representation	Null for text/voice; exactly 88 finite numbers for image/fusion.
filters	Category and price constraints	Supported category; separate exclusive/inclusive bounds; finite nonnegative values.
top_k	Requested result count	Integer from 1 to 100; booleans are rejected.
weights	Text/image fusion policy	Two nonnegative finite values summing to one.
customer_id	Reserved customer context	Product query field may be null; orders use their separate validated contract.
encoder	Image-vector provenance	Name, version and dimension whenever an image vector is present.

Using one schema permits retrieval, filtering and ranking to operate without separate modality-specific result contracts. Orders intentionally retain their own response because an exact scoped lookup is not a product relevance ranking problem.

Listing 2: Actual common text-query object from the saved demonstration

```

1 {
2   "type": "text",
3   "raw_input": "black shoes",
4   "query": "black shoes",
5   "tokens": [
6     "black",
7     "shoes"
8   ],
9   "embedding": null,
10  "filters": {
11    "category": "shoes",
12    "max_price_exclusive": null,
13    "max_price_inclusive": null
14  },
15  "top_k": 5,
16  "weights": {
17    "text": 1.0,
18    "image": 0.0
19  },
20  "customer_id": null,
21  "encoder": null
22 }
```

Listing 3: Actual first ranked product record from the same demonstration

```

1 {
2   "product_id": 1,
3   "name": "Nike Running Shoes",
```

```
4  "brand": "Nike",
5  "category": "shoes",
6  "color": "black",
7  "price": 120,
8  "stock": 10,
9  "description": "Black running shoes for everyday training.",
10 "image_path": "data/images/p001.png",
11 "raw_text_score": 2,
12 "text_score": 1.0,
13 "image_score": 0.0,
14 "business_score": 0.0,
15 "matched_terms": [
16     "black",
17     "shoes"
18 ],
19 "final_score": 1.0,
20 "rank": 1
21 }
```

7.6 Simulated speech service

SpeechService.transcribe accepts a nonempty transcript string no longer than 2000 characters and returns stripped text. Non-string inputs, including raw audio bytes, are rejected. The class exposes `simulated=True`; the search response exposes `simulated_stt=True`. The implementation demonstrates the adapter location and downstream reuse, not speech recognition. A real ASR adapter would additionally require audio capture/decoding, model execution, failure policy and evaluation on independent speech data.

7.7 Image encoding and index integrity

ImageService opens the file through Pillow, applies EXIF orientation, converts to RGB and rejects oversized input. It creates a 24-dimensional RGB histogram and a 64-dimensional grayscale thumbnail, normalizes the blocks and their concatenation, and returns a numeric list. File errors, unrecognized formats and decompression-bomb failures become clear application validation errors.

VectorIndex loads encoder metadata and product vectors from `data/embeddings.json`. It validates the encoder identity/version/dimension, checks every vector, verifies that product IDs equal catalogue IDs and confirms a SHA-256 fingerprint over normalized catalogue JSON and original image bytes. Similarity scoring produces an ID-to-score mapping without sorting. Its exact algorithm and limitations are explained in Section 8.

7.8 Candidate retrieval and hard filtering

SearchService tokenizes the concatenation of product name, brand, category, color and description. It intersects product terms with query terms and records matched terms in sorted order. Text/voice requests with lexical terms require at least one match. A filter-only request examines the whole catalogue. Image and fusion requests consider every product represented by the matching index, including those with no lexical match.

Pre-filter candidates are counted first. Category equality, exclusive price and inclusive price constraints are then applied. Remaining candidates contain copied metadata and raw/normalized text, image and business scores. Business score is explicitly zero. The unsorted list is passed to the ranking service; no modality-specific top-*k* is selected before fusion.

7.9 Ranking and response contract

RankingService validates exactly the text/image weights, their types and sum. It copies candidate records, calculates final score, sorts by `(-final_score, product_id)`, applies top- k and attaches one-based rank. Copying prevents result construction from changing retrieval candidates or catalogue data. Equal scores are reproducible rather than dependent on filesystem or dictionary iteration order.

SearchService returns the common query, processing statistics and ranked products. Both candidate counts are visible, and the processing description states lexical OR matching, category inference, image candidate union and absence of business boosts. Scores are heuristic relevance signals, not confidence probabilities.

7.10 Dataset preparation and reproducible index construction

`scripts/prepare_dataset.py` produces original Pillow illustrations and metadata. Existing outputs are preserved unless `--force` explicitly requests regeneration. Query samples use fixed brightness 0.97 and resize to 224×224 on selected product illustrations. They differ in pixels from catalogue files but are derived from the same source, so they are not independent real-world photographs. One intentionally invalid file tests image error handling.

`scripts/build_index.py` computes product descriptors through the same ImageService used at query time and writes encoder metadata, vectors and a source fingerprint. Rebuilding is required when products, image bytes or encoder identity change. The fixed dataset contains 12 products and 3 orders; the catalogue and sample images appear in Appendix C.

7.11 Running and reproducing the program

Execute the commands below from the assignment root. Data/index are already supplied, so regeneration is optional when inputs have not changed.

Listing 4: Runtime setup and supported interactions

```

1 python -m venv .venv
2 .venv\Scripts\python.exe -m pip install -r requirements.txt
3 .venv\Scripts\python.exe main.py --help
4 .venv\Scripts\python.exe main.py --demo
5 .venv\Scripts\python.exe main.py --mode text --query "black shoes"
6 .venv\Scripts\python.exe main.py --mode voice --query "find black running shoes"
7 .venv\Scripts\python.exe main.py --mode image --image data/queries/black_shoe_query.png
8 .venv\Scripts\python.exe main.py --mode multimodal --query "black shoes" --image data/queries/
  black_shoe_query.png --text-weight 0.5
9 .venv\Scripts\python.exe main.py --mode text --query "find Nike shoes under 100 dollars"
10 .venv\Scripts\python.exe main.py --mode order --order-id 0001 --customer-id C001
11 .venv\Scripts\python.exe main.py --mode product --product-id 1

```

For machine-readable evidence add `--json`; for a saved artifact add `--output path.json`. Missing image arguments, invalid parameters and invalid data fail explicitly. A request such as “unobtainium” is a valid search with no results and returns normal execution status.

8 Multimodal Search Method

8.1 Lexical representation and text score

Let $T(q)$ be the set of unique query terms after normalization/aliases/stopwords, and $T(p)$ the token set derived from searchable product metadata. The raw lexical score and normalized text score are

$$s_{\text{raw}}(p, q) = |T(q) \cap T(p)|, \quad (3)$$

$$s_t(p, q) = \begin{cases} |T(q) \cap T(p)|/|T(q)|, & |T(q)| > 0, \\ 0, & |T(q)| = 0. \end{cases} \quad (4)$$

This is whole-word OR retrieval followed by hard filters, not semantic language understanding. A product matching either “black” or “shoes” is a preliminary lexical candidate, but inferred category shoes excludes non-shoe products. Duplicated query words do not inflate the score. Filter-only queries receive zero text score and use the deterministic ID tie-break among eligible products.

8.2 Handcrafted image representation

For an image I , let $h(I) \in \mathbb{R}^{24}$ be eight-bin histograms for each RGB channel after resizing to 128×128 , and $g(I) \in \mathbb{R}^{64}$ the flattened 8×8 grayscale thumbnail divided by 255. The implementation normalizes each block and then the concatenation:

$$\hat{h} = \frac{h}{\|h\|_2}, \quad \hat{g} = \begin{cases} g/\|g\|_2, & \|g\|_2 > 0, \\ g, & \text{otherwise}, \end{cases} \quad z(I) = \frac{[\hat{h}; \hat{g}]}{\|[\hat{h}; \hat{g}]\|_2} \in \mathbb{R}^{88}. \quad (5)$$

For nonzero blocks the final normalization gives each block equal squared-norm contribution. A completely black thumbnail remains zero, while the nonzero histogram still provides a valid descriptor. RGB distributions capture coarse appearance and the grayscale thumbnail captures simple spatial structure. The encoder identity is `rgbhist-gray-thumbnail`, version 1.0, dimension 88.

These vectors are derived from actual pixels rather than product IDs or filenames. Nevertheless, they are not learned embeddings, CLIP features or object recognition. Background, dominant color and layout can dominate similarity, so visually close descriptors need not mean semantically equivalent products.

8.3 Cosine similarity and index scoring

For query/product descriptors z_q and z_p ,

$$\cos(z_q, z_p) = \begin{cases} \frac{z_q^\top z_p}{\|z_q\|_2 \|z_p\|_2}, & \|z_q\|_2 \|z_p\|_2 > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

The helper rejects nonnumeric, nonfinite or incompatible vectors, clips the computed cosine to $[-1, 1]$ for numerical stability and handles zero norm as zero. Index scoring then uses $s_i = \max(0, \cos)$ so its fusion input is nonnegative. This clamp is an explicit policy, not evidence that cosine is a probability. Every indexed product is scored in an exact scan.

8.4 Candidate union, filtering and fusion

Let $C_t(q)$ be lexical candidates and $C_i(q)$ indexed image candidates. Multimodal retrieval uses

$$C(q) = \text{Filter}(C_t(q) \cup C_i(q)). \quad (7)$$

The current image index covers the whole catalogue, so fusion examines the full catalogue before hard filters. It does not intersect modalities or take the top few from each modality first. Final ranking is

$$S(p | q) = w_t s_t(p, q) + w_i s_i(p, q), \quad w_t, w_i \geq 0, \quad w_t + w_i = 1. \quad (8)$$

Text/voice use $(1, 0)$, image uses $(0, 1)$ and fusion defaults to $(0.5, 0.5)$. A valid fusion text-weight sets w_t and $w_i = 1 - w_t$. Business score remains zero: stock is displayed but gives no hidden promotion or exclusion. The brief’s general business-weight term is a future extension, not an implemented signal.

The implementation combines *scores*, not a text token vector and an image pixel vector. Those representations have incompatible spaces, so directly summing them would be unjustified. A voice transcript uses the textual score; a simultaneous separate third audio input is not implemented. Products are finally sorted by decreasing S and increasing ID, then top- k is applied.

8.5 Worked examples and complexity

For “black shoes”, tokens are black and shoes, and category shoes is inferred. Products 1 and 5 match both terms and obtain text score 1; other shoes matching only the category term receive 0.5. The score tie places product 1 before 5. For “find Nike shoes under 100 dollars”, stopwords and the price phrase are removed, category shoes and price < 100 are enforced, and the Nike budget shoes at USD 95 become the expected top result. A product at exactly USD 100 is excluded by “under” but may pass “at most”.

In fusion, a candidate with strong image similarity but no lexical overlap is retained until filters/ranking; a weight can emphasize either modality without redefining the candidate set. Appendix and demonstration tables report actual component scores rather than substituting rounded scores into the algorithm.

For N products and descriptor dimension $d = 88$, image scoring is an $O(Nd)$ exact scan. Text retrieval also scans product metadata; there is no inverted text index. Ranking M surviving candidates uses $O(M \log M)$ sorting followed by slicing. Loading services/index occurs before measured query time. These choices are appropriate for twelve products but do not demonstrate large-catalogue scalability.

9 Experimental Results

9.1 Evaluation design and ground truth

The evaluation runs the actual injected application pipeline against `evaluation/queries.json`. The frozen file declares relevance IDs and expected behavior before implementation/evaluation. Its SHA-256 is recorded in metrics. The report generator verifies matching case IDs and group summaries against the saved result rows rather than manufacturing a success table.

There are 21 cases: twelve primary retrieval cases, two extension cases, two challenge cases and five robustness cases. Primary cases are balanced across text, voice and image (four each). A product query succeeds when its top-ranked product belongs to that case’s declared relevant-ID set; order success requires the expected exact order; robustness uses explicit empty/error/zero-score/scoping criteria. An execution error is never counted as ordinary retrieval success.

$$\text{SuccessRate}(G) = \frac{\sum_{q \in G} \mathbf{1}[\text{criterion satisfied}]}{|G|}. \quad (9)$$

The twelve-case primary denominator is not the count of all twenty-one rows. Extension, challenge and robustness summaries are reported separately because they assess different behaviors.

9.2 Aggregate results

Evaluation group	Cases	Successful	Success rate
Primary	12	12	100.00%
Extension	2	2	100.00%
Challenge	2	0	0.00%
Robustness	5	5	100.00%

Primary modality	Cases	Successful	Top-1 success
Text	4	4	100.00%
Voice	4	4	100.00%
Image	4	4	100.00%

All twelve primary cases pass the stated top-1 criterion. Both extension cases pass: the fused product query and scoped order lookup. Both challenge cases fail, and their judgments remain unchanged. All five robustness cases meet their expected outcomes. The successful primary image cases include an exact duplicate sanity query and mild transformations of catalogue illustrations; this makes them useful implementation checks but weak evidence of generalization.

9.3 Complete query-level evaluation

The table below includes every saved case. “Relevant/expected” lists acceptable IDs or a behavior criterion. “Actual top-1” is a product ID, returned order, validation error or no result. For R04 the success criterion concerns zero scores even though deterministic ranking still selects an ID.

ID	Mode	Input	Relevant / expected	Actual top-1	Verdict
T01	text	black shoes	1, 5	1	PASS
T02	text	blue shoes	4	4	PASS
T03	text	black leather bag	3	3	PASS
T04	text	find Nike shoes under 100 dollars	5	5	PASS
V01	voice	find running shoes	1, 2, 5	1	PASS
V02	voice	show me a white shoe	2, 7	2	PASS
V03	voice	black leather bag	3	3	PASS
V04	voice	green backpack	12	12	PASS
I01	image	data/images/p001.png	1	1	PASS
I02	image	data/queries/black_shoe_query.png	1, 5	5	PASS
I03	image	data/queries/blue_shoe_query.png	4	4	PASS
I04	image	data/queries/brown_bag_query.png	8	8	PASS
F01	multimodal	black shoes	1, 5	5	PASS
O01	order	O001	O001	O001	PASS
C01	text	sneakers	1, 2, 4, 5, 6, 7	none	FAIL
C02	text	shoes not black	2, 4, 6, 7	1	FAIL
R01	text	unobtainium	empty results	none	PASS
R02	text	empty text	expected error	error	PASS
R03	image	data/queries/corrupt.png	expected error	error	PASS
R04	image	zero vector (88D)	zero scores	1	PASS
R05	order	O002	empty order	none	PASS

Image paths are fixture identifiers; the three readable transformed samples are shown in Appendix C. The intentionally corrupt fixture cannot be displayed as an image, because its purpose is to fail image decoding.

9.4 Demonstration outputs and score visibility

Task 6 requires text, simulated voice and image demonstrations with input, processing steps, returned products and ranking scores. The saved demo artifact contains all three, plus fusion and order. The tables below are generated from that artifact and include every returned product, not only the top result. Screenshots use $\text{top-}k = 3$ for legibility; the complete CLI demo uses $\text{top-}k = 5$, so the tables contain more rows without a contradiction.

9.4.1 Text demonstration output

Input: black shoes. Normalized query: black shoes. Tokens: black, shoes. Candidates before / after filtering: 9 / 6; top-k: 5. Category: shoes; text/image weights: 1.0 / 0.0.

Rank / ID	Product	Text	Image	Final
1 / 1	Nike Running Shoes	1.000000	0.000000	1.000000
2 / 5	Nike Budget Running Shoes	1.000000	0.000000	1.000000
3 / 2	Adidas Running Shoes	0.500000	0.000000	0.500000
4 / 4	Blue Sports Shoes	0.500000	0.000000	0.500000
5 / 6	Red Canvas Shoes	0.500000	0.000000	0.500000

9.4.2 Voice demonstration output

Input: find black running shoes. Normalized query: find black running shoes. Tokens: black, running, shoes. Candidates before / after filtering: 9 / 6; top-k: 5. Category: shoes; text/image weights: 1.0 / 0.0.

Rank / ID	Product	Text	Image	Final
1 / 1	Nike Running Shoes	1.000000	0.000000	1.000000
2 / 5	Nike Budget Running Shoes	1.000000	0.000000	1.000000
3 / 2	Adidas Running Shoes	0.666667	0.000000	0.666667
4 / 4	Blue Sports Shoes	0.333333	0.000000	0.333333
5 / 6	Red Canvas Shoes	0.333333	0.000000	0.333333

9.4.3 Image demonstration output

Input: data\queries\black_shoe_query.png. Normalized query: (image only). Tokens: none. Candidates before / after filtering: 12 / 12; top-k: 5. Category: none; text/image weights: 0.0 / 1.0.

Rank / ID	Product	Text	Image	Final
1 / 5	Nike Budget Running Shoes	0.000000	0.998089	0.998089
2 / 1	Nike Running Shoes	0.000000	0.998042	0.998042
3 / 6	Red Canvas Shoes	0.000000	0.990605	0.990605
4 / 4	Blue Sports Shoes	0.000000	0.988564	0.988564
5 / 2	Adidas Running Shoes	0.000000	0.979388	0.979388

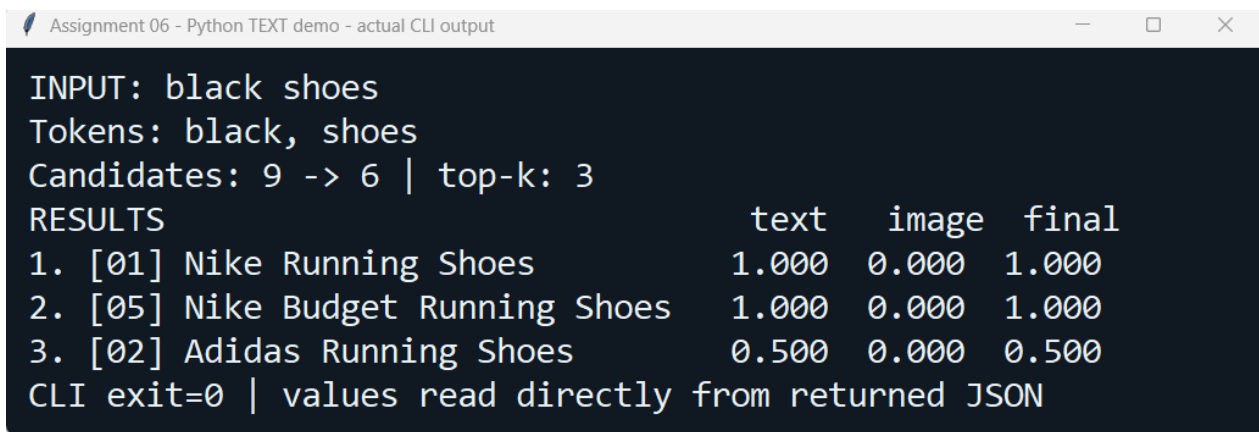
9.4.4 Multimodal demonstration output

Input: black shoes | data\queries\black_shoe_query.png. Normalized query: black shoes. Tokens: black, shoes. Candidates before / after filtering: 12 / 6; top-k: 5. Category: shoes; text/image weights: 0.5 / 0.5.

Rank / ID	Product	Text	Image	Final
1 / 5	Nike Budget Running Shoes	1.000000	0.998089	0.999044
2 / 1	Nike Running Shoes	1.000000	0.998042	0.999021
3 / 6	Red Canvas Shoes	0.500000	0.990605	0.745303
4 / 4	Blue Sports Shoes	0.500000	0.988564	0.744282
5 / 2	Adidas Running Shoes	0.500000	0.979388	0.739694

9.4.5 Order demonstration output

Order O001 belongs to C001; observed status: Shipped, total USD 150.0.

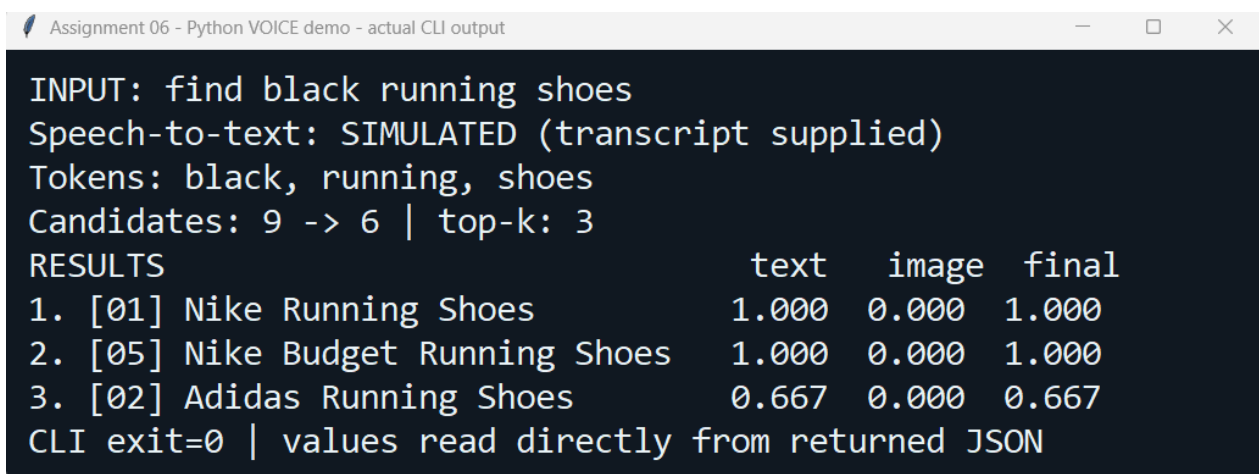


```

Assignment 06 - Python TEXT demo - actual CLI output
INPUT: black shoes
Tokens: black, shoes
Candidates: 9 -> 6 | top-k: 3
RESULTS
      text  image  final
1. [01] Nike Running Shoes      1.000  0.000  1.000
2. [05] Nike Budget Running Shoes 1.000  0.000  1.000
3. [02] Adidas Running Shoes    0.500  0.000  0.500
CLI exit=0 | values read directly from returned JSON

```

Figure 7: Actual Python result-viewer screenshot for text input “black shoes”. It shows tokens, category filtering and scores read from a fresh CLI JSON response.



```

Assignment 06 - Python VOICE demo - actual CLI output
INPUT: find black running shoes
Speech-to-text: SIMULATED (transcript supplied)
Tokens: black, running, shoes
Candidates: 9 -> 6 | top-k: 3
RESULTS
      text  image  final
1. [01] Nike Running Shoes      1.000  0.000  1.000
2. [05] Nike Budget Running Shoes 1.000  0.000  1.000
3. [02] Adidas Running Shoes    0.667  0.000  0.667
CLI exit=0 | values read directly from returned JSON

```

Figure 8: Actual Python viewer for the voice transcript “find black running shoes”. The displayed simulation label is part of the evidence.

```
Assignment 06 - Python IMAGE demo - actual CLI output

INPUT: data\queries\black_shoe_query.png
Encoder: pixels, 88D | similarity: cosine
Candidates: 12 -> 12 | top-k: 3
RESULTS
      text  image  final
1. [05] Nike Budget Running Shoes  0.000  0.998  0.998
2. [01] Nike Running Shoes         0.000  0.998  0.998
3. [06] Red Canvas Shoes           0.000  0.991  0.991
CLI exit=0 | values read directly from returned JSON
```

Figure 9: Actual Python viewer for the black-shoe query image, using an 88D pixel descriptor and cosine similarity.

```
Assignment 06 - Python MULTIMODAL demo - actual CLI output

INPUT: black shoes | data\queries\black_shoe_query.png
Tokens: black, shoes
Encoder: pixels, 88D | similarity: cosine
Candidates: 12 -> 6 | top-k: 3
RESULTS
      text  image  final
1. [05] Nike Budget Running Shoes  1.000  0.998  0.999
2. [01] Nike Running Shoes         1.000  0.998  0.999
3. [06] Red Canvas Shoes           0.500  0.991  0.745
CLI exit=0 | values read directly from returned JSON
```

Figure 10: Actual Python viewer for text–image fusion, with separate text, image and final scores. Existing screenshot pixels are preserved.

These screenshots were produced by `scripts/desktop/capture_demo.py`, which executes the real CLI, reads JSON and displays the returned values in Tkinter. They are program-running evidence, not terminal screenshots, not a production storefront and not fabricated UI mockups. Their raw payloads remain in `artifacts/screenshots/demo_*_payload.json`.

9.5 Tests, coverage and verification scope

The JUnit evidence records 78 passing test cases, with zero failures, errors or skips. Application/Data line coverage is 100.00% over 264 statements. This is line coverage for the specified packages; it is not branch coverage, coverage of all UI/scripts or a numerical guarantee of correctness.

Tests verify normalized whole-word matching, aliases, strict versus inclusive prices, filter-only requests, deterministic tie-breaks, retrieval/ranking separation, no mutation, simulated voice equivalence, descriptor dimension/path independence, cosine boundaries, corrupt/oversized images, schema errors, stale/invalid indexes, customer-scoped orders and CLI behavior from another working directory. Architecture tests inspect import boundaries; percentile testing checks the nearest-rank definition. Full test source is included in Appendix B.

Ruff, Pyright, Python compilation and dependency consistency evidence supplement tests. Native inventory and saved VP screenshots establish modeling evidence; source tests alone cannot establish native UML quality. Report compilation and visual review separately check presentation. ZIP validation/extraction addresses delivery rather than relevance quality.

9.6 Latency measurement

The preserved evaluation reports mean primary-query time 1.289 ms and nearest-rank p95 6.615 ms. With twelve samples, $\lceil 0.95 \cdot 12 \rceil = 12$, so p95 equals the maximum observed primary latency. Timing starts after `build_services`; dataset/index loading and process startup are excluded. Image encoding remains part of an image request's timed application call.

The environment is Python 3.12.8 on Windows 11. This is one small local run, without concurrent requests, cold-start analysis or repetitions for statistical confidence. Timing values are reproduced from the recorded experiment; compilation of this report does not rerun evaluation or overwrite its timing evidence.

9.7 Observed incorrect results

C01: “sneakers”. Relevant shoe IDs are declared in ground truth, but the tokenizer has no sneakers alias and no semantic text model. There are no lexical matches, so the query returns no products. The failure exposes vocabulary limits.

C02: “shoes not black”. The declared relevant set excludes black shoes. The lexical method treats “not” as another word without negation scope and rewards black/shoes overlap. It returns black product 1 first, giving a failed relevance judgment. The report retains this failure rather than modifying labels or calling the ranking semantically correct.

These two cases explain why 100% success on the primary set must not be generalized to unrestricted natural language. The relevant-ID judgments, output IDs, scores and processing metadata are all available in saved results.

10 Discussion

10.1 What the prototype demonstrates

The implementation demonstrates architecture before model sophistication. A common query lets services reuse filtering and ranking across text, transcripts and image inputs. Logical UML components have identifiable code responsibilities, while repositories protect storage boundaries. Explicit candidate/component statistics make score behavior inspectable, and deterministic ranking makes demonstrations repeatable.

The project exceeds the minimal text/voice example by including actual pixel encoding, index integrity checks, multimodal score fusion, filter semantics, exact scoped orders and extensive invalid-input handling. These additions remain within the same three-layer architecture. They are implementation coverage statements, not claimed assessment marks.

10.2 Validity limits of the experiments

The dataset has twelve educational illustrations and a small set of authored queries. The transformed image queries originate from catalogue illustrations, while one primary image case is an exact duplicate. Differences in pixels do not make these independent photographic test data. A strong score therefore measures consistency on familiar distributions, not robustness to unseen cameras, lighting, backgrounds, viewpoints or product identities.

Top-1 success against a declared relevant set does not measure full ranking quality or calibrated confidence. The relevant IDs are limited author judgments; precision/recall over a commercial catalogue are not established. Primary, challenge and robustness cases are deliberately separated rather than collapsed into one flattering number. The two failures are informative evidence, and the frozen criterion should remain fixed during future comparison.

10.3 Simulation and omitted production features

Voice receives an existing transcript; there is no microphone, speech model or word-error-rate experiment. Image descriptors are handcrafted, not semantic learned representations. Storage is JSON/filesystem with exact scans, not transactional SQL or a vector database. Stock is metadata, not an active ranking signal. An optional web UI, independent real photographs, real ASR, learned image/text embeddings and stock-aware ranking remain unimplemented advanced work.

The application checks order ownership against a caller-supplied context, but a caller can supply another customer's identifier. No login, signed session, authenticated authorization, encryption policy, payment processing or concurrent inventory consistency is present. Products/orders are fixtures. Data separation is useful engineering, but it must not be marketed as production privacy or security.

10.4 Design choices and trade-offs

JSON and exact scans keep the implementation offline, easy to inspect and reproducible. Copy-safe repository access and strict index fingerprints reduce silent errors at the cost of extra copying and required regeneration. Simple lexical OR matching provides clear scores but cannot resolve semantic synonyms or negation. Histogram/thumbnail features are inexpensive but are sensitive to visual appearance rather than object meaning.

Hard category/price filters precede ranking so ineligible products cannot become top results through similarity alone. Equal-weight default fusion is a declared heuristic, not an empirically optimized parameter. Candidate union avoids discarding useful modality evidence, but an image index spanning the whole catalogue also broadens the candidate pool. These trade-offs should be reevaluated using held-out data before changing weights or claiming quality gains.

10.5 Future improvements in a controlled order

First, expand relevance judgments and query samples independently of implementation outputs, including real photographs and difficult linguistic cases. Second, compare a semantic text encoder and learned image encoder against the transparent baseline with matching retrieval/filter contracts. Third, introduce a real ASR adapter and measure transcription errors separately from retrieval. Fourth, evaluate scalable indexes, concurrency and cold starts when catalogue size warrants them. Finally, authenticated customer context and production data policies are prerequisites for handling real orders.

Every encoder change requires new product vectors and a new fingerprint/version. Every retrieval-policy change requires comparable frozen tests and explicit score interpretation. Extensions should preserve the rule that UI does not query storage directly, and should keep failures visible rather than changing acceptance labels after observing outputs.

11 Conclusion

Assignment 06 is fulfilled by a connected requirements model, native Visual Paradigm diagrams and an executable Python prototype. Text, simulated voice and image inputs share a validated query contract and a retrieval/filter/ranking pipeline. Separate application services and repositories make the three-layer design observable in code. The demonstration includes all required modes, product metadata and ranking scores, while fusion and scoped order lookup provide coherent extensions.

The primary benchmark passes twelve of twelve declared top-1 cases; both extensions and all five robustness criteria pass, while synonym and negation challenges fail. These outcomes establish local prototype behavior and disclose its limits. They do not establish general AI understanding, independent photographic accuracy, speech recognition quality or production readiness.

This complete English LaTeX report preserves all approved visual assets, includes the required task evidence and all Python source, and supplies data/configuration appendices for review. The delivered native project, source, fixtures, README and demonstration results remain the authoritative accompanying artifacts. The cover identifies Vu Dinh Hieu (B23DCCE036), class E23TTNT02, under the guidance of Tran Dinh Que at the Posts and Telecommunications Institute of Technology.

References

- [1] Information System Analysis and Design. *Assignment 06: Multimodal Search System for E-Commerce*. Local assignment brief, 23 pages. Tasks: pp. 17–19; evaluation/report: pp. 20–21; rubric/principles: p. 22.
- [2] Project source. `main.py`, `presentation/`, `application/`, `data/`, `scripts/`, `tests/`. Complete file snapshots reproduced in Appendix B.
- [3] Frozen benchmark and recorded experiment. `evaluation/queries.json`; `artifacts/evaluation/metrics.json`, `results.json` and `results.csv` within the evaluation artifact folder.
- [4] Native model and verification evidence. `models/Assignment_06_Multimodal_Search.vpp`; UML exports, VP/Python screenshots, JUnit and coverage artifacts included in the submission.

A Assignment Coverage and Deliverables

The assignment was reread in full, including its six tasks, evaluation requirements, recommended report structure, submission list, rubric and design principles. The table records coverage and identifies recommended advanced work that is intentionally outside the implemented prototype.

Brief item	Report location	Evidence / outcome
Task 1: requirements	Sections 2–3	Customer, twelve functional and eight non-functional requirements, inputs, outputs and validation.
Task 2: use cases	Section 4	Customer and all seven required goals; native VP model, export, screenshot and seven textual specifications.
Task 3: architecture	Section 5	Three native packages, sixteen components, dependency explanation, physical mapping and no direct Presentation-to-Data dependency.
Task 4: voice sequence	Section 6	Seven required participants, fourteen native messages, message-to-code explanation and explicit simulated-STT note.
Task 5: Python	Sections 7–8; Appendix B	Repository, text retrieval, simulated voice, image cosine, ranking, common query, complete executable source.
Task 6: demonstrations	Section 9	Text/voice/image inputs, processing, returned products and scores; four actual Python screenshots plus fusion/order outputs.
Evaluation, brief §22	Section 9	All 21 rows; 12 primary; successful counts/rates, separate denominators and two incorrect cases.
Recommended eleven parts	Sections 1–11	Each named part appears in order. Detailed source and dataset appendices supplement the main report.
Screenshots	Sections 4–6, 9	All three VP screenshots and all four Python viewer screenshots, plus the three native exports.
Multimodal rubric item	Section 8	Normalized score fusion, candidate union, hard filters and explicit weights. No incompatible-vector summation.
Order extension	Sections 4, 7, 9	Exact customer-context lookup with status/date/total. Natural-language latest-order and OrderItem management are not implemented.
UI/business separation	Sections 5, 7	Injected services, protected repository boundary and architecture test; composition root is the wiring exception.
Common representation	Sections 7–8	One validated dictionary contract for all product-search modalities.
Retrieval vs ranking	Sections 5, 7–8	Unsorted candidates before separate final score/order/top- k operation.
Start-simple principle	Sections 8, 10	Interpretable lexical/pixel baseline and explicit production/AI limitations.
Seven submission groups	Delivery below	PDF, native VP, source, catalogue, sample images, README and demo results.
Optional advanced work	Section 10	Implemented fusion, price/category filtering and order status. Real ASR, learned embeddings, vector DB, semantic text, stock boost and web UI are not claimed.

The rubric assigns 10 points to requirements, 15 to use case, 20 to architecture, 10 to sequence, 20 to Python, 10 to multimodal search, 5 to evaluation, 5 to report and 5 to demonstration (100 total). Coverage in this

report is not a self-awarded mark.

A.1 Submission inventory and reproducibility

The canonical report is `artifacts/report/Assignment_06_Report.pdf` and its source is `docs/report.tex`. The native model is `models/Assignment_06_Multimodal_Search.vpp`. Python packages, `dataset/index`, `catalogue/query images`, `README`, saved demo outputs and `evaluation/test` artifacts accompany them. A curated ZIP and SHA-256 manifest provide byte-level delivery verification; local preparation does not claim an LMS submission.

The report builder regenerates tables from saved JSON/JUnit/coverage evidence, snapshots every Python file, compiles three XeLaTeX passes and audits fonts, black text, bounds, bookmarks, embedded images and original-asset hashes. It requires MiKTeX/TeX Live with the used standard packages and Times New Roman, Arial and Consolas fonts. Search runtime remains independent of report/VP tooling.

Listing 5: Testing, evaluation and report reproduction commands

```
1 .venv\Scripts\python.exe -m pip install -r requirements-dev.txt
2 .venv\Scripts\python.exe -m pytest -q --cov=application --cov=data --cov-report=json:artifacts/tests/
  coverage.json --junitxml=artifacts/tests/junit.xml
3 .venv\Scripts\python.exe -m ruff check main.py application data presentation scripts tests
4 .venv\Scripts\python.exe -m pyright
5 .venv\Scripts\python.exe scripts/run_evaluation.py --queries evaluation/queries.json --output artifacts/
  evaluation
6 .venv\Scripts\python.exe scripts/build_report.py --input docs/report.tex --output artifacts/report/
  Assignment_06_Report.pdf
7 .venv\Scripts\python.exe scripts/validate_submission.py
8 .venv\Scripts\python.exe scripts/package_submission.py
```

Re-running evaluation measures a new timing run; do so deliberately and rebuild the report afterward. The completed report uses the preserved evaluated snapshot. Report-building inputs and full Python listings are identified by hashes in `artifacts/report/build_audit.json`. Full source files remain authoritative for copying/execution; printed listings are a review aid.

B Complete Python Source Code

This appendix reproduces all project Python files in full, covering the runtime, data layer, tools and tests. Each file is snapshotted without changing its content. Long lines wrap visually and keep line numbering. Original literal strings/comments, including any non-English helper text, are preserved as source; the surrounding explanation remains English.

B.1 Entry point and presentation

B.1.1 main.py

This listing contains the complete file (94 source lines).

Listing 6: main.py

```

1  """Composition root and reproducible offline CLI for Assignment 06."""
2
3  import argparse
4  import json
5  from pathlib import Path
6  import sys
7
8  from application.image_service import ImageService
9  from application.order_service import OrderService
10 from application.query_service import QueryService
11 from application.ranking_service import RankingService
12 from application.search_service import SearchService
13 from application.speech_service import SpeechService
14 from data.order_repository import OrderRepository
15 from data.product_repository import ProductRepository
16 from data.vector_index import VectorIndex
17 from presentation.result_view import SearchResultView
18 from presentation.search_ui import SearchUI
19
20 ROOT = Path(__file__).resolve().parent
21
22
23 def build_services(root=None):
24     root = Path(root or ROOT)
25     repository = ProductRepository(root=root)
26     index = VectorIndex(root / "data/embeddings.json", repository.all_products(), root)
27     return SearchUI(
28         QueryService(),
29         SpeechService(),
30         ImageService(),
31         SearchService(repository, index, RankingService()),
32         OrderService(OrderRepository(root / "data/orders.json")),
33     )
34
35
36 def main(argv=None):
37     parser = argparse.ArgumentParser(description="Offline multimodal product search; voice is simulated STT.")
38     parser.add_argument("--mode", choices=["text", "voice", "image", "multimodal", "order", "product"])
39     parser.add_argument("--demo", action="store_true")
40     parser.add_argument("--query")
41     parser.add_argument("--image")
42     parser.add_argument("--top-k", type=int, default=5)
43     parser.add_argument("--category", choices=["shoes", "bag", "clothing"])
44     parser.add_argument("--max-price", type=float, help="exclusive upper price limit in USD")
45     parser.add_argument("--text-weight", type=float, default=0.5)

```

```

46 parser.add_argument("--order-id")
47 parser.add_argument("--customer-id")
48 parser.add_argument("--product-id", type=int)
49 parser.add_argument("--json", action="store_true", help="machine-readable JSON on stdout")
50 parser.add_argument("--output", type=Path, help="write the JSON result to this path")
51 args = parser.parse_args(argv)
52 try:
53     ui = build_services()
54     options = {"top_k": args.top_k, "category": args.category, "max_price": args.max_price}
55     path = Path(args.image) if args.image else None
56     if path is not None and not path.is_absolute():
57         path = ROOT / path
58     if args.demo or args.mode is None:
59         payload = [
60             ui.search_text("black shoes"),
61             ui.search_voice("find black running shoes"),
62             ui.search_image(ROOT / "data/queries/black_shoe_query.png"),
63             ui.search_multimodal("black shoes", ROOT / "data/queries/black_shoe_query.png"),
64             ui.search_order("0001", "C001"),
65         ]
66     elif args.mode == "text":
67         payload = ui.search_text(args.query, **options)
68     elif args.mode == "voice":
69         payload = ui.search_voice(args.query, **options)
70     elif args.mode in {"image", "multimodal"}:
71         if path is None:
72             raise ValueError("--image is required for image and multimodal mode")
73         payload = (
74             ui.search_image(path, **options)
75             if args.mode == "image"
76             else ui.search_multimodal(args.query, path, text_weight=args.text_weight, **options)
77         )
78     elif args.mode == "order":
79         payload = ui.search_order(args.order_id, args.customer_id)
80     else:
81         payload = ui.view_product(args.product_id)
82     serialized = json.dumps(payload, indent=2, ensure_ascii=False, allow_nan=False)
83     if args.output:
84         args.output.parent.mkdir(parents=True, exist_ok=True)
85         args.output.write_text(serialized + "\n", encoding="utf-8")
86     print(serialized if args.json else SearchResultView().render(payload))
87     return 0
88 except (ValueError, OSError) as exc:
89     print(f"Error: {exc}", file=sys.stderr)
90     return 2
91
92
93 if __name__ == "__main__":
94     raise SystemExit(main())

```

B.1.2 presentation/__init__.py

This listing contains the complete file (1 source lines).

Listing 7: presentation/__init__.py

```

1 """CLI input adapters and result presentation."""

```

B.1.3 presentation/result_view.py

This listing contains the complete file (31 source lines).

Listing 8: presentation/result_view.py

```

1  """Human readable CLI output without data access or ranking logic."""
2
3  import json
4
5
6  class SearchResultView:
7      def render(self, payload):
8          if isinstance(payload, list):
9              return "\n\n".join(self.render(item) for item in payload)
10         if "results" not in payload:
11             return json.dumps(payload, indent=2, ensure_ascii=False)
12         query, processing = payload["query"], payload["processing"]
13         lines = [f"=== {query['type'].upper()} SEARCH ===", f"Input: {query['raw_input']}"]
14         if processing["simulated_stt"]:
15             lines.append("Voice mode: simulated speech-to-text (transcript supplied)")
16         lines.extend(
17             [
18                 f"Normalized: {query['query']} | tokens={query['tokens']}",
19                 f"Filters: {query['filters']}",
20                 f"Candidates: {processing['candidates_before_filter']} -> {processing['candidates_after_filter']} | weights={query['weights']}",
21             ]
22         )
23         if processing["encoder"]:
24             lines.append(f"Pixel encoder: {processing['encoder']}")
25         for p in payload["results"]:
26             lines.append(
27                 f"{p['rank']}. [{p['product_id']:02d}] {p['name']} | ${p['price']:.2f} | stock={p['stock']} | text={p['text_score']:.6f} image={p['image_score']:.6f} final={p['final_score']:.6f}"
28             )
29         if not payload["results"]:
30             lines.append("No matching products.")
31         return "\n\n".join(lines)

```

B.1.4 presentation/search_ui.py

This listing contains the complete file (36 source lines).

Listing 9: presentation/search_ui.py

```

1  """Input adapters depend only on injected application services."""
2
3
4  class SearchUI:
5      def __init__(self, query_service, speech_service, image_service, search_service, order_service):
6          self.query_service = query_service
7          self.speech_service = speech_service
8          self.image_service = image_service
9          self.search_service = search_service
10         self.order_service = order_service
11
12         def search_text(self, text, **kwargs):

```

```
13         return self.search_service.search(self.query_service.text_query(text, **kwargs))
14
15     def search_voice(self, transcript, **kwargs):
16         text = self.speech_service.transcribe(transcript)
17         query = self.query_service.voice_query(text, **kwargs)
18         query["raw_input"] = transcript
19         return self.search_service.search(query)
20
21     def search_image(self, path, **kwargs):
22         embedding = self.image_service.encode(path)
23         return self.search_service.search(self.query_service.image_query(embedding, raw_input=str(path),
24                                     **kwargs))
25
26     def search_multimodal(self, text, path, text_weight=0.5, **kwargs):
27         embedding = self.image_service.encode(path)
28         query = self.query_service.multimodal_query(
29             text, embedding, raw_input=f"{text} | {path}", text_weight=text_weight, **kwargs
30         )
31         return self.search_service.search(query)
32
33     def search_order(self, order_id, customer_id):
34         return self.order_service.find_order(order_id, customer_id)
35
36     def view_product(self, product_id):
37         return self.search_service.view_product(product_id)
```

B.2 Application and intelligence

B.2.1 application/__init__.py

This listing contains the complete file (1 source lines).

Listing 10: application/__init__.py

```
1 """Application services; independent of the presentation layer."""
```

B.2.2 application/image_service.py

This listing contains the complete file (34 source lines).

Listing 11: application/image_service.py

```
1 """Deterministic pixel descriptors, not learned semantic embeddings."""
2
3 from pathlib import Path
4
5 import numpy as np
6 from PIL import Image, ImageOps, UnidentifiedImageError
7
8 from application.query_service import ENCODER
9
10
11 class ImageService:
12     encoder = ENCODER
13
14     def encode(self, image_path):
15         try:
16             with Image.open(Path(image_path)) as original:
17                 if original.width * original.height > 20_000_000:
18                     raise ValueError("image exceeds 20 million pixels")
19                 image = ImageOps.exif_transpose(original).convert("RGB")
20                 pixels = np.asarray(image.resize((128, 128)), dtype=float)
21                 histogram = np.concatenate(
22                     [np.histogram(pixels[:, :, channel], bins=8, range=(0, 256))[0] for channel in
23                      range(3)]
24                 ).astype(float)
25                 histogram /= np.linalg.norm(histogram)
26                 thumbnail = np.asarray(image.convert("L").resize((8, 8)), dtype=float).flatten() / 255
27                 norm = np.linalg.norm(thumbnail)
28                 if norm:
29                     thumbnail /= norm
30                 # Each block contributes half of squared norm before final L2 normalization.
31                 descriptor = np.concatenate([histogram, thumbnail])
32                 descriptor /= np.linalg.norm(descriptor)
33                 return descriptor.tolist()
34         except (OSError, UnidentifiedImageError, Image.DecompressionBombError) as exc:
35             raise ValueError(f"cannot read image: {image_path}") from exc
```

B.2.3 application/order_service.py

This listing contains the complete file (21 source lines).

Listing 12: application/order_service.py

```

1  """Customer-scoped lookup; deliberately distinct from product ranking."""
2
3  import re
4
5
6  class OrderService:
7      def __init__(self, repository):
8          self.repository = repository
9
10     def find_order(self, order_id, customer_id):
11         if not isinstance(order_id, str) or not re.fullmatch(r"0\d{3}", order_id):
12             raise ValueError("order-id must have format 0001")
13         if not isinstance(customer_id, str) or not re.fullmatch(r"C\d{3}", customer_id):
14             raise ValueError("customer-id is required with format C001")
15         order = self.repository.find_order(order_id, customer_id)
16         return {
17             "query": {"type": "order", "order_id": order_id, "customer_id": customer_id},
18             "processing": {"customer_scope": customer_id, "authentication": "simulated customer context"},
19             "order": order,
20             "message": "Order found" if order else "Order not found for this customer",
21         }

```

B.2.4 application/query_service.py

This listing contains the complete file (120 source lines).

Listing 13: application/query_service.py

```

1  """One validated query contract for all three input modalities."""
2
3  import math
4  import re
5
6  import numpy as np
7
8  STOPWORDS = {"find", "show", "me", "a", "an", "the", "please", "for", "dollar", "dollars", "usd"}
9  ALIASES = {"shoe": "shoes", "bags": "bag", "backpacks": "backpack", "shirts": "shirt"}
10 CATEGORIES = {"shoes", "bag", "clothing"}
11 ENCODER = {"name": "rgbhist-gray-thumbnail", "version": "1.0", "dimension": 88}
12
13
14 def tokenize(text):
15     return list(dict.fromkeys(ALIASES.get(t, t) for t in re.findall(r"[a-z0-9]+", text.lower()) if t not
16                             in STOPWORDS))
17
18 class QueryService:
19     def _base(
20         self,
21         mode,
22         raw_input,
23         text="",
24         embedding=None,
25         top_k=5,
26         category=None,
27         max_price=None,
28         customer_id=None,

```

```

29     text_weight=None,
30 ):
31     if not isinstance(top_k, int) or isinstance(top_k, bool) or not 1 <= top_k <= 100:
32         raise ValueError("top-k must be an integer from 1 to 100")
33     if category is not None and category not in CATEGORIES:
34         raise ValueError("category must be shoes, bag, or clothing")
35     filters = {"category": category, "max_price_exclusive": None, "max_price_inclusive": None}
36     if max_price is not None:
37         if (
38             isinstance(max_price, bool)
39             or not isinstance(max_price, (int, float))
40             or not math.isfinite(max_price)
41             or max_price < 0
42         ):
43             raise ValueError("max-price must be a finite nonnegative number")
44         filters["max_price_exclusive"] = float(max_price)
45     normalized = ""
46     terms = []
47     if mode in {"text", "voice", "multimodal"}:
48         if not isinstance(text, str) or not text.strip() or len(text) > 2000:
49             raise ValueError("query must contain 1-2000 characters")
50         normalized = text.lower().strip()
51         pattern = r"\b(under|below|at most)\s+(\d+(?:\.\d+)?)\b"
52         for match in re.finditer(pattern, normalized):
53             key = "max_price_inclusive" if match[1] == "at most" else "max_price_exclusive"
54             price = float(match[2])
55             if not math.isfinite(price):
56                 raise ValueError("extracted price must be finite")
57             filters[key] = min(filters[key], price) if filters[key] is not None else price
58         lexical = re.sub(pattern, "", normalized)
59         terms = tokenize(lexical)
60         inferred = (
61             "shoes"
62             if "shoes" in terms
63             else "bag"
64             if {"bag", "backpack"} & set(terms)
65             else "clothing"
66             if {"shirt", "jacket", "clothing"} & set(terms)
67             else None
68         )
69         if inferred and category and inferred != category:
70             raise ValueError("explicit category conflicts with query category")
71         filters["category"] = category or inferred
72         if not terms and not any(v is not None for v in filters.values()):
73             raise ValueError("query has no searchable terms or filters")
74     vector = None
75     if mode in {"image", "multimodal"}:
76         try:
77             arr = np.asarray(embedding, dtype=float)
78         except (TypeError, ValueError) as exc:
79             raise ValueError("embedding must contain finite numeric values") from exc
80         if arr.shape != (88,) or not np.isfinite(arr).all():
81             raise ValueError("embedding must contain exactly 88 finite numbers")
82         vector = arr.tolist()
83     weight = 0.5 if mode == "multimodal" and text_weight is None else text_weight
84     if mode == "multimodal":
85         if (
86             isinstance(weight, bool)
87             or not isinstance(weight, (int, float))
88             or not math.isfinite(weight)
89             or not 0 <= weight <= 1
90         ):
91             raise ValueError("text-weight must be a finite number between 0 and 1")
92         weights = {"text": float(weight), "image": 1 - float(weight)}
93     else:
94         weights = {"text": 0.0 if mode == "image" else 1.0, "image": 1.0 if mode == "image" else 0.0}

```

```

95         return {
96             "type": mode,
97             "raw_input": str(raw_input),
98             "query": normalized,
99             "tokens": terms,
100            "embedding": vector,
101            "filters": filters,
102            "top_k": top_k,
103            "weights": weights,
104            "customer_id": customer_id,
105            "encoder": ENCODER if vector is not None else None,
106        }
107
108    def text_query(self, text, **kwargs):
109        return self._base("text", text, text=text, **kwargs)
110
111    def voice_query(self, transcript, **kwargs):
112        return self._base("voice", transcript, text=transcript, **kwargs)
113
114    def image_query(self, embedding, raw_input="", **kwargs):
115        return self._base("image", raw_input, embedding=embedding, **kwargs)
116
117    def multimodal_query(self, text, embedding, raw_input=None, text_weight=0.5, **kwargs):
118        return self._base(
119            "multimodal", raw_input or text, text=text, embedding=embedding, text_weight=text_weight,
120            **kwargs
121        )

```

B.2.5 application/ranking_service.py

This listing contains the complete file (23 source lines).

Listing 14: application/ranking_service.py

```

1  """Final score, deterministic ordering, and top-k belong only here."""
2
3  from copy import deepcopy
4  import math
5
6
7  class RankingService:
8      def rank(self, candidates, query):
9          weights = query["weights"]
10         if (
11             set(weights) != {"text", "image"}
12             or any(
13                 isinstance(w, bool) or not isinstance(w, (int, float)) or not math.isfinite(w) or w < 0
14                 for w in weights.values()
15             )
16             or not math.isclose(sum(weights.values()), 1)
17         ):
18             raise ValueError("ranking weights must be nonnegative and sum to one")
19         results = deepcopy(candidates)
20         for result in results:
21             result["final_score"] = weights["text"] * result["text_score"] + weights["image"] *
22             result["image_score"]
23         results.sort(key=lambda p: (-p["final_score"], p["product_id"]))
24         return [dict(result, rank=i + 1) for i, result in enumerate(results[: query["top_k"]])]

```

B.2.6 application/search_service.py

This listing contains the complete file (79 source lines).

Listing 15: application/search_service.py

```

1  """Retrieval produces candidates; RankingService owns ordering and top-k."""
2
3  from application.query_service import tokenize
4
5
6  class SearchService:
7      def __init__(self, repository, index, ranking):
8          self.repository, self.index, self.ranking = repository, index, ranking
9
10     def _retrieve(self, query):
11         if query.get("type") not in {"text", "voice", "image", "multimodal"}:
12             raise ValueError("unsupported query type")
13         tokens = set(query["tokens"])
14         image_scores = self.index.score(query["embedding"]) if query["embedding"] is not None else {}
15         candidates = []
16         before_filter = 0
17         for p in self.repository.all_products():
18             searchable = set(
19                 tokenize(" ".join(str(p[k]) for k in ["name", "brand", "category", "color",
20                 "description"])))
21             matches = sorted(tokens & searchable)
22             # Fusion union: every image-index member is a candidate even with zero text match.
23             if query["type"] in {"text", "voice"} and tokens and not matches:
24                 continue
25             before_filter += 1
26             filters = query["filters"]
27             if filters["category"] is not None and p["category"] != filters["category"]:
28                 continue
29             if filters["max_price_exclusive"] is not None and p["price"] >=
30                 filters["max_price_exclusive"]:
31                 continue
32             if filters["max_price_inclusive"] is not None and p["price"] >
33                 filters["max_price_inclusive"]:
34                 continue
35             candidates.append(
36                 {
37                     "product_id": p["product_id"],
38                     "name": p["name"],
39                     "brand": p["brand"],
40                     "category": p["category"],
41                     "color": p["color"],
42                     "price": p["price"],
43                     "stock": p["stock"],
44                     "description": p["description"],
45                     "image_path": p["image"],
46                     "raw_text_score": len(matches),
47                     "text_score": len(matches) / len(tokens) if tokens else 0.0,
48                     "image_score": image_scores.get(p["product_id"], 0.0),
49                     "business_score": 0.0,
50                     "matched_terms": matches,
51                 }
52             )
53         return candidates, before_filter
54
55     def retrieve_candidates(self, query):
56         return self._retrieve(query)[0]
57
58     def search(self, query):

```

```

57     candidates, before_filter = self._retrieve(query)
58     ranked = self.ranking.rank(candidates, query)
59     processing = {
60         "modality": query["type"],
61         "normalized_query": query["query"],
62         "tokens": query["tokens"],
63         "filters": query["filters"],
64         "candidates_before_filter": before_filter,
65         "candidates_after_filter": len(candidates),
66         "encoder": query["encoder"],
67         "ranking_weights": query["weights"],
68         "top_k": query["top_k"],
69         "simulated_stt": query["type"] == "voice",
70         "retrieval": "whole-word OR matching; explicit category inferred; image candidates union",
71         "ranking": "weighted normalized scores; product_id ascending breaks ties; no business
72         boost",
73     }
74     return {"query": query, "processing": processing, "results": ranked}
75
76 def view_product(self, product_id):
77     if not isinstance(product_id, int) or isinstance(product_id, bool) or product_id <= 0:
78         raise ValueError("product-id must be a positive integer")
79     product = self.repository.get_by_id(product_id)
80     return {"product": product, "message": "Product found" if product else "Product not found"}

```

B.2.7 application/speech_service.py

This listing contains the complete file (10 source lines).

Listing 16: application/speech_service.py

```

1  """Offline simulated speech-to-text adapter: input already is a transcript."""
2
3
4  class SpeechService:
5      simulated = True
6
7      def transcribe(self, audio_input):
8          if not isinstance(audio_input, str) or not audio_input.strip() or len(audio_input) > 2000:
9              raise ValueError("simulated voice input must be a nonempty transcript (maximum 2000
10             characters)")
11             return audio_input.strip()

```

B.3 Data access and vector index

B.3.1 data/__init__.py

This listing contains the complete file (1 source lines).

Listing 17: data/__init__.py

```
1 """Validated local repositories and vector storage."""
```

B.3.2 data/order_repository.py

This listing contains the complete file (39 source lines).

Listing 18: data/order_repository.py

```
1 """Local order data; no production authentication is implied."""
2
3 from copy import deepcopy
4 import json
5 import math
6 from pathlib import Path
7
8
9 class OrderRepository:
10     def __init__(self, path=None):
11         try:
12             self.orders = json.loads(Path(path or
13             Path(__file__).with_name("orders.json")).read_text(encoding="utf-8"))
14         except (OSError, json.JSONDecodeError) as exc:
15             raise ValueError("cannot load orders") from exc
16         if not isinstance(self.orders, list):
17             raise ValueError("orders must be a list")
18         seen = set()
19         for order in self.orders:
20             fields = {"order_id", "customer_id", "date", "status", "total"}
21             if not isinstance(order, dict) or not fields <= order.keys():
22                 raise ValueError("order is missing required fields")
23             if any(not isinstance(order[k], str) or not order[k].strip() for k in fields - {"total"}):
24                 raise ValueError("order identifiers/status/date must be strings")
25             if order["order_id"] in seen:
26                 raise ValueError("duplicate order ID")
27             seen.add(order["order_id"])
28             total = order["total"]
29             if isinstance(total, bool) or not isinstance(total, (int, float)) or not
30                 math.isfinite(total) or total < 0:
31                 raise ValueError("invalid order total")
32
33     def find_order(self, order_id, customer_id=None):
34         return next(
35             (
36                 deepcopy(o)
37                 for o in self.orders
38                 if o["order_id"] == order_id and (customer_id is None or o["customer_id"] ==
39                 customer_id)
40             ),
41             None,
42         )
```

B.3.3 data/product_repository.py

This listing contains the complete file (49 source lines).

Listing 19: data/product_repository.py

```

1  """Local product catalogue with strict schema and portable image paths."""
2
3  from copy import deepcopy
4  import json
5  import math
6  from pathlib import Path
7
8
9  class ProductRepository:
10     def __init__(self, path=None, root=None):
11         self.root = Path(root or Path(__file__).resolve().parents[1]).resolve()
12         self.path = Path(path or self.root / "data/products.json")
13         try:
14             products = json.loads(self.path.read_text(encoding="utf-8"))
15         except (OSError, json.JSONDecodeError) as exc:
16             raise ValueError(f"cannot load product catalogue: {self.path}") from exc
17         if not isinstance(products, list) or not products:
18             raise ValueError("product catalogue must be a nonempty list")
19         ids = set()
20         for p in products:
21             required = {"product_id", "name", "brand", "category", "color", "price", "stock",
22                         "description", "image"}
23             if not isinstance(p, dict) or not required <= p.keys():
24                 raise ValueError("product is missing required fields")
25             identifier = p["product_id"]
26             if not isinstance(identifier, int) or isinstance(identifier, bool) or identifier <= 0 or
27                 identifier in ids:
28                 raise ValueError("product IDs must be unique positive integers")
29             ids.add(identifier)
30             if any(
31                 not isinstance(p[k], str) or not p[k].strip()
32                 for k in ["name", "brand", "category", "color", "description", "image"]
33             ):
34                 raise ValueError("product textual fields must be nonempty strings")
35             if p["category"] not in {"shoes", "bag", "clothing"}:
36                 raise ValueError("unsupported product category")
37             price, stock = p["price"], p["stock"]
38             if isinstance(price, bool) or not isinstance(price, (int, float)) or not
39                 math.isfinite(price) or price < 0:
40                 raise ValueError("price must be finite and nonnegative")
41             if isinstance(stock, bool) or not isinstance(stock, int) or stock < 0:
42                 raise ValueError("stock must be a nonnegative integer")
43             image = (self.root / p["image"]).resolve()
44             if not image.is_relative_to(self.root) or not image.is_file():
45                 raise ValueError("product image must exist inside project root")
46         self._products = products
47
48     def all_products(self):
49         return deepcopy(self._products)
50
51     def get_by_id(self, product_id):
52         return next((p for p in self.all_products() if p["product_id"] == product_id), None)

```

B.3.4 data/vector_index.py

This listing contains the complete file (53 source lines).

Listing 20: data/vector_index.py

```

1  """Unsorted cosine candidate scoring and source fingerprints."""
2
3  import hashlib
4  import json
5  from pathlib import Path
6
7  import numpy as np
8
9
10 def cosine_similarity(a, b):
11     try:
12         left, right = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
13     except (TypeError, ValueError) as exc:
14         raise ValueError("vectors must be numeric") from exc
15     if left.ndim != 1 or left.shape != right.shape or not np.isfinite(left).all() or not
16         np.isfinite(right).all():
17         raise ValueError("vectors require matching dimensions and finite values")
18     denominator = np.linalg.norm(left) * np.linalg.norm(right)
19     return float(np.clip(np.dot(left, right) / denominator, -1, 1)) if denominator else 0.0
20
21 def source_fingerprint(products, root):
22     digest = hashlib.sha256(json.dumps(products, sort_keys=True, ensure_ascii=False).encode("utf-8"))
23     for p in products:
24         digest.update((Path(root) / p["image"]).read_bytes())
25     return digest.hexdigest()
26
27
28 class VectorIndex:
29     def __init__(self, path=None, products=None, root=None):
30         self.path = Path(path or Path(__file__).with_name("embeddings.json"))
31         try:
32             payload = json.loads(self.path.read_text(encoding="utf-8"))
33             self.encoder = payload["encoder"]
34             self.dimension = self.encoder["dimension"]
35             self.vectors = {int(k): np.asarray(v, dtype=float).tolist() for k, v in
36                 payload["vectors"].items()}
37             if self.encoder != {"name": "rgbhist-gray-thumbnail", "version": "1.0", "dimension": 88}:
38                 raise ValueError("unsupported encoder metadata")
39             if any(np.asarray(v).shape != (self.dimension,) or not np.isfinite(v).all() for v in
40                 self.vectors.values()):
41                 raise ValueError("invalid stored embedding")
42             if products is not None:
43                 if set(self.vectors) != {p["product_id"] for p in products}:
44                     raise ValueError("index product IDs do not match catalogue")
45                 if payload["source_sha256"] != source_fingerprint(products, root):
46                     raise ValueError("stale image index; run scripts/build_index.py")
47             except (OSError, KeyError, TypeError, json.JSONDecodeError) as exc:
48                 raise ValueError("invalid image index; run scripts/build_index.py") from exc
49
50     def score(self, embedding):
51         if np.asarray(embedding).shape != (self.dimension,):
52             raise ValueError("query vector dimension does not match index")
53         return {
54             identifier: max(0.0, cosine_similarity(embedding, vector)) for identifier, vector in
55             self.vectors.items()
56         }

```

B.4 Dataset, evaluation, report and delivery utilities

B.4.1 scripts/build_index.py

This listing contains the complete file (40 source lines).

Listing 21: scripts/build_index.py

```

1  """Build the same 88-dimensional encoder for catalogue and query images."""
2
3  import json
4  from pathlib import Path
5  import sys
6
7  ROOT = Path(__file__).resolve().parents[1]
8  sys.path.insert(0, str(ROOT))
9
10 from application.image_service import ImageService
11 from data.product_repository import ProductRepository
12 from data.vector_index import source_fingerprint
13
14
15 def build(root=ROOT):
16     root = Path(root)
17     products = ProductRepository(root=root).all_products()
18     encoder = ImageService()
19     output = {
20         "encoder": encoder.encoder,
21         "descriptor_blocks": {
22             "rgb_histogram": 24,
23             "grayscale_thumbnail": 64,
24             "block_norms": "L2; equal block weight then final L2",
25         },
26         "source_sha256": source_fingerprint(products, root),
27         "vectors": {str(p["product_id"]): encoder.encode(root / p["image"]) for p in products},
28     }
29     path = root / "data/embeddings.json"
30     serialized = json.dumps(output, indent=2, allow_nan=False) + "\n"
31     if not path.exists() or path.read_text(encoding="utf-8") != serialized:
32         path.write_text(serialized, encoding="utf-8")
33         status = "rebuilt"
34     else:
35         status = "unchanged"
36     return {"products": len(products), "encoder": encoder.encoder, "status": status}
37
38
39 if __name__ == "__main__":
40     print(json.dumps(build(), indent=2))

```

B.4.2 scripts/build_report.py

This listing contains the complete file (160 source lines).

Listing 22: scripts/build_report.py

```

1  """Build the complete English LaTeX report, with exact Python source listings."""
2
3  import argparse
4  import hashlib

```

```

5 import io
6 import json
7 from pathlib import Path
8 import re
9 import shutil
10 import subprocess
11 import sys
12
13 import pymupdf
14 from PIL import Image
15
16 ROOT = Path(__file__).resolve().parents[1]
17 sys.path.insert(0, str(ROOT))
18
19 from scripts.report_facts import prepare_report_inputs
20
21 SECTIONS = [
22     "Introduction", "Problem Description", "Requirements Analysis", "Use Case Model",
23     "Three-Layer Architecture", "UML Sequence Diagram", "Python Implementation",
24     "Multimodal Search Method", "Experimental Results", "Discussion", "Conclusion",
25 ]
26
27
28 def sha256(path):
29     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
30
31
32 def build(source, output, engine="xelatex"):
33     executable = shutil.which(engine)
34     if not executable:
35         raise RuntimeError("XeLaTeX is required. Install MiKTeX or TeX Live and expose xelatex on PATH.")
36     source, output = Path(source).resolve(), Path(output).resolve()
37     if source.suffix.lower() != ".tex":
38         raise ValueError("The current report source is docs/report.tex; Markdown is no longer the report input.")
39     if not source.is_file():
40         raise FileNotFoundError(source)
41     work = ROOT / "artifacts/report/latex_build"
42     work.mkdir(parents=True, exist_ok=True)
43     inputs = prepare_report_inputs(ROOT, work)
44     inputs["input_sha256"][source.relative_to(ROOT).as_posix()] = sha256(source)
45     job = "Assignment_06_Report"
46     command = [executable, "-disable-installer", "-interaction=nonstopmode", "-halt-on-error",
47               "-file-line-error", f"-output-directory={work}", f"-jobname={job}", str(source)]
48     for number in range(1, 4):
49         completed = subprocess.run(command, cwd=ROOT, text=True, encoding="utf-8", errors="replace",
50                                   capture_output=True, timeout=240)
51         (work / f"compile_pass_{number}.txt").write_text(completed.stdout + completed.stderr,
52                                                         encoding="utf-8")
53         if completed.returncode:
54             raise RuntimeError(f"XeLaTeX pass {number} failed. See {work / f'compile_pass_{number}.txt'}\n"
55                               + completed.stdout[-2500:])
56         print(f"XeLaTeX pass {number}/3 complete", flush=True)
57     log = (work / f"{job}.log").read_text(encoding="utf-8", errors="replace")
58     bad = [line for line in log.splitlines() if "Overfull \\" in line or "Missing character:" in line or
59           re.search(r"(?:Reference|Citation).*undefined|There were undefined|Label\\(s\\) may have changed", line)]
60     if bad:
61         raise RuntimeError("Resolve LaTeX layout/reference warnings before publishing: " +
62                           "\n".join(bad[:30]))
63     compiled = work / f"{job}.pdf"
64     with pymupdf.open(compiled) as pdf:
65         pages = len(pdf)
66         outline = pdf.get_toc()

```

```

65     titles = [entry[1] for entry in outline]
66     section_pages = {}
67     for index, heading in enumerate(SECTIONS, 1):
68         matches = [entry for entry in outline if entry[1] == f"{index} {heading}"]
69         if not matches:
70             raise ValueError(f"Missing required report section: {heading}; outline={titles[:20]}")
71         section_pages[heading] = matches[0][2]
72     empty, nonblack, outside = [], [], []
73     fonts, seen_images = set(), set()
74     for index, page in enumerate(pdf):
75         if not page.get_text().strip():
76             empty.append(index + 1)
77         for block in page.get_text("dict")["blocks"]:
78             for line in block.get("lines", []):
79                 for span in line["spans"]:
80                     fonts.add(span["font"])
81                     if span["color"] != 0:
82                         nonblack.append({"page": index + 1, "text": span["text"]})
83                     x, y, right, bottom = span["bbox"]
84                     if x < 15 or right > page.rect.width - 15 or y < 12 or bottom > page.rect.height
                        - 12:
85                         outside.append({"page": index + 1, "text": span["text"], "bbox":
86                             list(span["bbox"])})
87         for item in page.get_images():
88             seen_images.add(item[0])
89     if empty or nonblack or outside:
90         raise ValueError(f"PDF audit failed: empty={empty}, nonblack={nonblack[:3]},
91             bounds={outside[:3]}")
92     if len(seen_images) < len(inputs["figures"]):
93         raise ValueError("Not all approved figures were embedded in the PDF")
94     def decoded_fingerprint(image):
95         return (image.size, hashlib.sha256(image.convert("RGB").tobytes()).hexdigest())
96     embedded_pixels = {}
97     for xref in seen_images:
98         with Image.open(io.BytesIO(pdf.extract_image(xref)["image"])) as decoded:
99             embedded_pixels[decoded_fingerprint(decoded)] = xref
100     figure_mapping = {}
101     for relative in inputs["figures"]:
102         with Image.open(ROOT / relative) as original:
103             identity = decoded_fingerprint(original)
104             if identity not in embedded_pixels:
105                 raise ValueError("Approved figure pixels were not preserved in PDF: " + relative)
106             figure_mapping[relative] = embedded_pixels[identity]
107     embedded_fonts = {}
108     for page in pdf:
109         for item in page.get_fonts(full=True):
110             embedded_fonts[item[3]] = bool(pdf.extract_font(item[0])[3])
111     if not all(embedded_fonts.values()):
112         raise ValueError("PDF contains a font that is not embedded")
113     text = "\n".join(page.get_text() for page in pdf)
114     for record in inputs["python_listings"]:
115         if record["path"] not in text:
116             raise ValueError("Source listing caption is missing: " + record["path"])
117     audit = {
118         "format": "assignment06-latex-report-v2", "pdf": output.relative_to(ROOT).as_posix(),
119         "language": "English", "engine": "XeLaTeX", "page_limit": None, "pages": pages,
120         "section_pages": section_pages, "all_required_sections_present": True,
121         "page_sizes": [list(page.rect) for page in pdf], "drawn_fonts": sorted(fonts),
122         "embedded_fonts": embedded_fonts, "nonblack_text": nonblack, "text_outside_safe_bounds":
123             outside,
124         "unique_embedded_images": len(seen_images), "figures": inputs["figures"],
125         "all_approved_image_pixels_matched": True, "figure_xrefs": figure_mapping,
126         "python_listings": inputs["python_listings"], "python_listing_count":
127             len(inputs["python_listings"]),
128         "complete_python_line_count": sum(r["lines"] for r in inputs["python_listings"]),
129         "evaluation": inputs["evaluation"], "tests": inputs["tests"],

```

```

126         "input_sha256": inputs["input_sha256"], "approved_UML_and_images_unchanged": True,
127         "preserved_assets": inputs["preserved_assets"], "latex_layout_and_references_clean": True,
128         "pdf_sha256": sha256(compiled),
129     }
130     review = output.parent / "review_pages"
131     review.mkdir(parents=True, exist_ok=True)
132     for index, page in enumerate(pdf):
133         page.get_pixmap(matrix=pymupdf.Matrix(1.25, 1.25)).save(review / f"page_{index +
134         1:03d}.png")
135     for name, expected in inputs["preserved_assets"].items():
136         if sha256(ROOT / name) != expected:
137             raise ValueError("An approved asset changed during the report build: " + name)
138     output.parent.mkdir(parents=True, exist_ok=True)
139     try:
140         shutil.copyfile(compiled, output)
141     except PermissionError as exc:
142         raise PermissionError(f"The compiled PDF is ready at {compiled}; close only the old report in
143         its viewer "
144         "and run this build again to publish it.") from exc
145     (output.parent / "build_audit.json").write_text(json.dumps(audit, indent=2, ensure_ascii=False) +
146     "\n",
147     encoding="utf-8")
148     print(f"Built {output}: {pages} pages, {len(inputs['python_listings'])} complete Python files, "
149     f"{len(inputs['figures'])} approved images; black text, embedded fonts, clean references.")
150     return audit
151
152 def main():
153     parser = argparse.ArgumentParser(description=__doc__)
154     parser.add_argument("--input", type=Path, default=ROOT / "docs/report.tex")
155     parser.add_argument("--output", type=Path, default=ROOT /
156     "artifacts/report/Assignment_06_Report.pdf")
157     parser.add_argument("--engine", default="xelatex")
158     args = parser.parse_args()
159     build(args.input, args.output, args.engine)
160
161 if __name__ == "__main__":
162     main()

```

B.4.3 scripts/package_submission.py

This listing contains the complete file (258 source lines).

Listing 23: scripts/package_submission.py

```

1  """Build a curated reproducible ZIP after validation and clean extraction smoke."""
2
3  import argparse
4  import hashlib
5  import json
6  import os
7  from pathlib import Path
8  import subprocess
9  import sys
10 import tempfile
11 import zipfile
12
13 ROOT = Path(__file__).resolve().parents[1]
14 sys.path.insert(0, str(ROOT))
15

```

```

16 from scripts.validate_submission import validate
17
18 FIXED_DATE = (2026, 10, 1, 0, 0, 0)
19
20
21 def selected_files(root):
22     selected = set()
23     for directory in ["application", "presentation", "data", "evaluation", "docs", "tests"]:
24         for path in (root / directory).rglob("*"):
25             if path.is_file() and "__pycache__" not in path.parts and path.suffix not in {".pyc",
26                 ".pyo"}:
27                 selected.add(path)
28     for name in [
29         "main.py",
30         "README.md",
31         ".gitignore",
32         ".gitattributes",
33         "pyproject.toml",
34         "requirements.txt",
35         "requirements-dev.txt",
36         "requirements-automation.txt",
37     ]:
38         path = root / name
39         if path.is_file():
40             selected.add(path)
41     for path in (root / "scripts").rglob("*"):
42         if (
43             path.is_file()
44             and path.suffix in {".py", ".java", ".xml"}
45             and "classes" not in path.parts
46             and "__pycache__" not in path.parts
47         ):
48             selected.add(path)
49     for directory in ["artifacts/diagrams", "artifacts/demo", "artifacts/evaluation",
50         "artifacts/tests"]:
51         for path in (root / directory).glob("*"):
52             if (
53                 path.is_file()
54                 and path.suffix in {".png", ".txt", ".json", ".csv", ".xml"}
55                 and "_pending" not in path.name
56             ):
57                 selected.add(path)
58     for path in (root / "artifacts/screenshots").glob("*"):
59         if path.is_file() and path.suffix in {".png", ".json"} and path.name.startswith(("vp_",
60             "demo_")):
61             selected.add(path)
62     for relative in [
63         "models/Assignment_06_Multimodal_Search.vpp",
64         "artifacts/report/Assignment_06_Report.pdf",
65         "artifacts/report/REVIEW.md",
66         "artifacts/report/build_audit.json",
67         "artifacts/automation/model_inventory.json",
68         "artifacts/automation/reopened_inventory.json",
69         "artifacts/automation/verify_result.txt",
70         "artifacts/automation/refine_result.txt",
71     ]:
72         path = root / relative
73         if path.is_file():
74             selected.add(path)
75     return sorted(selected, key=lambda p: p.relative_to(root).as_posix())
76
77 def smoke_clean_extraction(root, files, interpreter):
78     test_root = root / "artifacts/tests"
79     test_root.mkdir(parents=True, exist_ok=True)
80     environment = os.environ.copy()

```

```

79 environment.pop("PYTHONPATH", None)
80 environment["PYTHONIOENCODING"] = "utf-8"
81 commands = [
82     ["main.py", "--demo", "--json"],
83     ["main.py", "--mode", "text", "--query", "black shoes", "--json"],
84     ["main.py", "--mode", "voice", "--query", "find black running shoes", "--json"],
85     ["main.py", "--mode", "image", "--image", "data/queries/black_shoe_query.png", "--json"],
86     ["-m", "pytest", "-q"],
87 ]
88 runs = []
89 with tempfile.TemporaryDirectory(prefix="submission-smoke-", dir=test_root) as temporary:
90     extracted = Path(temporary)
91     # Build a ZIP snapshot, then really extract it; no imports use source workspace.
92     snapshot = extracted / "snapshot.zip"
93     with zipfile.ZipFile(snapshot, "w", zipfile.ZIP_DEFLATED) as archive:
94         for path in files:
95             archive.write(path, path.relative_to(root).as_posix())
96     destination = extracted / "clean"
97     with zipfile.ZipFile(snapshot) as archive:
98         archive.extractall(destination)
99     checks = validate(destination)
100     if not checks["passed"]:
101         raise RuntimeError("Clean extraction validation failed: " + json.dumps(checks))
102     for args in commands:
103         completed = subprocess.run(
104             [str(interpreter), *args],
105             cwd=destination,
106             env=environment,
107             text=True,
108             encoding="utf-8",
109             capture_output=True,
110             timeout=90,
111         )
112         record = {
113             "command": [str(interpreter), *args],
114             "cwd": "clean extracted assignment root",
115             "returncode": completed.returncode,
116             "stdout": completed.stdout,
117             "stderr": completed.stderr,
118         }
119         runs.append(record)
120         if completed.returncode:
121             raise RuntimeError("Clean extraction command failed: " + json.dumps(record))
122         if args[0] == "main.py":
123             payload = json.loads(completed.stdout)
124             if isinstance(payload, list):
125                 assert {"text", "voice", "image"} <= {p["query"]["type"] for p in payload}
126             else:
127                 assert payload["results"] and "final_score" in payload["results"][0]
128     return {
129         "passed": True,
130         "dependency_environment": "workspace .venv interpreter; dependencies reused, extracted source isolated",
131         "validation": checks,
132         "runs": runs,
133     }
134
135
136 def write_final_verification(root, validation, smoke):
137     tests = next(c["details"] for c in validation["checks"] if c["name"].startswith("Test, coverage"))
138     lines = [
139         "# Final Verification – Assignment 06",
140         "",
141         "Bản bàn giao được kiểm tra từ file thật, service output, native model inventory và ZIP extraction sạch. Báo cáo PDF dùng đúng `artifacts/report/Assignment_06_Report.pdf`.",
142         "",

```

```

143         "| Gate | Kết quả |",
144         "|---|---|",
145     ]
146     lines.extend(f"| {c['name']} | {'PASS' if c['passed'] else 'FAIL'} |" for c in validation["checks"])
147     lines.extend(
148         [
149             "",
150             f"Tests: **{tests['tests']} PASS**, failures/errors={tests['failures']},
151             skipped={tests['skipped']}; application/data line coverage
152             **{tests['application_data_line_coverage']:.2f}%**. Ruff/typecheck/pip-check evidence PASS.
153             Complete English LaTeX PDF has no page limit, includes all Python files and all 25 approved
154             images; required UML exports and screenshots exist and decode.",
155             "",
156             "Clean extraction smoke dùng interpreter `.venv` của workspace để reuse dependency, nhưng
157             chạy source từ ZIP snapshot giải nén trong cwd riêng, không dùng PYTHONPATH workspace. Đây
158             là portable-source smoke, không tuyên bố clean dependency installation.",
159             "",
160             "Các lệnh đã chạy thành công từ cwd của source giải nén, gọi đường dẫn interpreter workspace
161             tuyệt đối:",
162             "",
163             "```powershell",
164         ]
165     )
166     lines.extend(f'& "{run["command"]}" ' + subprocess.list2cmdline(run["command"][1:]) for run in
167     smoke["runs"])
168     lines.extend(
169         [
170             "```",
171             "",
172             "Kết quả chi tiết: `artifacts/tests/submission_validation.json`,
173             `artifacts/tests/clean_extraction.json`. Ground-truth và results được đối chiếu
174             SHA-256/IDs/counts; primary, extension, challenge, robustness có denominator riêng.
175             Challenge failures được giữ nguyên trong report/evaluation.",
176             "",
177             "ZIP chọn source Python, dataset/image/query fixtures, frozen ground truth, docs, report
178             PDF, native `.vpp`, UML exports, assignment screenshots, test/demo/evaluation evidence,
179             desktop script và Java/XML plugin source. Không đưa `.venv`, backup, VP
180             seed/workspace/plugin classes, ảnh project cũ, source PDF đề vào ZIP.",
181             "",
182             "Manifest chứa SHA-256 và byte size cho mỗi file payload, không hash chính manifest.
183             Timestamp ZIP cố định giúp tái tạo byte-identical archive từ cùng input.
184             `submission/manifest.json` ngoài ZIP bổ sung SHA-256 của archive; ZIP hash không được ghi
185             vào manifest bên trong để tránh vòng tự tham chiếu.",
186             "",
187             "Giới hạn: voice là simulated STT; ảnh synthetic và pixel descriptor không phải semantic
188             learned encoder; order context không phải production authentication; chưa nộp LMS. Native
189             model inventory chứng minh object types; việc reopen/render thật trong Visual Paradigm do
190             root ghi ở progress/screenshots.",
191             "",
192         ]
193     )
194     )
195     (root / "docs/FINAL_VERIFICATION.md").write_text("\n".join(lines), encoding="utf-8")
196
197 def package(root=ROOT, output=None, interpreter=None):
198     root = Path(root).resolve()
199     output = Path(output or root / "submission/Assignment_06_Submission.zip").resolve()
200     interpreter = Path(interpreter or root / ".venv/Scripts/python.exe").resolve()
201     validation = validate(root)
202     if not validation["passed"]:
203         raise RuntimeError("Deliverables are not ready: " + json.dumps(validation, ensure_ascii=False))
204     smoke = smoke_clean_extraction(root, selected_files(root), interpreter)
205     (root / "artifacts/tests/submission_validation.json").write_text(
206         json.dumps(validation, indent=2, ensure_ascii=False) + "\n", encoding="utf-8"
207     )
208     (root / "artifacts/tests/clean_extraction.json").write_text(

```

```

189     json.dumps(smoke, indent=2, ensure_ascii=False) + "\n", encoding="utf-8"
190 )
191 write_final_verification(root, validation, smoke)
192 payloads = {path.relative_to(root).as_posix(): path.read_bytes() for path in selected_files(root)}
193 manifest = {
194     "format": "assignment06-sha256-v1",
195     "assignment": "06",
196     "archive_path": output.name,
197     "file_count": len(payloads),
198     "files": [
199         {"path": name, "bytes": len(data), "sha256": hashlib.sha256(data).hexdigest()}
200         for name, data in payloads.items()
201     ],
202     "self_hash_policy": "manifest.json excluded from own file list; archive hash only in external
manifest",
203 }
204 output.parent.mkdir(parents=True, exist_ok=True)
205 temporary = output.with_suffix(".zip.tmp")
206 try:
207     with zipfile.ZipFile(temporary, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9) as
archive:
208         for name, data in [
209             *payloads.items(),
210             ("manifest.json", (json.dumps(manifest, indent=2, ensure_ascii=False) +
"\n").encode("utf-8")),
211         ]:
212             info = zipfile.ZipInfo(name, date_time=FIXED_DATE)
213             info.compress_type = zipfile.ZIP_DEFLATED
214             info.external_attr = 0o100644 << 16
215             archive.writestr(info, data)
216     with zipfile.ZipFile(temporary) as archive:
217         assert archive.testzip() is None, "Corrupted archive"
218         for record in manifest["files"]:
219             data = archive.read(record["path"])
220             assert hashlib.sha256(data).hexdigest() == record["sha256"], "Archive checksum mismatch"
221     temporary.replace(output)
222 finally:
223     if temporary.exists():
224         temporary.unlink()
225 external_manifest = dict(
226     manifest,
227     archive={
228         "path": output.name,
229         "bytes": output.stat().st_size,
230         "sha256": hashlib.sha256(output.read_bytes()).hexdigest(),
231     },
232     validation_passed=True,
233     clean_extraction_passed=True,
234 )
235 (output.parent / "manifest.json").write_text(
236     json.dumps(external_manifest, indent=2, ensure_ascii=False) + "\n", encoding="utf-8"
237 )
238 return external_manifest
239
240
241 if __name__ == "__main__":
242     parser = argparse.ArgumentParser(description=__doc__)
243     parser.add_argument("--root", type=Path, default=ROOT)
244     parser.add_argument("--output", type=Path)
245     parser.add_argument("--python", type=Path, help="absolute interpreter for isolated source smoke")
246     args = parser.parse_args()
247     result = package(args.root, args.output, args.python)
248     print(
249         json.dumps(
250             {
251                 "file_count": result["file_count"],

```

```

252         "archive": result["archive"],
253         "validation_passed": result["validation_passed"],
254         "clean_extraction_passed": result["clean_extraction_passed"],
255     },
256     indent=2,
257 )
258 )

```

B.4.4 scripts/prepare_dataset.py

This listing contains the complete file (180 source lines).

Listing 24: scripts/prepare_dataset.py

```

1  """Create original synthetic illustrations. Existing data are preserved by default."""
2
3  import argparse
4  import json
5  from pathlib import Path
6  import sys
7
8  from PIL import Image, ImageDraw, ImageEnhance
9
10 ROOT = Path(__file__).resolve().parents[1]
11 sys.path.insert(0, str(ROOT))
12
13 PRODUCTS = [
14     (1, "Nike Running Shoes", "Nike", "shoes", "black", 120, 10, "Black running shoes for everyday
15      training."),
16     (2, "Adidas Running Shoes", "Adidas", "shoes", "white", 100, 15, "White running shoes for daily
17      exercise."),
18     (3, "Black Leather Bag", "Local", "bag", "black", 80, 20, "Black leather shoulder bag with a carry
19      handle."),
20     (4, "Blue Sports Shoes", "Local", "shoes", "blue", 75, 12, "Blue sports shoes with a flexible
21      sole."),
22     (5, "Nike Budget Running Shoes", "Nike", "shoes", "black", 95, 8, "Affordable black running shoes
23      for training."),
24     (6, "Red Canvas Shoes", "Local", "shoes", "red", 60, 18, "Red canvas shoes for casual walks."),
25     (7, "White Casual Shoes", "Local", "shoes", "white", 45, 20, "White casual shoes for everyday
26      wear."),
27     (8, "Brown Leather Bag", "Local", "bag", "brown", 90, 6, "Brown leather shoulder bag with a carry
28      handle."),
29     (9, "Black Backpack", "Local", "bag", "black", 55, 10, "Black backpack with a front pocket."),
30     (10, "Blue Cotton T-Shirt", "Local", "clothing", "blue", 25, 30, "Blue cotton shirt with short
31        sleeves."),
32     (
33         11,
34         "Black Sports Jacket",
35         "Local",
36         "clothing",
37         "black",
38         85,
39         7,
40         "Black sports jacket with long sleeves and a zipper.",
41     ),
42     (12, "Green Travel Backpack", "Local", "bag", "green", 65, 0, "Green travel backpack with a front
43        pocket."),
44 ]
45
46 COLORS = {
47     "black": "#20242a",
48     "white": "#e5e7e9",

```

```

39     "blue": "#2563c5",
40     "red": "#ce3848",
41     "brown": "#98613e",
42     "green": "#398051",
43 }
44
45
46 def illustration(product):
47     image = Image.new("RGB", (256, 256), "#f6f2eb")
48     draw = ImageDraw.Draw(image)
49     color = COLORS[product["color"]]
50     outline = "#4b5158"
51     identifier = product["product_id"]
52     if product["category"] == "shoes":
53         shift = (identifier % 3) * 3
54         draw.polygon(
55             [
56                 (39, 159),
57                 (54, 141),
58                 (72, 131),
59                 (87, 88 + shift),
60                 (121, 93),
61                 (148, 129),
62                 (207, 149),
63                 (224, 176),
64                 (212, 187),
65                 (46, 187),
66             ],
67             fill=color,
68             outline=outline,
69             width=3,
70         )
71         draw.rounded_rectangle((43, 179, 221, 195), radius=8, fill="#d0d1d4", outline=outline, width=2)
72         for y in range(111, 142, 8):
73             draw.line((93, y, 127, y + 7), fill="#989a9d", width=3)
74             draw.line((149, 151, 178, 168), fill="#f3ce66" if identifier == 5 else "#8494a8", width=6)
75         elif "Backpack" in product["name"]:
76             draw.arc((100, 31, 156, 73), 180, 360, fill=outline, width=8)
77             draw.rounded_rectangle((65, 54, 193, 219), radius=35, fill=color, outline=outline, width=4)
78             draw.rounded_rectangle((82, 137, 176, 204), radius=16, fill=color, outline="#b2b5b9", width=3)
79             draw.line((91, 153, 166, 153), fill="#b2b5b9", width=3)
80         elif product["category"] == "bag":
81             draw.arc((81, 35, 177, 125), 180, 360, fill=color, width=13)
82             draw.rounded_rectangle((51, 87, 207, 209), radius=15, fill=color, outline=outline, width=4)
83             draw.line((60, 126, 198, 126), fill="#b2b5b9", width=3)
84             draw.rounded_rectangle((111, 114, 146, 139), radius=3, fill="#d0b174")
85         else:
86             long = "Jacket" in product["name"]
87             points = [
88                 (83, 55),
89                 (102, 47),
90                 (125, 66),
91                 (149, 47),
92                 (169, 55),
93                 (221, 172 if long else 94),
94                 (190, 187 if long else 122),
95                 (172, 113),
96                 (175, 217),
97                 (78, 217),
98                 (82, 113),
99                 (61, 187 if long else 122),
100                (29, 172 if long else 94),
101            ]
102             draw.polygon(points, fill=color, outline=outline, width=3)
103             if long:
104                 draw.line((126, 66, 126, 214), fill="#b2b5b9", width=4)

```

```

105     return image
106
107
108 def write_json(path, payload, force):
109     if path.exists() and not force:
110         return False
111     path.parent.mkdir(parents=True, exist_ok=True)
112     path.write_text(json.dumps(payload, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
113     return True
114
115
116 def prepare(root=ROOT, force=False):
117     root = Path(root)
118     products = [
119         dict(
120             zip(["product_id", "name", "brand", "category", "color", "price", "stock", "description"],
121                 row),
122             image=f"data/images/p{row[0]:03d}.png",
123         )
124         for row in PRODUCTS
125     ]
126     catalogue = root / "data/products.json"
127     if catalogue.exists() and not force:
128         products = json.loads(catalogue.read_text(encoding="utf-8"))
129     else:
130         write_json(catalogue, products, force)
131     for p in products:
132         if not (root / str(p["image"])).resolve().is_relative_to(root.resolve()):
133             raise ValueError("product image must stay inside dataset root")
134     for p in products:
135         path = root / str(p["image"])
136         if not path.exists() or force:
137             path.parent.mkdir(parents=True, exist_ok=True)
138             illustration(p).save(path)
139     queries = root / "data/queries"
140     queries.mkdir(parents=True, exist_ok=True)
141     for name, identifier in [("black_shoe_query", 1), ("blue_shoe_query", 4), ("brown_bag_query", 8)]:
142         path = queries / f"{name}.png"
143         if not path.exists() or force:
144             source = next(p for p in products if p["product_id"] == identifier)
145             with Image.open(root / str(source["image"])) as original:
146                 # Different pixels and resolution; fixed, mild transformation.
147                 transformed = ImageEnhance.Brightness(original).enhance(0.97).resize((224, 224))
148                 transformed.save(path)
149     if not (queries / "corrupt.png").exists() or force:
150         (queries / "corrupt.png").write_bytes(b"Intentional invalid image fixture for robustness evaluation.\n")
151     write_json(
152         root / "data/orders.json",
153         [
154             {"order_id": "0001", "customer_id": "C001", "date": "2026-09-29", "status": "Shipped",
155              "total": 150.0},
156             {"order_id": "0002", "customer_id": "C002", "date": "2026-09-28", "status": "Delivered",
157              "total": 80.0},
158             {"order_id": "0003", "customer_id": "C001", "date": "2026-10-01", "status": "Processing",
159              "total": 95.0},
160         ],
161         force,
162     )
163     write_json(
164         root / "data/dataset_metadata.json",
165         {
166             "origin": "Original synthetic Pillow illustrations authored for this assignment; not product photographs.",
167             "brands": "Names are educational sample metadata; no brand affiliation implied.",
168             "generator": "scripts/prepare_dataset.py",
169         },
170     )

```

```

165         "deterministic": True,
166         "currency": "USD",
167         "products": len(products),
168         "query_transform": "brightness 0.97, resize 224x224",
169         "corrupt_fixture": "data/queries/corrupt.png intentionally invalid",
170     },
171     force,
172 )
173 return len(products)
174
175
176 if __name__ == "__main__":
177     parser = argparse.ArgumentParser(description=__doc__)
178     parser.add_argument("--force", action="store_true", help="explicitly regenerate and overwrite dataset")
179     args = parser.parse_args()
180     print(f"Prepared {prepare(force=args.force)} products; existing outputs preserved unless --force.")

```

B.4.5 scripts/report_facts.py

This listing contains the complete file (195 source lines).

Listing 25: scripts/report_facts.py

```

1  """Generate report tables/listings from authoritative artifacts, never fabricated results."""
2
3  import hashlib
4  import json
5  from pathlib import Path
6  import shutil
7  import xml.etree.ElementTree as ET
8
9
10 def escape(value):
11     mapping = {"\\": r"\textbackslash{}", "&": r"&", "%": r"%", "$": r"$", "#": r"#",
12               "_": r"\_", "{": r"\{", "(": r"\(", "~": r"\textasciitilde{}", "^":
13               r"\textasciicircum{}}"}
14     return "".join(mapping.get(char, char) for char in str(value))
15
16 def table(headers, rows, widths):
17     columns = "".join(r">\raggedright\arraybackslash}p{" + width + "}" for width in widths)
18     header = " & ".join(r"\textbf{" + escape(h) + "}" for h in headers) + r" \\"
19     body = [r"\begin{group}\small\setlength{\tabcolsep}{4pt}\renewcommand{\arraystretch}{1.2}",
20            r"\begin{longtable}" + columns + "}", r"\toprule", header, r"\midrule\endfirsthead",
21            r"\toprule", header, r"\midrule\endhead", r"\bottomrule\endfoot"]
22     body += [" & ".join(escape(cell) for cell in row) + r" \\" for row in rows]
23     return "\n".join(body + [r"\end{longtable}\endgroup"])
24
25
26 def prepare_report_inputs(root, work):
27     root, work = Path(root).resolve(), Path(work).resolve()
28     hashes = {}
29
30     def load(relative):
31         path = root / relative
32         hashes[relative] = hashlib.sha256(path.read_bytes()).hexdigest()
33         return json.loads(path.read_text(encoding="utf-8-sig"))
34
35     products = load("data/products.json")
36     orders = load("data/orders.json")

```

```

37 metrics = load("artifacts/evaluation/metrics.json")
38 rows = load("artifacts/evaluation/results.json")
39 truth = load("evaluation/queries.json")
40 demos = load("artifacts/demo/demo_results.json")
41 junit_path = root / "artifacts/tests/junit.xml"
42 junit = ET.parse(junit_path).getroot()
43 suites = [junit] if junit.tag == "testsuite" else list(junit.iter("testsuite"))
44 tests = {k: sum(int(s.get(k, 0)) for s in suites) for k in ["tests", "failures", "errors",
45 "skipped"]}
46 coverage = load("artifacts/tests/coverage.json")["totals"]
47 tests.update(application_data_coverage=coverage["percent_covered"],
48 statements=coverage["num_statements"])
49 if tests["failures"] or tests["errors"] or tests["skipped"]:
50     raise ValueError("Test evidence is not fully passing")
51 if {r["id"] for r in rows} != {q["id"] for q in truth["queries"]}:
52     raise ValueError("Evaluation rows do not match the frozen query set")
53 if metrics["ground_truth_sha256"] != hashes["evaluation/queries.json"]:
54     raise ValueError("Frozen ground truth fingerprint mismatch")
55 for group, summary in metrics["groups"].items():
56     selected = [r for r in rows if r["group"] == group]
57     successful = sum(bool(r["success"]) for r in selected)
58     if (summary["total"], summary["successful"], summary["success_rate"]) != (
59         len(selected), successful, successful / len(selected)
60     ):
61         raise ValueError("Evaluation summary mismatch: " + group)
62 primary = metrics["groups"]["primary"]
63 if (metrics["total_queries"], metrics["successful_queries"], metrics["success_rate"]) != (
64     primary["total"], primary["successful"], primary["success_rate"]
65 ):
66     raise ValueError("Primary denominator mismatch")
67
68 def write(name, content):
69     (work / name).write_text(content + "\n", encoding="utf-8")
70
71 write("query_example.json", json.dumps(demos[0]["query"], indent=2, ensure_ascii=False))
72 write("result_example.json", json.dumps(demos[0]["results"][0], indent=2, ensure_ascii=False))
73
74 def macro(name, value):
75     return "\\newcommand{\\\" + name + \"}\"{\" + escape(value) + \"}"
76
77 write("facts.tex", "\n".join([
78     macro("ProductCount", len(products)), macro("OrderCount", len(orders)),
79     macro("TestCount", tests["tests"]), macro("StatementCount", tests["statements"]),
80     macro("CoveragePercent", f"{tests['application_data_coverage']:.2f}%"),
81     macro("MeanLatency", f"{metrics['timing_ms']['mean']:.3f}"),
82     macro("PninetyfiveLatency", f"{metrics['timing_ms']['p95']:.3f}"), macro("EvaluationRowCount",
83     len(rows)),
84 ]))
85 write("products_table.tex", table(
86     ["ID", "Product", "Category / color", "USD", "Stock"],
87     [(p["product_id"], p["name"], p["category"] + " / " + p["color"], f"{p['price']:.2f}",
88     p["stock"]) for p in products],
89     ["0.7cm", "6.0cm", "4.0cm", "1.4cm", "1.1cm"]))
90 write("orders_table.tex", table(
91     ["Order", "Customer", "Date", "Status", "Total USD"],
92     [(o["order_id"], o["customer_id"], o["date"], o["status"], f"{o['total']:.2f}") for o in
93     orders],
94     ["1.7cm", "2.0cm", "2.8cm", "4.4cm", "2.3cm"]))
95 write("metrics_table.tex", table(
96     ["Evaluation group", "Cases", "Successful", "Success rate"],
97     [(group.title(), m["total"], m["successful"], f"{100*m['success_rate']:.2f}%") for group, m in
98     metrics["groups"].items()],
99     ["6cm", "2cm", "2.5cm", "3.0cm"]))
100 write("mode_table.tex", table(
101     ["Primary modality", "Cases", "Successful", "Top-1 success"],

```

```

96     [(mode.title(), m["total"], m["successful"], f"{100*m['success_rate']:.2f}%") for mode, m in
    metrics["by_mode"].items()],
97     ["6cm", "2cm", "2.5cm", "3cm"])))
98 result_rows = []
99 ground_truth = {q["id"]: q for q in truth["queries"]}
100 for r in rows:
101     q = ground_truth[r["id"]]
102     input_text = q.get("query") or q.get("image") or q.get("order_id") or "zero vector (88D)"
103     if q.get("query") == "":
104         input_text = "empty text"
105     expectation = ", ".join(map(str, q.get("relevant_ids", [])))
106     if not expectation:
107         expectation = q.get("expected_order") or next(
108             (text for key, text in [("expected_error", "expected error"), ("expected_empty_order",
109                                     "empty order"),
110                                     ("expected_zero_scores", "zero scores")] if q.get(key)), "empty
111                                     results")
112     actual = str(r["top_1_product_id"]) if r["top_1_product_id"] is not None else (
113         "error" if r.get("error") else r.get("actual_order_id") or "none")
114     result_rows.append((r["id"], r["mode"], input_text, expectation, actual, "PASS" if r["success"]
115         else "FAIL"))
116 write("evaluation_table.tex", table(
117     ["ID", "Mode", "Input", "Relevant / expected", "Actual top-1", "Verdict"], result_rows,
118     ["0.65cm", "1.7cm", "5.0cm", "3.25cm", "1.5cm", "1.2cm"]))
119 demo_sections = []
120 for demo in demos:
121     mode = demo["query"]["type"]
122     demo_sections.append(r"\subsubsection{" + escape(mode.title()) + " demonstration output}")
123     if "results" in demo:
124         raw_input = demo["query"]["raw_input"].replace(str(root) + "\\", "").replace(str(root) +
125             "/", "")
126         demo_sections.append(r"\textbf{Input:} \texttt{" + escape(raw_input) + "}.")
127         processing = demo["processing"]
128         demo_sections.append(escape(
129             f"Normalized query: {processing['normalized_query'] or '(image only)'}." +
130             f"Tokens: {'', ' '.join(processing['tokens']) or 'none'}." +
131             f"Candidates before / after filtering: {processing['candidates_before_filter']} / "
132             f"{processing['candidates_after_filter']}; top-k: {processing['top_k']}. "
133             f"Category: {processing['filters']['category'] or 'none'}; "
134             f"text/image weights: {processing['ranking_weights']['text']} / "
135             f"{processing['ranking_weights']['image']}."
136         ))
137     demo_sections.append(table(
138         ["Rank / ID", "Product", "Text", "Image", "Final"],
139         [(f"{p['rank']} / {p['product_id']}", p["name"], f"{p['text_score']:.6f}",
140             f"{p['image_score']:.6f}", f"{p['final_score']:.6f}") for p in demo["results"]],
141         ["1.5cm", "6.0cm", "2.0cm", "2.0cm", "2.0cm"]))
142 else:
143     demo_sections.append("Order 0001 belongs to C001; observed status: " +
144         escape(demo["order"]["status"]) +
145         ", total USD " + escape(demo["order"]["total"]) + ".")
146 write("demo_tables.tex", "\n".join(demo_sections))
147
148 figures = [f"artifacts/diagrams/{name}.png" for name in ["use_case", "three_layer_architecture",
149     "voice_sequence"]]
150 figures += [p.relative_to(root).as_posix() for p in sorted((root /
151     "artifacts/screenshots").glob("*.png"))]
152 figures += [p["image"] for p in products]
153 figures += ["data/queries/" + name + ".png" for name in ["black_shoe_query", "blue_shoe_query",
154     "brown_bag_query"]]
155 galleries = []
156 for start in range(0, len(products), 6):
157     galleries.append(r"\begin{figure}[p]\centering")
158     for i, product in enumerate(products[start:start+6]):
159         galleries.append(r"\begin{minipage}[t]{0.31\textwidth}\centering\includegraphics[width=\lin
160             ewidth]{") +

```

```

151         product["image"] + "\\[3pt]\\small " +
152         escape(f"{product['product_id']:02d}. {product['name']}") +
            r"\end{minipage}")
153     galleries.append(r"\hfill if i % 3 != 2 else r"\par\vspace{10pt}")
154     galleries.append(r"\caption{Original synthetic catalogue illustrations, products " +
        str(start+1) +
155         "--" + str(min(start+6, len(products))) + r". Images are preserved from the
            approved dataset.}\end{figure}")
156 write("catalogue_gallery.tex", "\n".join(galleries))
157
158 listings, listing_tex = [], []
159 groups = {
160     "Entry point and presentation": [root / "main.py", *sorted((root /
        "presentation").glob("*.py"))],
161     "Application and intelligence": sorted((root / "application").glob("*.py")),
162     "Data access and vector index": sorted((root / "data").glob("*.py")),
163     "Dataset, evaluation, report and delivery utilities": sorted((root / "scripts").glob("*.py")),
164     "Desktop evidence and Visual Paradigm launcher": sorted((root /
        "scripts/desktop").glob("*.py")),
165     "Complete automated test suite": sorted((root / "tests").glob("*.py")),
166 }
167 for heading, paths in groups.items():
168     listing_tex.append((r"\clearpage" if listings else "") + r"\subsection{" + heading + "}")
169     for path in paths:
170         relative = path.relative_to(root).as_posix()
171         snapshot = work / "code_snapshot" / relative
172         snapshot.parent.mkdir(parents=True, exist_ok=True)
173         shutil.copyfile(path, snapshot)
174         data = path.read_bytes()
175         hashes[relative] = hashlib.sha256(data).hexdigest()
176         record = {"path": relative, "lines": len(data.decode("utf-8-sig").splitlines()), "sha256":
            hashes[relative]}
177         listings.append(record)
178         listing_tex.append(r"\subsubsection{\texorpdfstring{\texttt{" + escape(relative) + "}}{" +
            escape(relative) + "}}")
179         listing_tex.append("This listing contains the complete file (" + str(record["lines"]) + "
            source lines).")
180         listing_tex.append(r"\CodeFile{" + escape(relative) + "}" +
            snapshot.relative_to(root).as_posix() + ".")
181 write("python_listings.tex", "\n".join(listing_tex))
182 fixture_tex = []
183 for relative in ["data/products.json", "data/orders.json", "data/dataset_metadata.json",
    "evaluation/queries.json",
184     "requirements.txt", "requirements-dev.txt", "requirements-automation.txt",
        "pyproject.toml"]:
185     hashes[relative] = hashlib.sha256((root / relative).read_bytes()).hexdigest()
186     fixture_tex.append(r"\subsubsection{\texorpdfstring{\texttt{" + escape(relative) + "}}{" +
        escape(relative) + "}}")
187     fixture_tex.append(r"\lstinputlisting[language={},caption={" + escape(relative) + "}]{" +
        relative + ".")
188 write("fixture_listings.tex", "\n".join(fixture_tex))
189 preserved = {name: hashlib.sha256((root / name).read_bytes()).hexdigest()
190     for name in figures + ["models/Assignment_06_Multimodal_Search.vpp"]}
191 hashes.update(preserved)
192 hashes["artifacts/tests/junit.xml"] = hashlib.sha256(junit_path.read_bytes()).hexdigest()
193 (work / "listing_manifest.json").write_text(json.dumps(listings, indent=2) + "\n", encoding="utf-8")
194 return {"input_sha256": hashes, "figures": figures, "python_listings": listings,
195     "preserved_assets": preserved, "evaluation": metrics, "tests": tests}

```

B.4.6 scripts/run_evaluation.py

This listing contains the complete file (151 source lines).

Listing 26: scripts/run_evaluation.py

```

1  """Evaluate frozen ground truth against the actual application pipeline."""
2
3  import argparse
4  import csv
5  import hashlib
6  import json
7  import math
8  from pathlib import Path
9  import platform
10 import statistics
11 import sys
12 from time import perf_counter
13
14 ROOT = Path(__file__).resolve().parents[1]
15 sys.path.insert(0, str(ROOT))
16
17 from main import build_services
18
19
20 def percentile_nearest_rank(samples, probability):
21     return sorted(samples)[max(0, math.ceil(probability * len(samples)) - 1)]
22
23
24 def evaluate(queries_path, output):
25     queries_path, output = Path(queries_path), Path(output)
26     ground_truth_bytes = queries_path.read_bytes()
27     ground_truth = json.loads(ground_truth_bytes)
28     ui = build_services(ROOT)
29     rows = []
30     for case in ground_truth["queries"]:
31         started = perf_counter()
32         error, payload = None, {}
33         try:
34             if case["mode"] == "text":
35                 payload = ui.search_text(case["query"])
36             elif case["mode"] == "voice":
37                 payload = ui.search_voice(case["query"])
38             elif case["mode"] == "image":
39                 if case.get("embedding_zero"):
40                     payload = ui.search_service.search(
41                         ui.query_service.image_query([0.0] * 88, raw_input="zero-vector robustness
42                         fixture")
43                     )
44                 else:
45                     payload = ui.search_image(ROOT / case["image"])
46             elif case["mode"] == "multimodal":
47                 payload = ui.search_multimodal(case["query"], ROOT / case["image"])
48             else:
49                 payload = ui.search_order(case["order_id"], case["customer_id"])
50         except ValueError as exc:
51             error = str(exc)
52             elapsed = (perf_counter() - started) * 1000
53             results = payload.get("results", [])
54             ids = [p["product_id"] for p in results]
55             if case.get("expected_error"):
56                 success = error is not None
57             elif case.get("expected_empty"):
58                 success = error is None and not ids
59             elif case.get("expected_zero_scores"):
60                 success = error is None and bool(results) and all(p["image_score"] == 0 for p in results)
61             elif case.get("expected_empty_order"):
62                 success = error is None and payload.get("order") is None
63             elif "expected_order" in case:

```

```

63         success = error is None and (payload.get("order") or {}).get("order_id") ==
        case["expected_order"]
64     else:
65         success = error is None and bool(ids) and ids[0] in case["relevant_ids"]
66     rows.append(
67         {
68             "id": case["id"],
69             "group": case["group"],
70             "mode": case["mode"],
71             "input": {k: v for k, v in case.items() if k not in {"id", "group", "relevant_ids"}},
72             "relevant_ids": case.get("relevant_ids", []),
73             "top_1_product_id": ids[0] if ids else None,
74             "top_k_ids": ids,
75             "scores": [p["final_score"] for p in results],
76             "actual_order_id": (payload.get("order") or {}).get("order_id"),
77             "success": success,
78             "processing_ms": elapsed,
79             "error": error,
80             "processing": payload.get("processing"),
81             "note": case.get("note", ""),
82         }
83     )
84
85     def summarize(selected):
86         total = len(selected)
87         successful = sum(r["success"] for r in selected)
88         return {"total": total, "successful": successful, "success_rate": successful / total if total
            else 0.0}
89
90     primary = [r for r in rows if r["group"] == "primary"]
91     primary_summary = summarize(primary)
92     times = sorted(r["processing_ms"] for r in primary)
93     metrics = {
94         "total_queries": primary_summary["total"],
95         "successful_queries": primary_summary["successful"],
96         "success_rate": primary_summary["success_rate"],
97         "criterion": ground_truth["criterion"],
98         "by_mode": {mode: summarize([r for r in primary if r["mode"] == mode]) for mode in ["text",
99             "voice", "image"]},
100         "groups": {
101             group: summarize([r for r in rows if r["group"] == group])
102             for group in ["primary", "extension", "challenge", "robustness"]
103         },
104         "encoder": ui.image_service.encoder,
105         "ranking": "weighted normalized text/image scores; ties product_id ascending; business weight
106             0",
107         "ground_truth_version": ground_truth["version"],
108         "ground_truth_sha256": hashlib.sha256(ground_truth_bytes).hexdigest(),
109         "timing_ms": {
110             "mean": statistics.mean(times),
111             "p95": percentile_nearest_rank(times, 0.95),
112             "max": max(times),
113             "percentile_method": "nearest rank ceil(p*n)",
114             "startup_excluded": True,
115             "samples": len(times),
116         },
117         "environment": {"python": platform.python_version(), "platform": platform.platform()},
118         "incorrect_examples": [r for r in rows if not r["success"]],
119     }
120     output.mkdir(parents=True, exist_ok=True)
121     (output / "results.json").write_text(json.dumps(rows, indent=2, allow_nan=False) + "\n",
122         encoding="utf-8")
123     (output / "metrics.json").write_text(json.dumps(metrics, indent=2, allow_nan=False) + "\n",
124         encoding="utf-8")
125     fields = [
126         "id",

```

```

123     "group",
124     "mode",
125     "input",
126     "relevant_ids",
127     "top_1_product_id",
128     "top_k_ids",
129     "scores",
130     "actual_order_id",
131     "success",
132     "processing_ms",
133     "error",
134     "note",
135 ]
136 with (output / "results.csv").open("w", encoding="utf-8", newline="") as handle:
137     writer = csv.DictWriter(handle, fieldnames=fields, extrasaction="ignore")
138     writer.writeheader()
139     for row in rows:
140         writer.writerow(
141             {k: json.dumps(v, ensure_ascii=False) if isinstance(v, (list, dict)) else v for k, v in
142              row.items()})
143     return metrics
144
145
146 if __name__ == "__main__":
147     parser = argparse.ArgumentParser(description=__doc__)
148     parser.add_argument("--queries", type=Path, default=ROOT / "evaluation/queries.json")
149     parser.add_argument("--output", type=Path, default=ROOT / "artifacts/evaluation")
150     args = parser.parse_args()
151     print(json.dumps(evaluate(args.queries, args.output), indent=2))

```

B.4.7 scripts/validate_submission.py

This listing contains the complete file (348 source lines).

Listing 27: scripts/validate_submission.py

```

1  """Verify assignment deliverables from files and actual service behaviour."""
2
3  import argparse
4  from collections import Counter
5  import hashlib
6  import json
7  from pathlib import Path
8  import sys
9  import xml.etree.ElementTree as ET
10
11 import pymupdf as fitz
12 from PIL import Image
13
14 ROOT = Path(__file__).resolve().parents[1]
15 sys.path.insert(0, str(ROOT))
16
17 from data.product_repository import ProductRepository
18 from data.vector_index import VectorIndex
19
20 REQUIRED = [
21     "README.md",
22     "main.py",
23     "requirements.txt",
24     "requirements-dev.txt",

```

```

25     "pyproject.toml",
26     "docs/requirements.md",
27     "docs/use_cases.md",
28     "docs/architecture.md",
29     "docs/traceability.md",
30     "docs/report.md",
31     "docs/report.tex",
32     "docs/report_requirements_audit.md",
33     "docs/decisions_and_limitations.md",
34     "docs/testing/prototype.tdd.md",
35     "data/products.json",
36     "data/orders.json",
37     "data/embeddings.json",
38     "evaluation/queries.json",
39     "models/Assignment_06_Multimodal_Search.vpp",
40     "artifacts/report/Assignment_06_Report.pdf",
41     "artifacts/diagrams/use_case.png",
42     "artifacts/diagrams/three_layer_architecture.png",
43     "artifacts/diagrams/voice_sequence.png",
44     "artifacts/screenshots/vp_use_case.png",
45     "artifacts/screenshots/vp_architecture.png",
46     "artifacts/screenshots/vp_sequence.png",
47     "artifacts/screenshots/demo_text.png",
48     "artifacts/screenshots/demo_voice.png",
49     "artifacts/screenshots/demo_image.png",
50     "artifacts/demo/demo_output.txt",
51     "artifacts/demo/demo_results.json",
52     "artifacts/evaluation/metrics.json",
53     "artifacts/evaluation/results.json",
54     "artifacts/evaluation/results.csv",
55     "artifacts/tests/junit.xml",
56     "artifacts/tests/coverage.json",
57     "artifacts/tests/test_output.txt",
58     "artifacts/tests/lint.txt",
59     "artifacts/tests/typecheck.txt",
60     "artifacts/tests/pip-check.txt",
61     "artifacts/automation/model_inventory.json",
62     "artifacts/automation/reopened_inventory.json",
63     "artifacts/automation/verify_result.txt",
64     "scripts/prepare_dataset.py",
65     "scripts/build_index.py",
66     "scripts/run_evaluation.py",
67     "scripts/build_report.py",
68     "scripts/report_facts.py",
69     "scripts/validate_submission.py",
70     "scripts/package_submission.py",
71 ]
72
73
74 def load_json(root, relative):
75     return json.loads((root / relative).read_text(encoding="utf-8-sig"))
76
77
78 def read_evidence_text(path):
79     data = path.read_bytes()
80     return data.decode("utf-16" if data.startswith((b"\xff\xfe", b"\xfe\xff")) else "utf-8-sig")
81
82
83 def validate(root=ROOT):
84     root = Path(root).resolve()
85     checks = []
86
87     def check(name, function):
88         try:
89             details = function()
90             checks.append({"name": name, "passed": True, "details": details})

```

```

91     except (
92         AssertionError,
93         OSError,
94         ValueError,
95         KeyError,
96         TypeError,
97         ET.ParseError,
98         fitz.FileDataError,
99         fitz.FileNotFoundError,
100     ) as exc:
101         checks.append({"name": name, "passed": False, "error": str(exc)})
102
103 def required_files():
104     missing = [p for p in REQUIRED if not (root / p).is_file() or (root / p).stat().st_size == 0]
105     assert not missing, "Missing or empty required files: " + ", ".join(missing)
106     assert list((root / "tests").glob("test_*.py")), "No tests present"
107     return {"required_files": len(REQUIRED)}
108
109 check("Required submission groups and evidence", required_files)
110
111 def catalogue_and_index():
112     products = ProductRepository(root=root).all_products()
113     assert len(products) >= 10, "Fewer than 10 products"
114     index = VectorIndex(root / "data/embeddings.json", products, root)
115     for product in products:
116         with Image.open(root / product["image"]) as image:
117             image.verify()
118     return {"products": len(products), "encoder": index.encoder, "source_fingerprint_valid": True}
119
120 check("Dataset, real images, and matching pixel index", catalogue_and_index)
121
122 def report():
123     report_path = root / "artifacts/report/Assignment_06_Report.pdf"
124     assert report_path.is_file(), "Final PDF is not present yet"
125     with fitz.open(report_path) as pdf:
126         assert len(pdf) >= 11, f"Detailed report is unexpectedly short: {len(pdf)} pages"
127         empty = [i + 1 for i in range(len(pdf)) if not str(pdf[i].get_text("text")).strip()]
128         assert not empty, f"Empty report pages: {empty}"
129         embedded_images = sum(len(pdf[i].get_images()) for i in range(len(pdf)))
130         audit = load_json(root, "artifacts/report/build_audit.json")
131         assert audit["format"] == "assignment06-latex-report-v2", "Expected the complete LaTeX report"
132         assert audit["pages"] == len(pdf) and audit["language"] == "English"
133         assert hashlib.sha256(report_path.read_bytes()).hexdigest() == audit["pdf_sha256"], "Stale PDF audit"
134         for relative, expected in audit["input_sha256"].items():
135             assert hashlib.sha256((root / relative).read_bytes()).hexdigest() == expected, f"Stale report source: {relative}"
136         assert audit["all_required_sections_present"] and len(audit["section_pages"]) == 11
137         assert audit["approved_UML_and_images_unchanged"] and
138             audit["latex_layout_and_references_clean"]
139         assert audit["all_approved_image_pixels_matched"] and len(audit["figure_xrefs"]) == 25
140         assert not audit["nonblack_text"] and not audit["text_outside_safe_bounds"]
141         assert all(audit["embedded_fonts"].values()), "Unembedded report font"
142         assert embedded_images >= 25 and audit["unique_embedded_images"] >= 25, "Missing approved figures"
143         paths = {r["path"] for r in audit["python_listings"]}
144         actual = {p.relative_to(root).as_posix() for folder in ["application", "presentation",
145             "data", "scripts", "tests"]
146             for p in (root / folder).rglob("*.py") if "__pycache__" not in p.parts}
147         actual.add("main.py")
148         assert paths == actual, "Complete Python listing inventory is missing files"
149         return {"pages": len(pdf), "embedded_images": embedded_images, "nonempty_text_pages": True,
150             "English_LaTeX": True, "full_Python_files": len(paths), "page_limit": None}
151
152 check("Complete English LaTeX PDF, full Python listings and approved visual evidence", report)

```

```

151
152 def diagrams_and_screenshots():
153     relatives = [p for p in REQUIRED if p.endswith(".png")]
154     sizes = {}
155     for relative in relatives:
156         with Image.open(root / relative) as image:
157             assert image.width >= 400 and image.height >= 250, f"Unreadably small image: {relative}"
158             sizes[relative] = [image.width, image.height]
159             image.verify()
160     return sizes
161
162 check("UML exports and real screenshot image files", diagrams_and_screenshots)
163
164 def native_model_evidence():
165     path = root / "models/Assignment_06_Multimodal_Search.vpp"
166     assert path.stat().st_size > 10_000, "VPP is unexpectedly small"
167     inventory = load_json(root, "artifacts/automation/model_inventory.json")
168     reopened = load_json(root, "artifacts/automation/reopened_inventory.json")
169     assert inventory == reopened, "Final inventory differs from reopened project inventory"
170     assert "PASS reopened native diagrams=3" in read_evidence_text(
171         root / "artifacts/automation/verify_result.txt"
172     ), "Native project reopen did not pass"
173     diagrams = {d["name"]: d for d in inventory["diagrams"]}
174     expected = {"UC_Multimodal_Ecommerce_Search", "CMP_Three_Layer_Architecture",
175         "SEQ_Voice_Product_Search"}
176     assert expected <= diagrams.keys(), "Required native diagrams are missing"
177     uc = diagrams["UC_Multimodal_Ecommerce_Search"]
178     names = {s["name"] for s in uc["shapes"]}
179     assert {
180         "Customer",
181         "Search Product",
182         "Search by Keyword",
183         "Search by Voice",
184         "Search by Image",
185         "Search Order",
186         "View Product",
187         "View Order",
188     } <= names, "Required actor/use cases missing"
189     generalizations = [c for c in uc["connectors"] if c["type"] == "Generalization"]
190     associations = [c for c in uc["connectors"] if c["type"] == "Association"]
191     # VP API Generalization.from is the general end, where the triangle points.
192     assert len(generalizations) == 3 and {(c["from"], c["to"]) for c in generalizations} == {
193         ("Search Product", mode) for mode in ["Search by Keyword", "Search by Voice", "Search by
194         Image"]
195     }, "Incorrect use-case general/special ends"
196     assert len(associations) == 4 and {(c["from"], c["to"]) for c in associations} == {
197         ("Customer", name) for name in ["Search Product", "Search Order", "View Product", "View
198         Order"]
199     }, "Incorrect Customer associations"
200     component = diagrams["CMP_Three_Layer_Architecture"]
201     package_names = {s["name"] for s in component["shapes"] if s["type"] == "Package"}
202     assert {"Presentation Layer", "Application / Intelligence Layer", "Data Layer"} <=
203     package_names, (
204         "Required layer packages missing"
205     )
206     ownership = {s["name"]: s["parent"] for s in component["shapes"] if s["type"] == "Component"}
207     expected_ownership = {
208         **{name: "Presentation Layer" for name in ["SearchUI", "VoiceInput", "ImageUpload",
209         "SearchResultView"]},
210         **{
211             name: "Application / Intelligence Layer"
212             for name in [
213                 "QueryService",
214                 "SpeechService",
215                 "ImageService",
216                 "SearchService",

```

```

212         "RankingService",
213         "OrderService",
214     ]
215 },
216 **{
217     name: "Data Layer"
218     for name in [
219         "ProductRepository",
220         "OrderRepository",
221         "VectorIndex",
222         "ProductDatabase",
223         "OrderDatabase",
224         "ImageStorage",
225     ]
226 },
227 }
228 assert ownership == expected_ownership, "Incorrect component names/package ownership"
229 forbidden = [
230     c
231     for c in component["connectors"]
232     if c["type"] == "Dependency"
233     and ownership.get(c["from"]) == "Presentation Layer"
234     and ownership.get(c["to"]) == "Data Layer"
235 ]
236 assert not forbidden, "Direct Presentation-to-Data dependency found"
237 seq = diagrams["SEQ_Voice_Product_Search"]
238 participant_types = Counter(s["type"] for s in seq["shapes"])
239 assert (
240     participant_types["InteractionActor"] == 1
241     and participant_types["InteractionLifeLine"] == 6
242     and participant_types["Activation"] == 6
243 ), "Voice participants missing"
244 messages = {c["name"] for c in seq["connectors"]}
245 assert len(seq["connectors"]) == 14 and all(c["type"] == "Message" for c in seq["connectors"]),
246 (
247     "Voice sequence needs exactly 14 native messages"
248 )
249 assert {
250     "transcribe(transcriptInput)",
251     "voice_query(transcript)",
252     "search(query)",
253     "all_products()",
254     "rank(candidates, query)",
255 } <= messages, "Voice messages missing"
256 return {
257     "native_diagram_names": sorted(expected),
258     "use_case_elements": len(names),
259     "generalizations": len(generalizations),
260     "customer_associations": len(associations),
261     "component_package_ownership": ownership,
262     "presentation_to_data_dependencies": len(forbidden),
263     "voice_messages": len(seq["connectors"]),
264     "voice_participant_counts": dict(participant_types),
265     "note": "Inventory verifies native model types; GUI reopen/layout verification is recorded separately by root.",
266 }
267
268 check("Native VP model inventory and required UML semantics", native_model_evidence)
269
270 def demos():
271     payload = load_json(root, "artifacts/demo/demo_results.json")
272     modes = {r["query"]["type"] for r in payload}
273     assert {"text", "voice", "image", "multimodal", "order"} <= modes, "Required demo modalities missing"
274     for item in payload:
275         if "results" in item:

```

```

275         assert item["results"] and item["processing"], "Demo needs actual processing/results"
276         assert all("final_score" in r for r in item["results"]), "Ranking scores missing"
277     assert "simulated speech-to-text" in read_evidence_text(root /
278         "artifacts/demo/demo_output.txt"), (
279         "Voice simulation is not disclosed"
280     )
281     return {"modes": sorted(modes), "queries": len(payload)}
282
283 check("Three mandatory demo modes, fusion, order, and scores", demos)
284
285 def evaluation():
286     ground_truth = load_json(root, "evaluation/queries.json")
287     metrics = load_json(root, "artifacts/evaluation/metrics.json")
288     rows = load_json(root, "artifacts/evaluation/results.json")
289     expected_sha = hashlib.sha256((root / "evaluation/queries.json").read_bytes()).hexdigest()
290     assert metrics["ground_truth_sha256"] == expected_sha, "Evaluation used different labels"
291     assert {c["id"] for c in ground_truth["queries"]} == {r["id"] for r in rows}, "Evaluation is
292     missing cases"
293     primary = [r for r in rows if r["group"] == "primary"]
294     assert len(primary) >= 12, "At least 12 primary retrieval queries required"
295     assert Counter(r["mode"] for r in primary) == {"text": 4, "voice": 4, "image": 4}, (
296         "Primary mode breakdown incorrect"
297     )
298     for group, summary in metrics["groups"].items():
299         selected = [r for r in rows if r["group"] == group]
300         successful = sum(r["success"] for r in selected)
301         assert summary["total"] == len(selected) and summary["successful"] == successful, (
302             f"Incorrect {group} denominator/count"
303         )
304         assert abs(summary["success_rate"] - successful / len(selected)) < 1e-12, f"Incorrect
305         {group} rate"
306     assert metrics["total_queries"] == len(primary), "Primary metric denominator mismatch"
307     assert metrics["successful_queries"] == sum(r["success"] for r in primary), "Primary numerator
308     mismatch"
309     assert metrics["incorrect_examples"], "Observed failure examples must be retained"
310     return {
311         "primary": metrics["groups"]["primary"],
312         "separate_groups": metrics["groups"],
313         "ground_truth_sha256": expected_sha,
314     }
315
316 check("Frozen ground truth, metrics, separate denominators, and failures", evaluation)
317
318 def tests_and_checks():
319     junit = ET.parse(root / "artifacts/tests/junit.xml").getroot()
320     suites = [junit] if junit.tag == "testsuite" else list(junit.iter("testsuite"))
321     total = sum(int(s.get("tests", 0)) for s in suites)
322     failures = sum(int(s.get("failures", 0)) + int(s.get("errors", 0)) for s in suites)
323     skipped = sum(int(s.get("skipped", 0)) for s in suites)
324     assert total >= 70 and failures == 0 and skipped == 0, (
325         f"Unmet tests: total={total}, failures/errors={failures}, skipped={skipped}"
326     )
327     coverage = load_json(root, "artifacts/tests/coverage.json")["totals"]["percent_covered"]
328     assert coverage >= 80, f"Application/data coverage {coverage} <80%"
329     for path, marker in [
330         ("lint.txt", "All checks passed!"),
331         ("typecheck.txt", "0 errors"),
332         ("pip-check.txt", "No broken requirements found"),
333     ]:
334         assert marker in read_evidence_text(root / "artifacts/tests" / path), f"{path} does not show
335         PASS"
336     return {"tests": total, "failures": failures, "skipped": skipped,
337         "application_data_line_coverage": coverage}
338
339 check("Test, coverage, lint, typecheck, and dependency consistency evidence", tests_and_checks)
340 return {"passed": all(c["passed"] for c in checks), "root": str(root), "checks": checks}

```

```

335
336
337 if __name__ == "__main__":
338     parser = argparse.ArgumentParser(description=__doc__)
339     parser.add_argument("--root", type=Path, default=ROOT)
340     parser.add_argument("--output", type=Path)
341     args = parser.parse_args()
342     result = validate(args.root)
343     serialized = json.dumps(result, indent=2, ensure_ascii=False)
344     if args.output:
345         args.output.parent.mkdir(parents=True, exist_ok=True)
346         args.output.write_text(serialized + "\n", encoding="utf-8")
347     print(serialized)
348     raise SystemExit(0 if result["passed"] else 1)

```

B.4.8 scripts/verify_uuml_layout.py

This listing contains the complete file (155 source lines).

Listing 28: scripts/verify_uuml_layout.py

```

1  """Audit VP-exported geometry; sequence lifeline crossings are intentional UML."""
2
3  import argparse
4  import hashlib
5  from itertools import combinations
6  import json
7  from pathlib import Path
8
9  ROOT = Path(__file__).resolve().parents[1]
10 BLACK = "java.awt.Color[r=0,g=0,b=0]"
11 VP_FILL = "java.awt.Color[r=122,g=207,b=245]" # Observed native factory default.
12
13
14 def segments(points):
15     return [(a, b) for a, b in zip(points, points[1:]) if a != b]
16
17
18 def intersection(first, second):
19     a, b = first
20     c, d = second
21     horizontal = a[1] == b[1]
22     other_horizontal = c[1] == d[1]
23     if horizontal == other_horizontal:
24         fixed, moving = (1, 0) if horizontal else (0, 1)
25         if a[fixed] != c[fixed]:
26             return None
27         lo = max(min(a[moving], b[moving]), min(c[moving], d[moving]))
28         hi = min(max(a[moving], b[moving]), max(c[moving], d[moving]))
29         if lo < hi:
30             return "collinear overlap"
31         if lo == hi:
32             return "touch"
33         return None
34     h, v = (first, second) if horizontal else (second, first)
35     x, y = v[0][0], h[0][1]
36     if min(h[0][0], h[1][0]) <= x <= max(h[0][0], h[1][0]) and min(
37         v[0][1], v[1][1]
38     ) <= y <= max(v[0][1], v[1][1]):
39         return "crossing"
40     return None

```

```

41
42
43 def penetrates(segment, bounds):
44     a, b = segment
45     x, y, w, h = bounds
46     if a[1] == b[1]:
47         return y < a[1] < y + h and max(min(a[0], b[0]), x) < min(
48             max(a[0], b[0]), x + w
49         )
50     return x < a[0] < x + w and max(min(a[1], b[1]), y) < min(
51         max(a[1], b[1]), y + h
52     )
53
54
55 def semantic_signature(inventory):
56     return {
57         d["id"]: {
58             "name": d["name"],
59             "shapes": sorted(
60                 (s["id"], s["type"], s["name"], s["parent"]) for s in d["shapes"])
61             ),
62             "relationships": sorted(
63                 (c["id"], c["type"], c["name"], c["from"], c["to"])
64                 for c in d["connectors"])
65             ),
66         }
67         for d in inventory["diagrams"]
68     }
69
70
71 def audit(inventory, baseline=None):
72     findings = []
73     details = []
74     expected = {"UC_": (9, 7), "SEQ_": (14, 14), "CMP_": (20, 15)}
75     if len(inventory["diagrams"]) != 3:
76         findings.append("Expected exactly three native diagrams")
77     for diagram in inventory["diagrams"]:
78         name = diagram["name"]
79         prefix = next((p for p in expected if name.startswith(p)), None)
80         if prefix is None or expected[prefix] != (
81             len(diagram["shapes"]), len(diagram["connectors"])
82         ):
83             findings.append(f"{name}: unexpected object counts")
84         for shape in diagram["shapes"]:
85             if shape["fill"] != VP_FILL or any(
86                 shape[key] != BLACK for key in ("line_color", "font_color")
87             ):
88                 findings.append(f"{name}: nondefault shape colors: {shape['name']}")
89         for connector in diagram["connectors"]:
90             label = f"{connector['from']} -> {connector['to']} ({connector['name']})"
91             if any(connector[key] != BLACK for key in ("line_color", "font_color")):
92                 findings.append(f"{name}: nondefault connector colors: {label}")
93             points = connector["points"]
94             if len(points) < 2 or any(
95                 a[0] != b[0] and a[1] != b[1] for a, b in segments(points)
96             ):
97                 findings.append(f"{name}: connector is not orthogonal: {label}")
98     pair_findings = []
99     # Lifelines intersect messages by design; audit only UC/component crossings.
100     if prefix in {"UC_", "CMP_"}:
101         for first, second in combinations(diagram["connectors"], 2):
102             issues = {
103                 hit
104                 for a in segments(first["points"])
105                 for b in segments(second["points"])
106                 if (hit := intersection(a, b))

```

```

107         }
108         if issues:
109             pair_findings.append(
110                 {"first": first["id"], "second": second["id"], "issues": sorted(issues)}
111             )
112         for connector in diagram["connectors"]:
113             for shape in diagram["shapes"]:
114                 if shape["type"] in {"Package", "System", "NOTE"} or shape[
115                     "name"
116                 ] in {connector["from"], connector["to"]}:
117                     continue
118                 if any(penetrates(s, shape["bounds"]) for s in segments(connector["points"])):
119                     findings.append(f"{name}: edge {connector['id']} crosses {shape['name']}")
120         if pair_findings:
121             findings.append(f"{name}: {len(pair_findings)} connector pairs cross/overlap/touch")
122         details.append(
123             {"diagram": name, "shapes": len(diagram["shapes"]),
124              "connectors": len(diagram["connectors"]), "connector_conflicts": pair_findings}
125         )
126         unchanged = semantic_signature(baseline) == semantic_signature(inventory) if baseline else None
127         if unchanged is False:
128             findings.append("Native model IDs or UML semantics changed")
129         return {
130             "passed": not findings, "native_semantics_compared": baseline is not None,
131             "native_semantics_unchanged": unchanged, "fill_default_observed_in_VP": "#7ACFF5",
132             "outline_and_text_default": "#000000", "diagrams": details, "findings": findings,
133             "scope": "Axis-aligned connectors in all diagrams; intersections and nonendpoint box penetration
134                     in UC/CMP. Sequence lifeline intersections are intentional.",
135         }
136
137     def main():
138         parser = argparse.ArgumentParser(description=__doc__)
139         parser.add_argument("--inventory", type=Path, default=ROOT /
140                             "artifacts/automation/model_inventory.json")
141         parser.add_argument("--baseline", type=Path)
142         parser.add_argument("--output", type=Path, default=ROOT / "artifacts/tests/uml_layout_audit.json")
143         args = parser.parse_args()
144         result = audit(json.loads(args.inventory.read_text(encoding="utf-8")),
145                        json.loads(args.baseline.read_text(encoding="utf-8")) if args.baseline else None)
146         result["inventory_sha256"] = hashlib.sha256(args.inventory.read_bytes()).hexdigest()
147         native = ROOT / "models/Assignment_06_Multimodal_Search.vpp"
148         result["native_project_sha256"] = hashlib.sha256(native.read_bytes()).hexdigest()
149         args.output.parent.mkdir(parents=True, exist_ok=True)
150         args.output.write_text(json.dumps(result, indent=2) + "\n", encoding="utf-8")
151         print(json.dumps(result, indent=2))
152         raise SystemExit(0 if result["passed"] else 1)
153
154     if __name__ == "__main__":
155         main()

```

B.5 Desktop evidence and Visual Paradigm launcher

B.5.1 scripts/desktop/capture_demo.py

This listing contains the complete file (65 source lines).

Listing 29: scripts/desktop/capture_demo.py

```

1  """Capture a real Python result viewer; every displayed value comes from CLI JSON."""
2  import json
3  from pathlib import Path
4  import subprocess
5  import sys
6  import tkinter as tk
7
8  import pyautogui
9
10 ROOT = Path(__file__).resolve().parents[2]
11 QUERIES = {
12     "text": ["--query", "black shoes"],
13     "voice": ["--query", "find black running shoes"],
14     "image": ["--image", "data/queries/black_shoe_query.png"],
15     "multimodal": ["--query", "black shoes", "--image", "data/queries/black_shoe_query.png"],
16 }
17
18
19 def main():
20     mode = sys.argv[1]
21     command = [sys.executable, "main.py", "--mode", mode, *QUERIES[mode], "--top-k", "3", "--json"]
22     completed = subprocess.run(command, cwd=ROOT, capture_output=True, text=True, check=True)
23     payload = json.loads(completed.stdout)
24     p = payload["processing"]
25     input_label = payload['query']['raw_input'].replace(str(ROOT) + "\\", "")
26     lines = [f"INPUT: {input_label}"]
27     if mode == "voice":
28         lines.append("Speech-to-text: SIMULATED (transcript supplied)")
29     if p["tokens"]:
30         lines.append("Tokens: " + ", ".join(p["tokens"]))
31     if p["encoder"]:
32         lines.append(f"Encoder: pixels, {p['encoder']['dimension']}D | similarity: cosine")
33     lines += [f"Candidates: {p['candidates_before_filter']} -> {p['candidates_after_filter']} | top-k: 3",
34             "RESULTS", "text", "image", "final"]
35     for row in payload["results"]:
36         lines.append(f"{row['rank']}. [{row['product_id']:02d}] {row['name']:<27} "
37                     f"{row['text_score']:.3f} {row['image_score']:.3f} {row['final_score']:.3f}")
38     lines.append("CLI exit=0 | values read directly from returned JSON")
39     folder = ROOT / "artifacts/screenshots"
40     folder.mkdir(parents=True, exist_ok=True)
41     (folder / f"demo_{mode}_payload.json").write_text(completed.stdout, encoding="utf-8")
42     window = tk.Tk()
43     window.title(f"Assignment 06 - Python {mode.upper()} demo - actual CLI output")
44     window.geometry(f"1030x{30 + 36 * len(lines)}+430+300")
45     window.attributes("-topmost", True)
46     text = tk.Text(window, bg="#101821", fg="#e8f1f5", font=("Consolas", 18), padx=18, pady=14,
47                   borderwidth=0, wrap="none")
48     text.pack(fill="both", expand=True)
49     text.insert("1.0", "\n".join(lines))
50     text.configure(state="disabled")
51
52 def capture():
53     window.update_idletasks()
54     x, y = window.winfo_rootx(), window.winfo_rooty()
55     pyautogui.screenshot(region=(x, y - 31, window.winfo_width(), window.winfo_height() + 31)).save(

```

```

56         folder / f"demo_{mode}.png")
57         window.destroy()
58
59     window.after(1200, capture)
60     window.mainloop()
61     print(f"PASS real {mode} screenshot and raw CLI payload")
62
63
64 if __name__ == "__main__":
65     main()

```

B.5.2 scripts/desktop/run_vp_plugin.py

This listing contains the complete file (64 source lines).

Listing 30: scripts/desktop/run_vp_plugin.py

```

1  """Compile and invoke this assignment's plugin in its own VP workspace.
2
3  Keeps the already open Visual Paradigm instance and other installed plugins intact.
4  """
5  from pathlib import Path
6  import argparse
7  import os
8  import shutil
9  import subprocess
10 import time
11
12 ROOT = Path(__file__).resolve().parents[2]
13 VP = Path(r"C:\Program Files\Visual Paradigm CE 18.0")
14 PLUGIN = ROOT / "scripts/desktop/vp_plugin"
15 PLUGIN_ID = "edu.assignment06.multimodal"
16
17
18 def main():
19     parser = argparse.ArgumentParser()
20     parser.add_argument("action", choices=["smoke", "build", "verify", "refine"])
21     parser.add_argument("--project", type=Path)
22     args = parser.parse_args()
23     classes = PLUGIN / "classes"
24     classes.mkdir(parents=True, exist_ok=True)
25     subprocess.run([
26         "javac", "-encoding", "UTF-8", "-source", "8", "-target", "8",
27         "-classpath", str(VP / "lib/openapi.jar"), "-d", str(classes),
28         *map(str, (PLUGIN / "src").rglob("*.java")),
29     ], check=True)
30     installed = Path(os.environ["APPDATA"]) / "VisualParadigm/plugins/assignment06-multimodal-builder"
31     installed.mkdir(parents=True, exist_ok=True)
32     shutil.copy2(PLUGIN / "plugin.xml", installed / "plugin.xml")
33     shutil.copytree(classes, installed / "classes", dirs_exist_ok=True)
34     work = ROOT / "artifacts/automation"
35     work.mkdir(parents=True, exist_ok=True)
36     (work / "vp_workspace").mkdir(parents=True, exist_ok=True)
37     seed = work / "native_seed.vpp"
38     if not seed.exists():
39         shutil.copy2(VP / "resources/BaggageSchemas.vpp", seed)
40     project = args.project.resolve() if args.project else seed
41     if not project.is_file():
42         raise FileNotFoundError(project)
43     if args.action in {"build", "smoke"}:
44         disposable = work / "launch_seed.vpp"

```

```
45     shutil.copy2(project, disposable)
46     project = disposable
47     command = [
48         str(VP / "jre/bin/java.exe"), "-Xmx1024m",
49         f"-Dassignment06.root={ROOT}",
50         "-cp", f"{VP / 'lib/*'};{VP / 'ormlib/*'}",
51         "com.vp.cmd.Plugin", "-workspace", str(work / "vp_workspace"),
52         "-project", str(project), "-upgrade-project", "-pluginid", PLUGIN_ID,
53         "-pluginargs", args.action,
54     ]
55     started = time.time()
56     completed = subprocess.run(command, cwd=VP / "bin", text=True, encoding="utf-8", errors="replace")
57     marker = work / f"{args.action}_result.txt"
58     if completed.returncode == 0 and (not marker.exists() or marker.stat().st_mtime < started):
59         raise RuntimeError("VP command did not produce fresh verification evidence; inspect its\n        output.")
60     raise SystemExit(completed.returncode)
61
62
63 if __name__ == "__main__":
64     main()
```

B.6 Complete automated test suite

B.6.1 tests/test_data_boundaries.py

This listing contains the complete file (184 source lines).

Listing 31: tests/test_data_boundaries.py

```

1  """Invalid local data must fail at boundaries, not inside retrieval."""
2
3  from copy import deepcopy
4  import json
5  from pathlib import Path
6
7  import pytest
8
9  from application.order_service import OrderService
10 from application.query_service import QueryService
11 from application.ranking_service import RankingService
12 from data.order_repository import OrderRepository
13 from data.product_repository import ProductRepository
14 from data.vector_index import VectorIndex, cosine_similarity
15 from main import build_services
16 from scripts.prepare_dataset import prepare
17
18 ROOT = Path(__file__).resolve().parents[1]
19
20
21 def write(path, value):
22     path.write_text(json.dumps(value), encoding="utf-8")
23     return path
24
25
26 @pytest.mark.parametrize(
27     "mutation",
28     [
29         "empty",
30         "missing",
31         "duplicate",
32         "price_nan",
33         "price_negative",
34         "stock_bool",
35         "name_empty",
36         "category_wrong",
37         "image_missing",
38         "id_bool",
39     ],
40 )
41 def test_product_schema_rejects_invalid_data(tmp_path, mutation):
42     products = json.loads((ROOT / "data/products.json").read_text())
43     if mutation == "empty":
44         products = []
45     elif mutation == "missing":
46         del products[0]["name"]
47     elif mutation == "duplicate":
48         products[1]["product_id"] = products[0]["product_id"]
49     else:
50         key, value = {
51             "price_nan": ("price", float("nan")),
52             "price_negative": ("price", -1),
53             "stock_bool": ("stock", True),
54             "name_empty": ("name", ""),
55             "category_wrong": ("category", "food"),
56             "image_missing": ("image", "data/images/missing.png"),

```

```

57         "id_bool": ("product_id", True),
58     }[mutation]
59     products[0][key] = value
60     with pytest.raises(ValueError):
61         ProductRepository(write(tmp_path / "products.json", products), ROOT)
62
63
64 def test_product_load_error(tmp_path):
65     with pytest.raises(ValueError):
66         ProductRepository(tmp_path / "missing.json")
67
68
69 @pytest.mark.parametrize("mutation", ["not_list", "missing", "duplicate", "blank", "negative"])
70 def test_order_schema_rejects_invalid_data(tmp_path, mutation):
71     orders = json.loads((ROOT / "data/orders.json").read_text())
72     if mutation == "not_list":
73         orders = {}
74     elif mutation == "missing":
75         del orders[0]["status"]
76     elif mutation == "duplicate":
77         orders[1]["order_id"] = orders[0]["order_id"]
78     elif mutation == "blank":
79         orders[0]["customer_id"] = ""
80     else:
81         orders[0]["total"] = -1
82     with pytest.raises(ValueError):
83         OrderRepository(write(tmp_path / "orders.json", orders))
84
85
86 def test_order_bad_id_and_load_errors(tmp_path):
87     with pytest.raises(ValueError):
88         OrderRepository(tmp_path / "missing.json")
89     with pytest.raises(ValueError):
90         OrderService(OrderRepository()).find_order("invalid", "C001")
91
92
93 @pytest.mark.parametrize("mutation", ["encoder", "dimension", "ids", "fingerprint", "missing"])
94 def test_index_consistency_and_stale_detection(tmp_path, mutation):
95     payload = json.loads((ROOT / "data/embeddings.json").read_text())
96     if mutation == "encoder":
97         payload["encoder"]["version"] = "unknown"
98     elif mutation == "dimension":
99         payload["vectors"]["1"] = [1, 2]
100     elif mutation == "ids":
101         del payload["vectors"]["1"]
102     elif mutation == "fingerprint":
103         payload["source_sha256"] = "stale"
104     else:
105         del payload["encoder"]
106     with pytest.raises(ValueError):
107         VectorIndex(write(tmp_path / "index.json", payload), ProductRepository().all_products(), ROOT)
108
109
110 def test_invalid_cosine_and_index_query():
111     with pytest.raises(ValueError):
112         cosine_similarity(["nonnumeric"], [1])
113     with pytest.raises(ValueError):
114         VectorIndex().score([1])
115
116
117 @pytest.mark.parametrize(
118     "options",
119     [{"category": "food"}, {"max_price": -1}, {"max_price": float("inf")}, {"max_price": True},
120     {"category": "bag"}],
121 )
122 def test_query_filter_boundary(options):

```

```

122     with pytest.raises(ValueError):
123         QueryService().text_query("shoes", **options)
124
125
126 def test_query_price_minimum_and_stopword_only():
127     assert QueryService().text_query("shoes under 100 below 80")["filters"]["max_price_exclusive"] == 80
128     assert QueryService().text_query("shoes", max_price=60)["filters"]["max_price_exclusive"] == 60
129     with pytest.raises(ValueError):
130         QueryService().text_query("find me please")
131     with pytest.raises(ValueError):
132         QueryService().image_query(["nonnumeric"] * 88)
133
134
135 def test_extracted_price_overflow_is_rejected():
136     with pytest.raises(ValueError, match="finite"):
137         QueryService().text_query("shoes under " + "9" * 400)
138
139
140 def test_dataset_generator_rejects_paths_outside_root(tmp_path):
141     products = json.loads((ROOT / "data/products.json").read_text())
142     products[0]["image"] = "../escaped.png"
143     catalogue = tmp_path / "data/products.json"
144     catalogue.parent.mkdir()
145     write(catalogue, products)
146     with pytest.raises(ValueError, match="inside"):
147         prepare(tmp_path)
148     assert not (tmp_path.parent / "escaped.png").exists()
149
150
151 def test_ranking_boundary_and_no_hidden_stock_signal():
152     ranker = RankingService()
153     q = QueryService().text_query("backpack")
154     candidates = [
155         dict(product_id=12, stock=0, text_score=1, image_score=0),
156         dict(product_id=9, stock=10, text_score=0.5, image_score=0),
157     ]
158     assert ranker.rank(candidates, q)[0]["product_id"] == 12
159     original = deepcopy(candidates)
160     q["weights"] = {"text": 0.5, "image": 0.6}
161     with pytest.raises(ValueError):
162         ranker.rank(candidates, q)
163     assert candidates == original
164
165
166 def test_search_rejects_unsupported_contract():
167     ui = build_services()
168     with pytest.raises(ValueError):
169         ui.search_service.search({"type": "unknown"})
170     with pytest.raises(ValueError):
171         ui.view_product(-1)
172
173
174 def test_dataset_regeneration_preserves_customized_files(tmp_path):
175     prepare(tmp_path)
176     catalogue = tmp_path / "data/products.json"
177     products = json.loads(catalogue.read_text())
178     products[0]["price"] = 123
179     write(catalogue, products)
180     image = tmp_path / "data/images/p001.png"
181     before = image.read_bytes()
182     prepare(tmp_path)
183     assert json.loads(catalogue.read_text())[0]["price"] == 123
184     assert image.read_bytes() == before

```

B.6.2 tests/test_evaluation.py

This listing contains the complete file (11 source lines).

Listing 32: tests/test_evaluation.py

```

1  """Metrics are auditable and use the declared nearest-rank percentile."""
2
3  import math
4
5  from scripts import run_evaluation
6
7
8  def test_p95_uses_nearest_rank():
9      times = list(range(1, 13))
10     expected = times[math.ceil(0.95 * len(times)) - 1]
11     assert run_evaluation.percentile_nearest_rank(times, 0.95) == expected

```

B.6.3 tests/test_prototype.py

This listing contains the complete file (242 source lines).

Listing 33: tests/test_prototype.py

```

1  """Acceptance and boundary tests written before the prototype implementation."""
2
3  from copy import deepcopy
4  import importlib
5  import json
6  from pathlib import Path
7  import subprocess
8  import sys
9
10 import numpy as np
11 import pytest
12
13 ROOT = Path(__file__).resolve().parents[1]
14 sys.path.insert(0, str(ROOT))
15
16
17 @pytest.fixture
18 def ui():
19     return importlib.import_module("main").build_services(ROOT)
20
21
22 @pytest.fixture
23 def qs():
24     return importlib.import_module("application.query_service").QueryService()
25
26
27 def test_text_normalizes_alias_punctuation_and_unique_terms(qs):
28     q = qs.text_query(" FIND black, BLACK shoe! ")
29     assert q["tokens"] == ["black", "shoes"]
30     assert q["filters"]["category"] == "shoes"
31
32
33 @pytest.mark.parametrize("value", [",", " ", None, 32, "a" * 2001])
34 def test_text_rejects_invalid_input(qs, value):
35     with pytest.raises(ValueError):
36         qs.text_query(value)

```

```

37
38
39 @pytest.mark.parametrize("value", [0, -1, True, 101, 2.5])
40 def test_top_k_boundary(qs, value):
41     with pytest.raises(ValueError):
42         qs.text_query("shoes", top_k=value)
43
44
45 def test_price_strict_vs_inclusive(qs):
46     strict = qs.text_query("shoes under 100 dollars")
47     inclusive = qs.text_query("shoes at most 100 dollars")
48     assert strict["filters"]["max_price_exclusive"] == 100
49     assert inclusive["filters"]["max_price_inclusive"] == 100
50
51
52 @pytest.mark.parametrize("embedding", [[0] * 87, [float("nan")] * 88, [float("inf")] * 88, "wrong", [[1]
53 * 88]])
54 def test_embedding_validation(qs, embedding):
55     with pytest.raises(ValueError):
56         qs.image_query(embedding)
57
58
59 @pytest.mark.parametrize("weight", [-0.1, 1.1, float("nan"), True])
60 def test_fusion_weight_validation(qs, weight):
61     with pytest.raises(ValueError):
62         qs.multimodal_query("shoes", [1] * 88, text_weight=weight)
63
64
65 def test_dataset_and_repository_are_immutable(ui):
66     products = ui.search_service.repository.all_products()
67     assert len(products) == 12
68     assert len({p["product_id"] for p in products}) == 12
69     assert all((ROOT / p["image"]).is_file() for p in products)
70     original = deepcopy(products)
71     products[0]["name"] = "tampered"
72     ui.search_text("black shoes")
73     assert ui.search_service.repository.all_products() == original
74
75
76 def test_text_filters_relevance_and_no_matches(ui):
77     result = ui.search_text("find Nike shoes under 100 dollars")
78     assert result["results"][0]["product_id"] == 5
79     assert all(p["price"] < 100 and p["category"] == "shoes" for p in result["results"])
80     assert ui.search_text("unobtainium")["results"] == []
81
82
83 def test_filter_only_and_inclusive_price(ui):
84     strict = ui.search_text("under 100", category="shoes")
85     inclusive = ui.search_text("at most 100", category="shoes")
86     assert all(r["price"] < 100 for r in strict["results"])
87     assert 2 in [r["product_id"] for r in inclusive["results"]]
88
89
90 def test_voice_calls_speech_adapter_and_matches_text(ui, monkeypatch):
91     calls = []
92     monkeypatch.setattr(ui.speech_service, "transcribe", lambda s: calls.append(s) or s)
93     voice = ui.search_voice("black running shoes")
94     text = ui.search_text("black running shoes")
95     assert calls == ["black running shoes"]
96     assert voice["query"]["type"] == "voice"
97     assert voice["results"] == text["results"]
98
99
100 def test_speech_rejects_real_audio_and_empty():
101     speech = importlib.import_module("application.speech_service").SpeechService()
102     for value in [b"audio", "", None]:

```

```

102         with pytest.raises(ValueError):
103             speech.transcribe(value)
104         assert speech.transcribe(" find shoes ") == "find shoes"
105
106
107 def test_pixel_encoder_is_path_independent_and_dimension_88(ui, tmp_path):
108     from PIL import Image
109
110     source = ROOT / "data/images/p001.png"
111     copied = tmp_path / "unrelated_name.png"
112     copied.write_bytes(source.read_bytes())
113     encoded = ui.image_service.encode(source)
114     assert len(encoded) == 88
115     assert np.linalg.norm(encoded) == pytest.approx(1)
116     assert encoded == ui.image_service.encode(copied)
117     Image.new("RGB", (128, 128), "red").save(copied)
118     assert encoded != ui.image_service.encode(copied)
119
120
121 def test_image_missing_and_corrupt_are_clear_errors(ui, tmp_path):
122     invalid = tmp_path / "broken.png"
123     invalid.write_text("not an image")
124     for path in [invalid, tmp_path / "missing.png"]:
125         with pytest.raises(ValueError):
126             ui.search_image(path)
127
128
129 def test_image_pixel_limit(ui, tmp_path):
130     from PIL import Image
131
132     path = tmp_path / "oversized.png"
133     Image.new("RGB", (5000, 4001), "white").save(path)
134     with pytest.raises(ValueError, match="20 million"):
135         ui.image_service.encode(path)
136
137
138 def test_cosine_boundaries():
139     cosine = importlib.import_module("data.vector_index").cosine_similarity
140     assert cosine([1, 0], [1, 0]) == pytest.approx(1)
141     assert cosine([1, 0], [0, 1]) == pytest.approx(0)
142     assert cosine([0, 0], [1, 1]) == 0
143     assert cosine([-1, 0], [1, 0]) == -1
144     for a, b in [( [1], [1, 2]), ([float("nan")], [1])]:
145         with pytest.raises(ValueError):
146             cosine(a, b)
147
148
149 def test_image_exact_and_transformed_samples(ui):
150     exact = ui.search_image(ROOT / "data/images/p001.png")
151     query = ui.search_image(ROOT / "data/queries/black_shoe_query.png")
152     assert exact["results"][0]["product_id"] == 1
153     assert exact["results"][0]["image_score"] == pytest.approx(1)
154     assert query["results"][0]["product_id"] in {1, 5}
155
156
157 def test_retrieval_separate_from_ranking_and_tie_break(ui, qs):
158     q = qs.text_query("black shoes", top_k=1)
159     candidates = ui.search_service.retrieve_candidates(q)
160     assert len(candidates) > 1
161     assert "rank" not in candidates[0]
162     ranked = ui.search_service.ranking.rank(list(reversed(candidates)), q)
163     assert len(ranked) == 1
164     assert ranked[0]["product_id"] == 1
165
166
167 def test_fusion_has_union_candidates_and_component_formula(ui, qs):

```

```

168     image = ui.image_service.encode(ROOT / "data/images/p003.png")
169     q = qs.multimodal_query("blue", image, top_k=12)
170     candidates = ui.search_service.retrieve_candidates(q)
171     assert {3, 4} <= {c["product_id"] for c in candidates}
172     result = ui.search_service.search(q)
173     for item in result["results"]:
174         assert item["final_score"] == pytest.approx(0.5 * item["text_score"] + 0.5 *
175             item["image_score"])
176     text_only = ui.search_service.search(qs.multimodal_query("blue", image, text_weight=1))
177     image_only = ui.search_service.search(qs.multimodal_query("blue", image, text_weight=0))
178     assert text_only["results"][0]["product_id"] == 4
179     assert image_only["results"][0]["product_id"] == 3
180
181 def test_order_customer_scope(ui):
182     assert ui.search_order("0001", "C001")["order"]["status"] == "Shipped"
183     assert ui.search_order("0002", "C001")["order"] is None
184     assert ui.search_order("0999", "C001")["order"] is None
185     with pytest.raises(ValueError):
186         ui.search_order("0001", "")
187
188
189 def test_product_detail(ui):
190     assert ui.view_product(1)["product"]["name"] == "Nike Running Shoes"
191     assert ui.view_product(999)["product"] is None
192
193
194 @pytest.mark.parametrize(
195     "args",
196     [
197         [],
198         ["--mode", "text", "--query", "black shoes"],
199         ["--mode", "voice", "--query", "find running shoes"],
200         ["--mode", "image", "--image", "data/queries/black_shoe_query.png"],
201         ["--mode", "multimodal", "--query", "black shoes", "--image",
202             "data/queries/black_shoe_query.png"],
203         ["--mode", "order", "--order-id", "0001", "--customer-id", "C001"],
204     ],
205 )
206 def test_cli_end_to_end_json(args, tmp_path):
207     output = tmp_path / "result.json"
208     completed = subprocess.run(
209         [sys.executable, str(ROOT / "main.py"), *args, "--json", "--output", str(output)],
210         cwd=tmp_path,
211         capture_output=True,
212         text=True,
213     )
214     assert completed.returncode == 0, completed.stderr
215     payload = json.loads(completed.stdout)
216     assert payload == json.loads(output.read_text(encoding="utf-8"))
217     if isinstance(payload, list):
218         assert {"text", "voice", "image"} <= {r.get("query", {}).get("type") for r in payload}
219     elif "results" in payload:
220         assert {"query", "processing", "results"} <= payload.keys()
221         assert payload["results"][0]["final_score"] >= 0
222
223 def test_cli_invalid_input_without_traceback():
224     completed = subprocess.run(
225         [sys.executable, str(ROOT / "main.py"), "--mode", "text", "--query", ""], capture_output=True,
226         text=True
227     )
228     assert completed.returncode != 0
229     assert "Traceback" not in completed.stderr
230

```

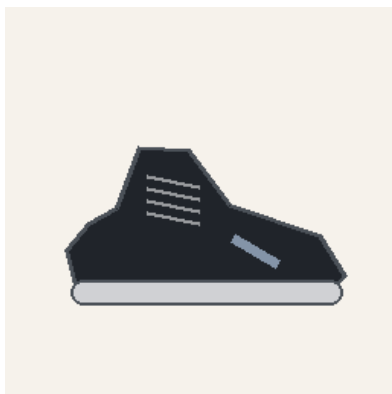
```
231 def test_architecture_import_boundaries():
232     import ast
233
234     forbidden = {"presentation": {"data"}, "data": {"application", "presentation"}, "application":
{"presentation"}}
235     for package, blocked in forbidden.items():
236         files = list((ROOT / package).glob("*.py"))
237         assert files
238         for file in files:
239             tree = ast.parse(file.read_text(encoding="utf-8"))
240             imports = [node.module for node in ast.walk(tree) if isinstance(node, ast.ImportFrom)]
241             imports += [alias.name for node in ast.walk(tree) if isinstance(node, ast.Import) for alias
in node.names]
242             assert not {name.split(".")[0] for name in imports if name} & blocked
```

C Dataset and Approved Image Catalogue

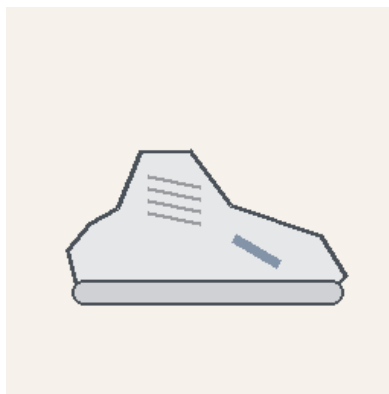
The catalogue has 12 products in shoes, bag and clothing categories. Prices are USD and stock is metadata, not a boost. The table records every product; full descriptions, brands and image paths are reproduced in Appendix D.

ID	Product	Category / color	USD	Stock
1	Nike Running Shoes	shoes / black	120.00	10
2	Adidas Running Shoes	shoes / white	100.00	15
3	Black Leather Bag	bag / black	80.00	20
4	Blue Sports Shoes	shoes / blue	75.00	12
5	Nike Budget Running Shoes	shoes / black	95.00	8
6	Red Canvas Shoes	shoes / red	60.00	18
7	White Casual Shoes	shoes / white	45.00	20
8	Brown Leather Bag	bag / brown	90.00	6
9	Black Backpack	bag / black	55.00	10
10	Blue Cotton T-Shirt	clothing / blue	25.00	30
11	Black Sports Jacket	clothing / black	85.00	7
12	Green Travel Backpack	bag / green	65.00	0

Order	Customer	Date	Status	Total USD
O001	C001	2026-09-29	Shipped	150.00
O002	C002	2026-09-28	Delivered	80.00
O003	C001	2026-10-01	Processing	95.00



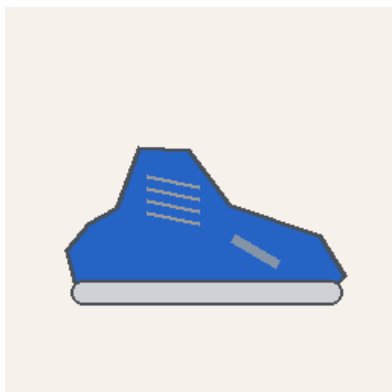
01. Nike Running Shoes



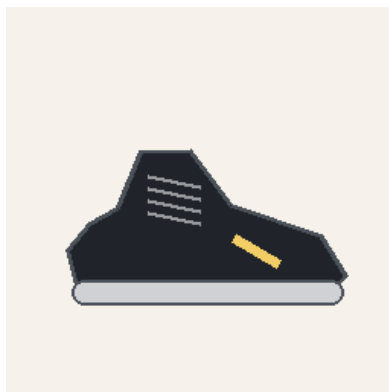
02. Adidas Running Shoes



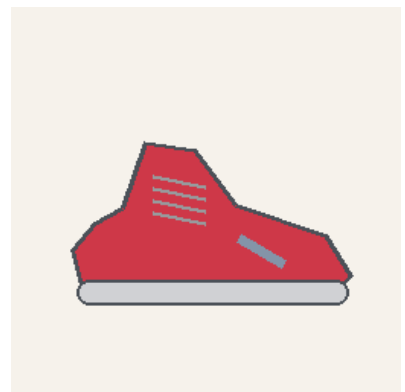
03. Black Leather Bag



04. Blue Sports Shoes

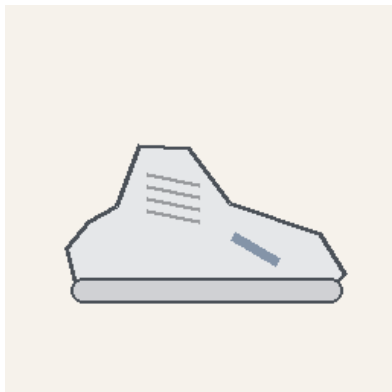


05. Nike Budget Running Shoes



06. Red Canvas Shoes

Figure 11: Original synthetic catalogue illustrations, products 1–6. Images are preserved from the approved dataset.



07. White Casual Shoes



08. Brown Leather Bag



09. Black Backpack



10. Blue Cotton T-Shirt

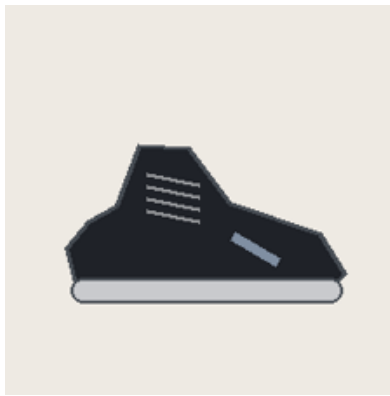


11. Black Sports Jacket

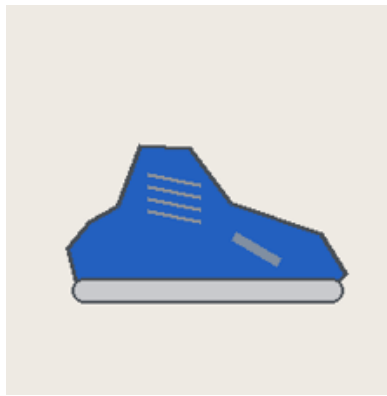


12. Green Travel Backpack

Figure 12: Original synthetic catalogue illustrations, products 7–12. Images are preserved from the approved dataset.



Black-shoe query



Blue-shoe query



Brown-bag query

Figure 13: All three readable transformed image-query fixtures. Brightness and resolution differ from their source illustration; these are derived synthetic samples, not independent photographs.

`data/queries/corrupt.png` deliberately contains invalid image bytes. It is included as a robustness fixture in the source package and is intentionally not embedded as a figure. Product vectors in `data/embeddings.json` contain 88 values per product with encoder metadata and a source fingerprint. The complete numeric index accompanies the submission; the report explains its format and reproduces the code that constructs and validates it.

D Complete Data Fixtures and Configuration

These exact files record catalogue/owner metadata, frozen benchmark judgments and pinned environment configuration. They are reproduced without rewriting their contents. JSON vector arrays remain in the delivered index file rather than adding repetitive numeric pages; every Python file that computes or consumes them is included in Appendix B.

D.1 data/products.json

Listing 34: data/products.json

```
1 [
2   {
3     "product_id": 1,
4     "name": "Nike Running Shoes",
5     "brand": "Nike",
6     "category": "shoes",
7     "color": "black",
8     "price": 120,
9     "stock": 10,
10    "description": "Black running shoes for everyday training.",
11    "image": "data/images/p001.png"
12  },
13  {
14    "product_id": 2,
15    "name": "Adidas Running Shoes",
16    "brand": "Adidas",
17    "category": "shoes",
18    "color": "white",
19    "price": 100,
20    "stock": 15,
21    "description": "White running shoes for daily exercise.",
22    "image": "data/images/p002.png"
23  },
24  {
25    "product_id": 3,
26    "name": "Black Leather Bag",
27    "brand": "Local",
28    "category": "bag",
29    "color": "black",
30    "price": 80,
31    "stock": 20,
32    "description": "Black leather shoulder bag with a carry handle.",
33    "image": "data/images/p003.png"
34  },
35  {
36    "product_id": 4,
37    "name": "Blue Sports Shoes",
38    "brand": "Local",
39    "category": "shoes",
40    "color": "blue",
41    "price": 75,
42    "stock": 12,
43    "description": "Blue sports shoes with a flexible sole.",
44    "image": "data/images/p004.png"
45  },
46  {
47    "product_id": 5,
48    "name": "Nike Budget Running Shoes",
49    "brand": "Nike",
50    "category": "shoes",
51    "color": "black",
```

```
52     "price": 95,  
53     "stock": 8,  
54     "description": "Affordable black running shoes for training.",  
55     "image": "data/images/p005.png"  
56 },  
57 {  
58     "product_id": 6,  
59     "name": "Red Canvas Shoes",  
60     "brand": "Local",  
61     "category": "shoes",  
62     "color": "red",  
63     "price": 60,  
64     "stock": 18,  
65     "description": "Red canvas shoes for casual walks.",  
66     "image": "data/images/p006.png"  
67 },  
68 {  
69     "product_id": 7,  
70     "name": "White Casual Shoes",  
71     "brand": "Local",  
72     "category": "shoes",  
73     "color": "white",  
74     "price": 45,  
75     "stock": 20,  
76     "description": "White casual shoes for everyday wear.",  
77     "image": "data/images/p007.png"  
78 },  
79 {  
80     "product_id": 8,  
81     "name": "Brown Leather Bag",  
82     "brand": "Local",  
83     "category": "bag",  
84     "color": "brown",  
85     "price": 90,  
86     "stock": 6,  
87     "description": "Brown leather shoulder bag with a carry handle.",  
88     "image": "data/images/p008.png"  
89 },  
90 {  
91     "product_id": 9,  
92     "name": "Black Backpack",  
93     "brand": "Local",  
94     "category": "bag",  
95     "color": "black",  
96     "price": 55,  
97     "stock": 10,  
98     "description": "Black backpack with a front pocket.",  
99     "image": "data/images/p009.png"  
100 },  
101 {  
102     "product_id": 10,  
103     "name": "Blue Cotton T-Shirt",  
104     "brand": "Local",  
105     "category": "clothing",  
106     "color": "blue",  
107     "price": 25,  
108     "stock": 30,  
109     "description": "Blue cotton shirt with short sleeves.",  
110     "image": "data/images/p010.png"  
111 },  
112 {  
113     "product_id": 11,  
114     "name": "Black Sports Jacket",  
115     "brand": "Local",  
116     "category": "clothing",
```

```
117     "color": "black",
118     "price": 85,
119     "stock": 7,
120     "description": "Black sports jacket with long sleeves and a zipper.",
121     "image": "data/images/p011.png"
122 },
123 {
124     "product_id": 12,
125     "name": "Green Travel Backpack",
126     "brand": "Local",
127     "category": "bag",
128     "color": "green",
129     "price": 65,
130     "stock": 0,
131     "description": "Green travel backpack with a front pocket.",
132     "image": "data/images/p012.png"
133 }
134 ]
```

D.2 data/orders.json

Listing 35: data/orders.json

```
1 [
2   {
3     "order_id": "0001",
4     "customer_id": "C001",
5     "date": "2026-09-29",
6     "status": "Shipped",
7     "total": 150.0
8   },
9   {
10    "order_id": "0002",
11    "customer_id": "C002",
12    "date": "2026-09-28",
13    "status": "Delivered",
14    "total": 80.0
15  },
16  {
17    "order_id": "0003",
18    "customer_id": "C001",
19    "date": "2026-10-01",
20    "status": "Processing",
21    "total": 95.0
22  }
23 ]
```

D.3 data/dataset_metadata.json

Listing 36: data/dataset_metadata.json

```
1 {
2   "origin": "Original synthetic Pillow illustrations authored for this assignment; not product
3     photographs.",
4   "brands": "Names are educational sample metadata; no brand affiliation implied.",
5   "generator": "scripts/prepare_dataset.py",
6   "deterministic": true,
7   "currency": "USD",
8   "products": 12,
9   "query_transform": "brightness 0.97, resize 224x224",
```

```

9 | "corrupt_fixture": "data/queries/corrupt.png intentionally invalid"
10 | }

```

D.4 evaluation/queries.json

Listing 37: evaluation/queries.json

```

1 | {
2 |   "version": "ground-truth-v1-before-implementation",
3 |   "criterion": "top-1 product belongs to relevant_ids; order exact id; robustness evaluated separately",
4 |   "queries": [
5 |     {"id": "T01", "group": "primary", "mode": "text", "query": "black shoes", "relevant_ids": [1, 5]},
6 |     {"id": "T02", "group": "primary", "mode": "text", "query": "blue shoes", "relevant_ids": [4]},
7 |     {"id": "T03", "group": "primary", "mode": "text", "query": "black leather bag", "relevant_ids": [3]},
8 |     {"id": "T04", "group": "primary", "mode": "text", "query": "find Nike shoes under 100 dollars", "relevant_ids": [5]},
9 |     {"id": "V01", "group": "primary", "mode": "voice", "query": "find running shoes", "relevant_ids": [1, 2, 5]},
10 |    {"id": "V02", "group": "primary", "mode": "voice", "query": "show me a white shoe", "relevant_ids": [2, 7]},
11 |    {"id": "V03", "group": "primary", "mode": "voice", "query": "black leather bag", "relevant_ids": [3]},
12 |    {"id": "V04", "group": "primary", "mode": "voice", "query": "green backpack", "relevant_ids": [12]},
13 |    {"id": "I01", "group": "primary", "mode": "image", "image": "data/images/p001.png", "relevant_ids": [1], "note": "Exact duplicate sanity case"},
14 |    {"id": "I02", "group": "primary", "mode": "image", "image": "data/queries/black_shoe_query.png", "relevant_ids": [1, 5], "note": "Mildly transformed independent query image"},
15 |    {"id": "I03", "group": "primary", "mode": "image", "image": "data/queries/blue_shoe_query.png", "relevant_ids": [4], "note": "Mildly transformed blue shoe"},
16 |    {"id": "I04", "group": "primary", "mode": "image", "image": "data/queries/brown_bag_query.png", "relevant_ids": [8], "note": "Mildly transformed different category"},
17 |    {"id": "F01", "group": "extension", "mode": "multimodal", "query": "black shoes", "image": "data/queries/black_shoe_query.png", "relevant_ids": [1, 5]},
18 |    {"id": "O01", "group": "extension", "mode": "order", "order_id": "O001", "customer_id": "C001", "expected_order": "O001"},
19 |    {"id": "C01", "group": "challenge", "mode": "text", "query": "sneakers", "relevant_ids": [1, 2, 4, 5, 6, 7], "note": "Unimplemented semantic synonym"},
20 |    {"id": "C02", "group": "challenge", "mode": "text", "query": "shoes not black", "relevant_ids": [2, 4, 6, 7], "note": "Negation unsupported by lexical OR matching"},
21 |    {"id": "R01", "group": "robustness", "mode": "text", "query": "unobtainium", "expected_empty": true},
22 |    {"id": "R02", "group": "robustness", "mode": "text", "query": "", "expected_error": true},
23 |    {"id": "R03", "group": "robustness", "mode": "image", "image": "data/queries/corrupt.png", "expected_error": true},
24 |    {"id": "R04", "group": "robustness", "mode": "image", "embedding_zero": true, "expected_zero_scores": true},
25 |    {"id": "R05", "group": "robustness", "mode": "order", "order_id": "O002", "customer_id": "C001", "expected_empty_order": true}
26 |   ]
27 | }

```

D.5 requirements.txt

Listing 38: requirements.txt

```

1 | numpy==2.5.3
2 | Pillow==12.3.0

```

D.6 requirements-dev.txt

Listing 39: requirements-dev.txt

```
1 -r requirements.txt
2 pytest==9.1.1
3 pytest-cov==7.1.0
4 ruff==0.16.9
5 pyright==1.1.414
6 reportlab==5.0.1
7 PyMuPDF==1.28.2
```

D.7 requirements-automation.txt

Listing 40: requirements-automation.txt

```
1 -r requirements-dev.txt
2 pyautogui==0.9.54
3 pywinauto==0.6.9
4 pyperclip==1.11.0
5 pip-audit==2.10.1
```

D.8 pyproject.toml

Listing 41: pyproject.toml

```
1 [tool.pytest.ini_options]
2 testpaths = ["tests"]
3 pythonpath = ["."]
4
5 [tool.ruff]
6 line-length = 120
7 target-version = "py312"
8
9 [tool.ruff.lint]
10 select = ["E4", "E7", "E9", "F"]
11
12 [tool.ruff.lint.per-file-ignores]
13 "scripts/*.py" = ["E402"]
14
15 [tool.pyright]
16 include = ["main.py", "application", "data", "presentation", "scripts/prepare_dataset.py", "scripts/
    build_index.py", "scripts/run_evaluation.py"]
17 typeCheckingMode = "basic"
18 pythonVersion = "3.12"
19 venvPath = "."
20 venv = ".venv"
```